

**Universidad Autónoma de Tamaulipas
Unidad Académica Multidisciplinaria Mante Centro**

Nombre del trabajo:

Simulador Webots

Asignatura:

Programación de microcontroladores

Profesor:

Daniel López Piña

Alumno:

Hernández Don Juan Neyzer Nassau

Grado: 7° Grupo: E

VERDAD, BELLEZA, PROBIDAD

Introducción

Webots es un simulador de robots de código abierto desarrollado originalmente en el EPFL (Suiza) y comercializado por Cyberbotics, utilizado en educación, investigación e industria para modelar, programar y simular robots, vehículos y sistemas mecánicos. Como parte del curso, se realizaron los ocho tutoriales oficiales de la guía de usuario de Webots, cuyo objetivo es familiarizarse progresivamente con la interfaz del programa, la creación y modificación de escenas, la programación de controladores y el uso de sensores y actuadores en un entorno simulado. El presente reporte documenta el proceso, los procedimientos realizados y los resultados obtenidos en cada uno de los tutoriales llevados a cabo en equipo.

Desarrollo

Tutorial 1: Tu primera simulación en Webots

Se realizó una introducción a la interfaz de Webots, creando un mundo nuevo desde cero y agregando un robot e-puck. Se exploró la ventana 3D, el árbol de escena (Scene Tree) y se ejecutó la simulación por primera vez, observando el comportamiento básico del robot dentro del entorno.

Tutorial 2: Modificación del entorno

Se modificó el mundo de simulación agregando objetos, paredes y obstáculos, así como ajustes al terreno y la arena donde se desplaza el robot, con el fin de crear escenarios personalizados de simulación.

Tutorial 3: Apariencia

Se aplicaron texturas, materiales y colores a los distintos objetos y robots presentes en la simulación, logrando una representación visual más realista del entorno.

Tutorial 4: Más sobre controladores

Se profundizó en la programación de controladores, implementando la lectura de sensores de proximidad y utilizando esa información para definir el comportamiento del robot, permitiéndole evitar obstáculos de forma autónoma.

Tutorial 5: Sólidos compuestos y atributos de física

Se trabajó con nodos "Solid" compuestos y se configuraron propiedades físicas como masa, fricción e inercia, con el fin de lograr un comportamiento físico más realista dentro de la simulación.

Tutorial 6: Robot de 4 ruedas

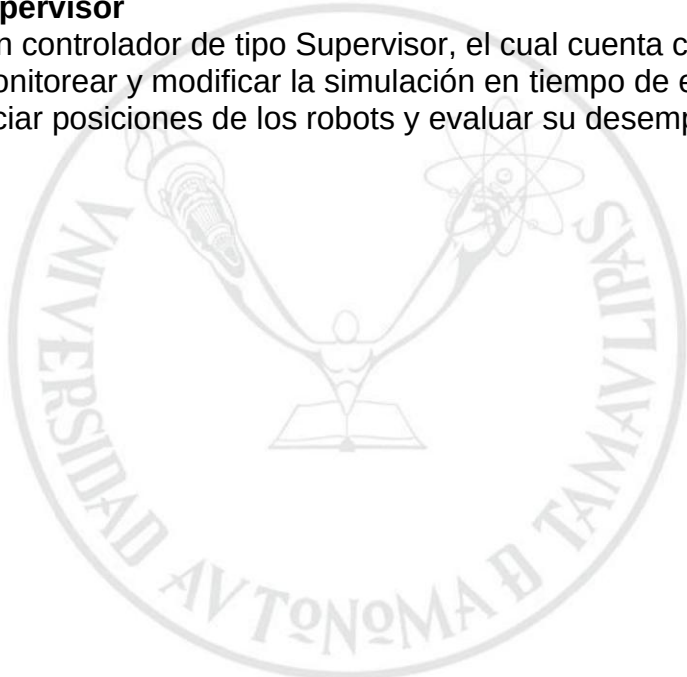
Se construyó un robot completo desde cero, compuesto por cuatro ruedas, motores diferenciales y sensores, integrando los conocimientos adquiridos en los tutoriales anteriores en un solo proyecto.

Tutorial 7: Tu primer PROTO

Se creó un nodo PROTO, es decir, un prototipo reutilizable que encapsula un objeto o robot, permitiendo su reutilización en distintos mundos de simulación sin necesidad de reconstruirlo cada vez.

Tutorial 8: El Supervisor

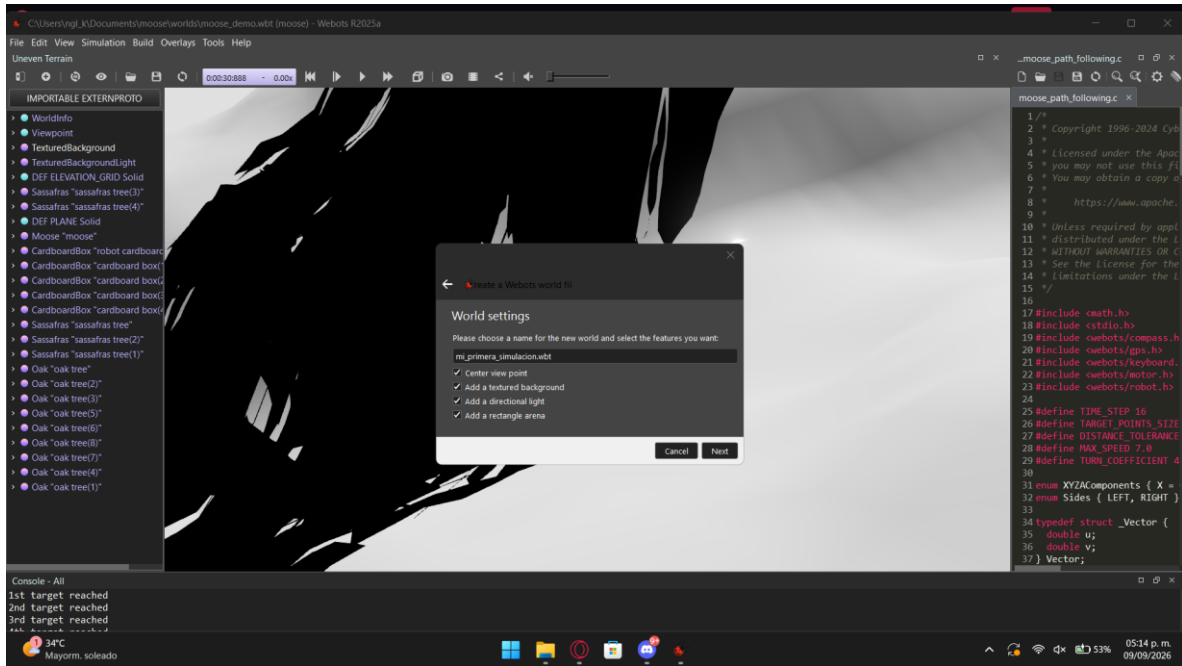
Se implementó un controlador de tipo Supervisor, el cual cuenta con permisos elevados para monitorear y modificar la simulación en tiempo de ejecución, permitiendo reiniciar posiciones de los robots y evaluar su desempeño durante la ejecución.



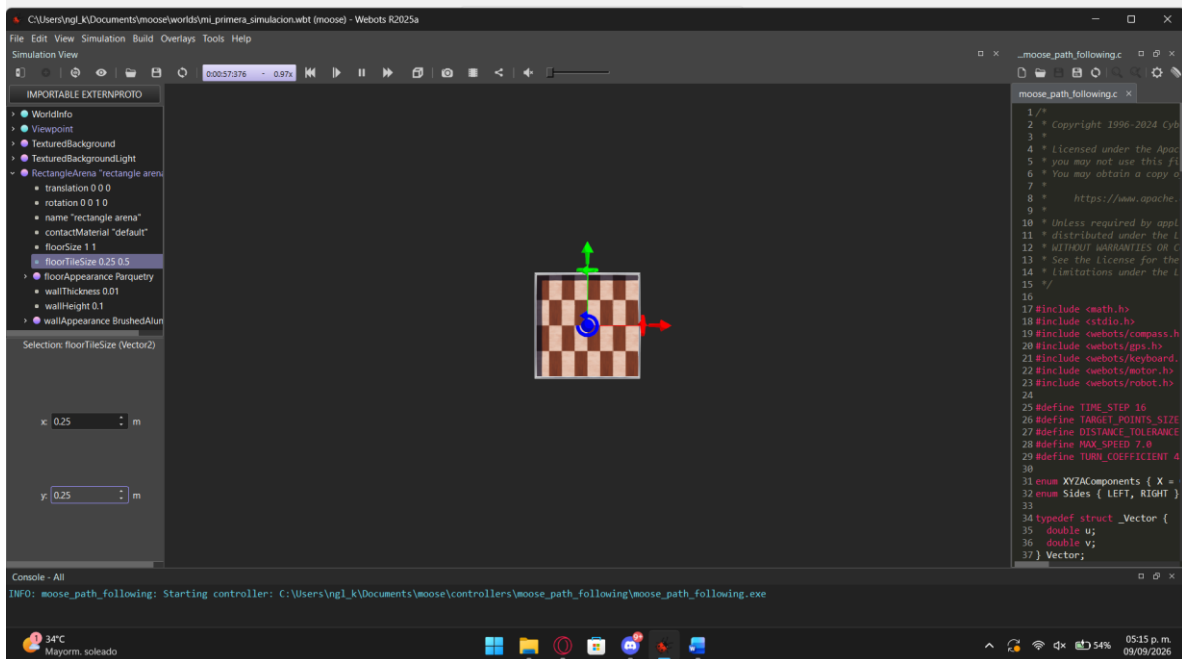
VERDAD, BELLEZA, PROBIDAD

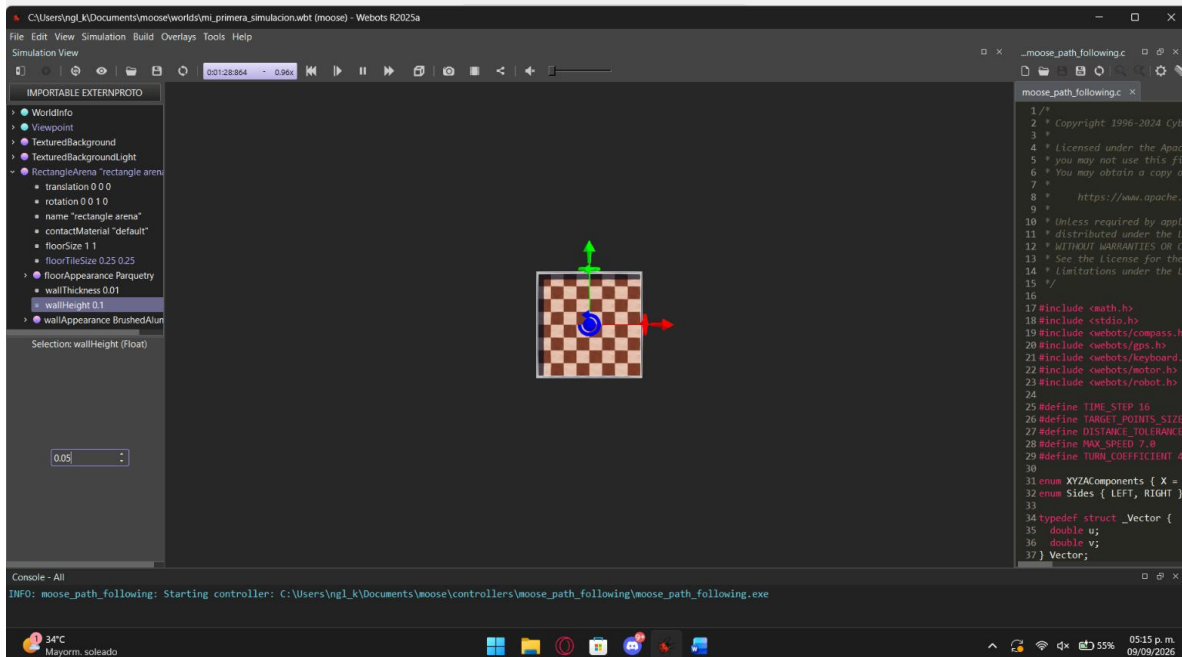
Tutorial #1

1.- Creación y configuración del mundo

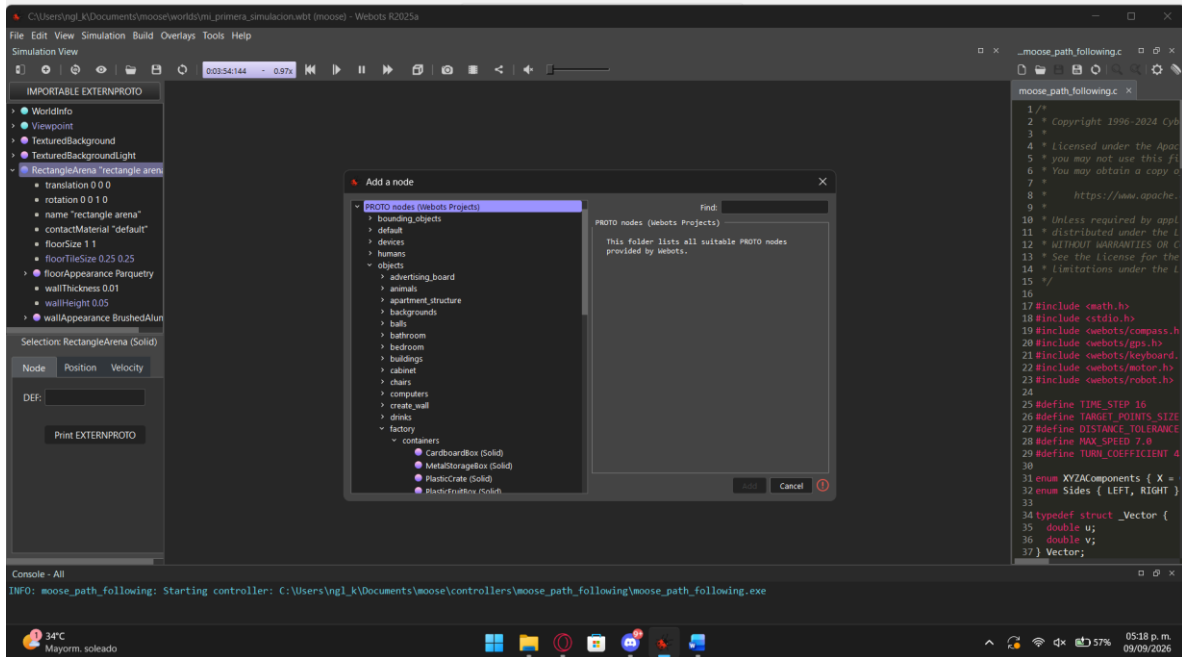


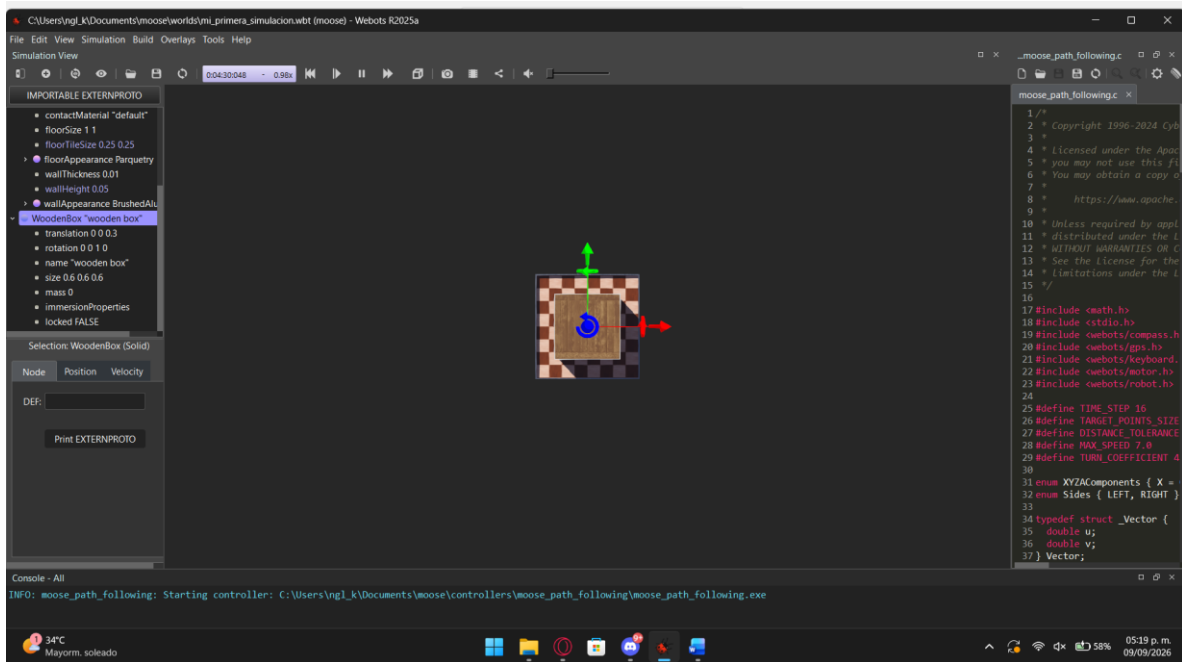
2.-Modificacion de tamaño del suelo del mundo



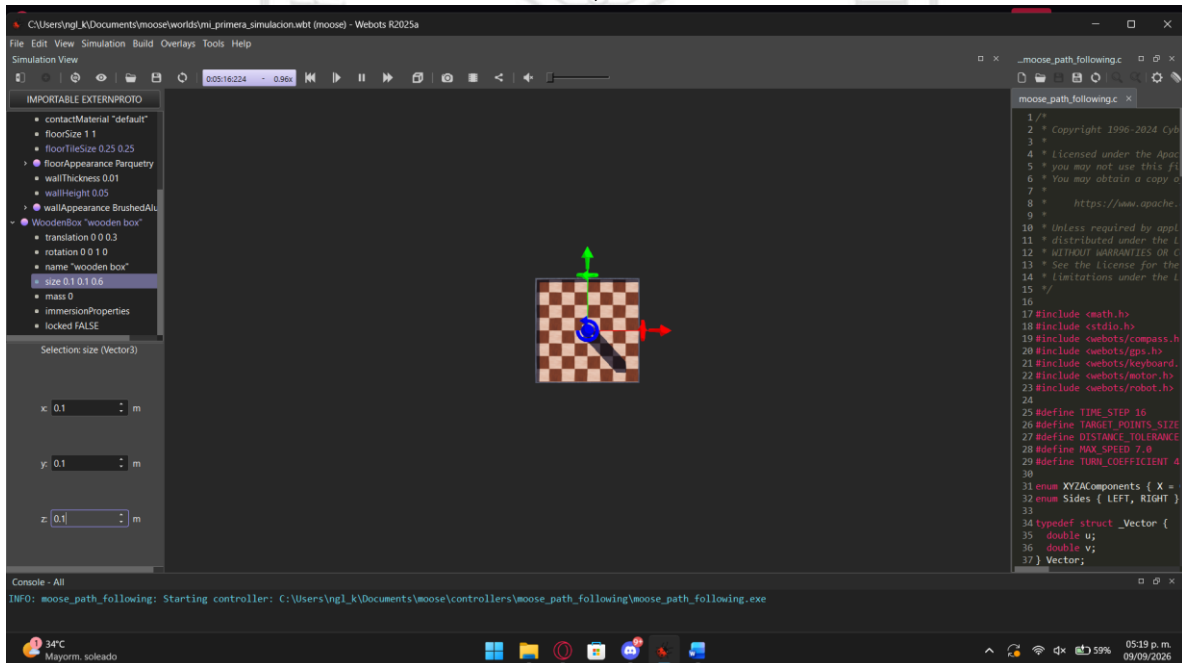


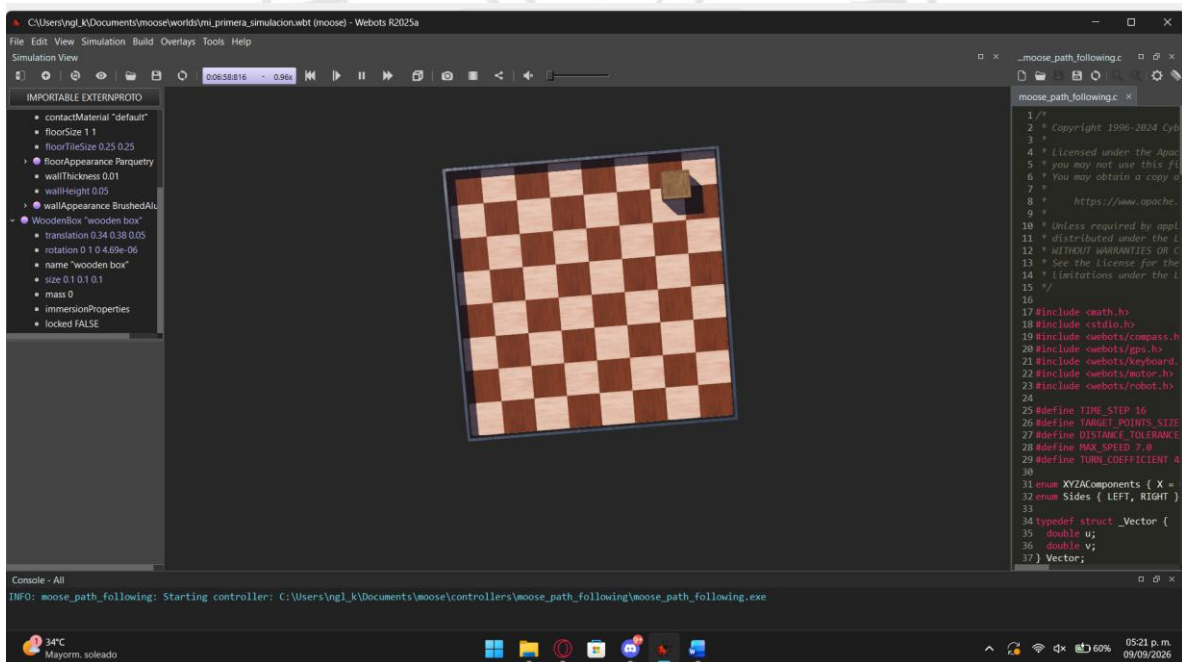
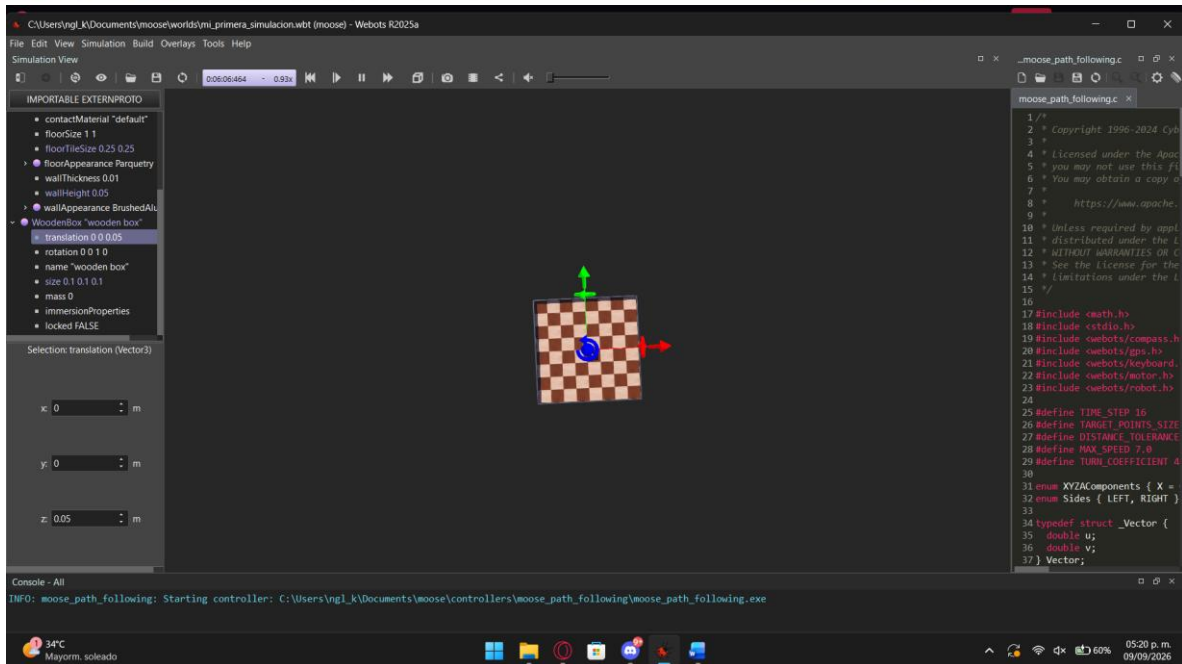
3.- Integración de obstáculos o cajas en el mundo



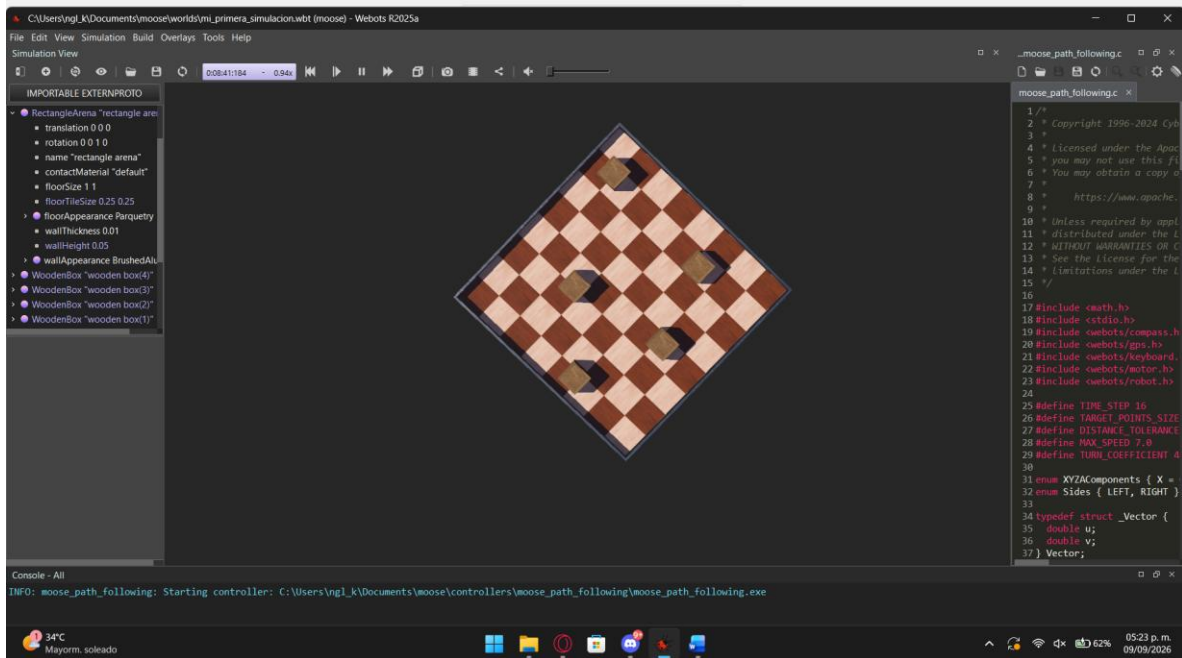


4.-Modificacion del tamaño de los obstáculos, así como también su ubicación

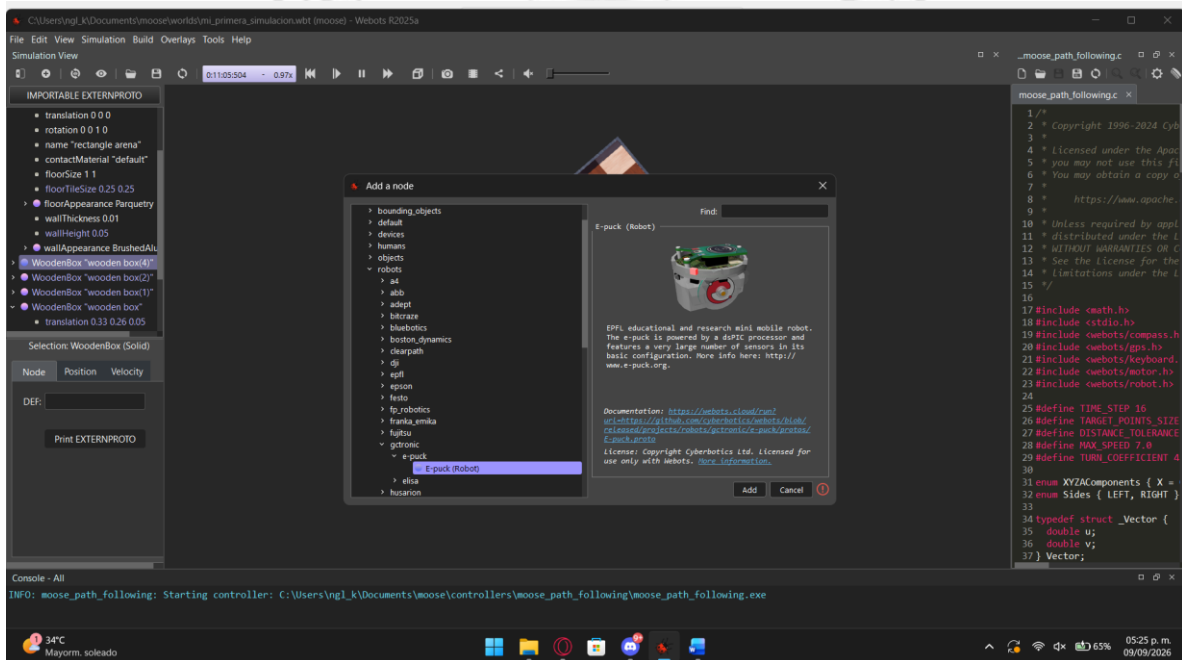


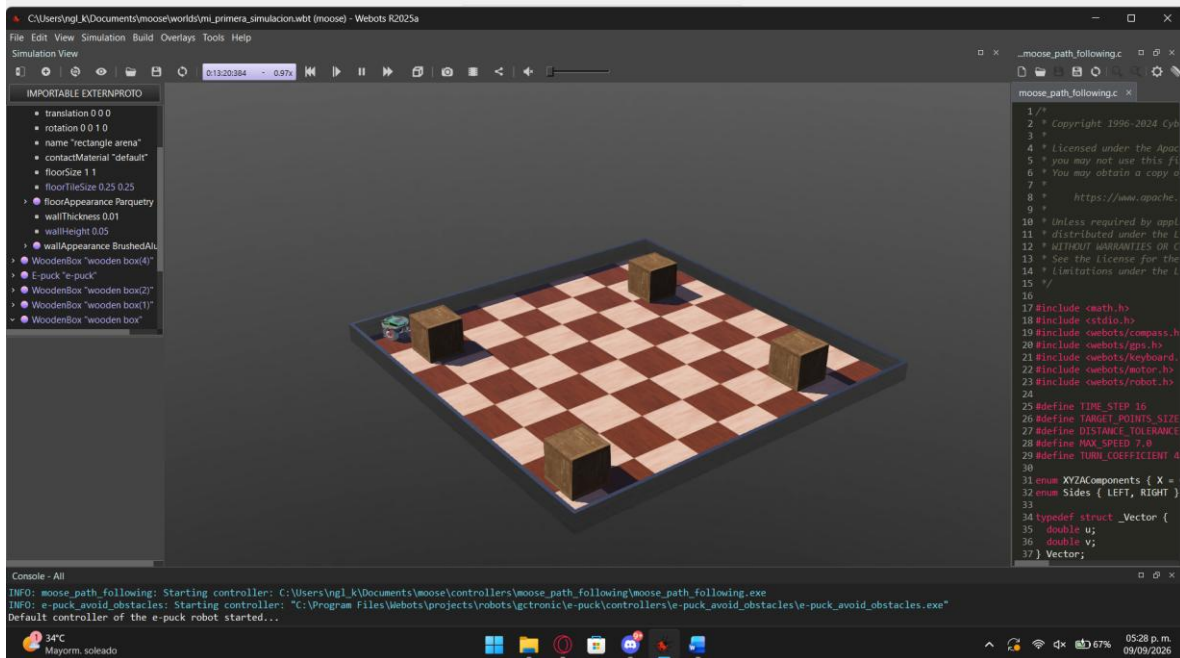


5.- Duplicar el obstáculo 4 veces mas

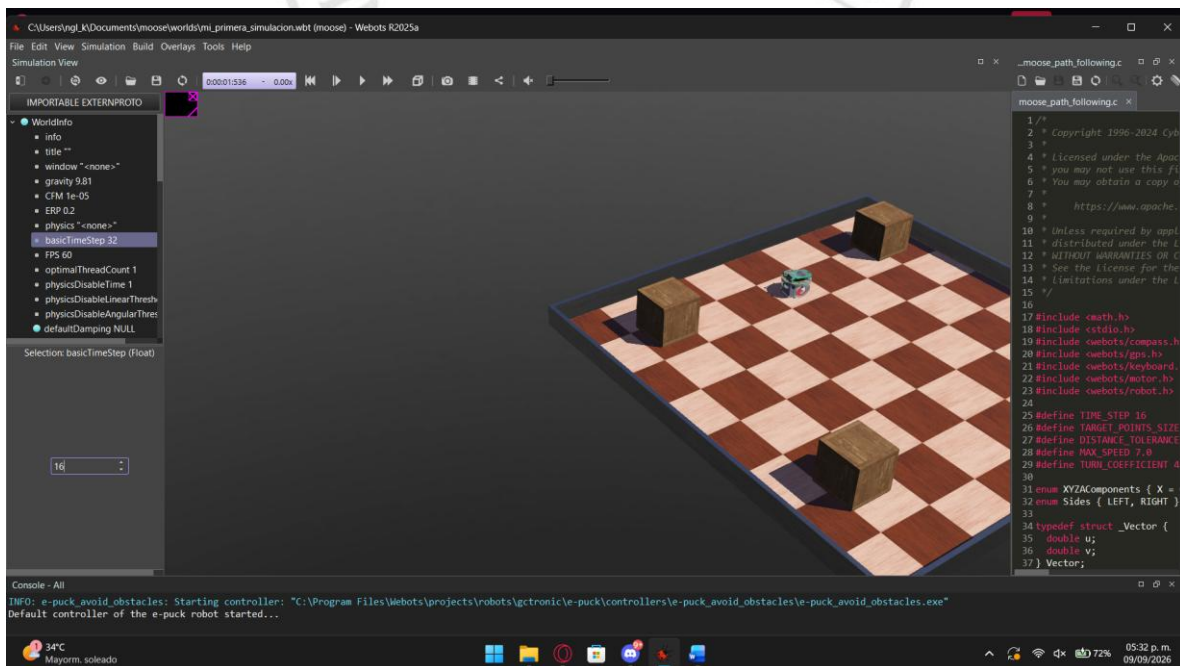


6.- Añadir robot E-puck al mundo

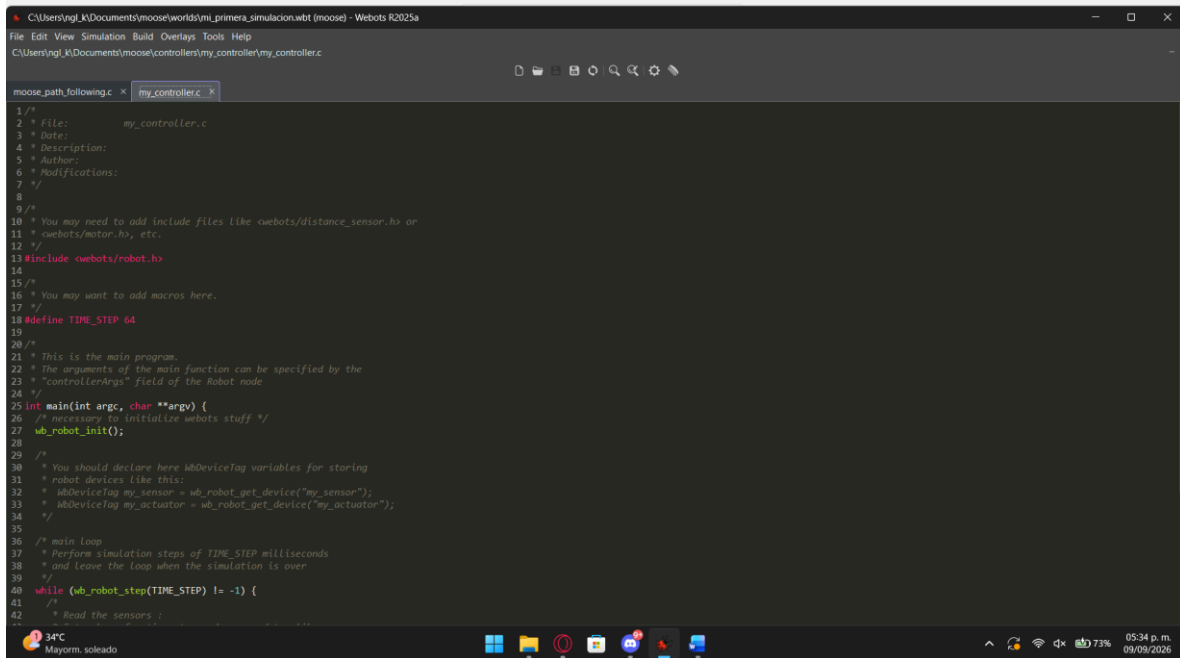




7.- Modificar basicTimeStep del robot a 16



8.- Creador archivo controlador de código tipo C para el robot E-puck

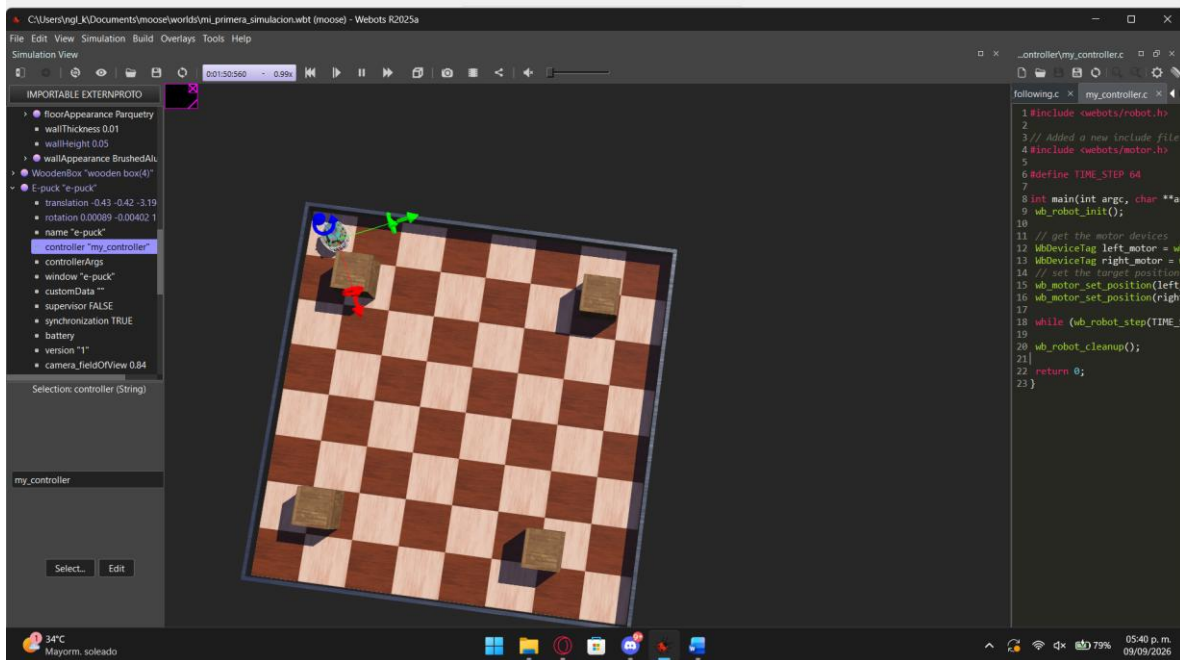


```

1 /*
2  * File:          my_controller.c
3  * Date:
4  * Description:
5  * Author:
6  * Modifications:
7  */
8
9 /*
10  * You may need to add include files like <webots/distance_sensor.h> or
11  * <webots/motor.h>, etc.
12  */
13 #include <webots/robot.h>
14
15 /*
16  * You may want to add macros here.
17  */
18 #define TIME_STEP 64
19
20 /*
21  * This is the main program.
22  * The arguments of the main function can be specified by the
23  * "controllerArgs" field of the Robot node
24  */
25 int main(int argc, char **argv) {
26     /* necessary to initialize webots stuff */
27     wb_robot_init();
28
29     /*
30      * You should declare here WbDeviceTag variables for storing
31      * robot devices like this:
32      * WbDeviceTag my_sensor = wb_robot_get_device("my_sensor");
33      * WbDeviceTag my_actuator = wb_robot_get_device("my_actuator");
34      */
35
36     /* main loop
37      * Perform simulation steps of TIME_STEP milliseconds
38      * and leave the loop when the simulation is over
39      */
40     while (wb_robot_step(TIME_STEP) != -1) {
41         /* Read the sensors:
42          * ...
43          */
44     }
45 }

```

9.- Asignar el controlador anteriormente creador a nuestro robot E-puck

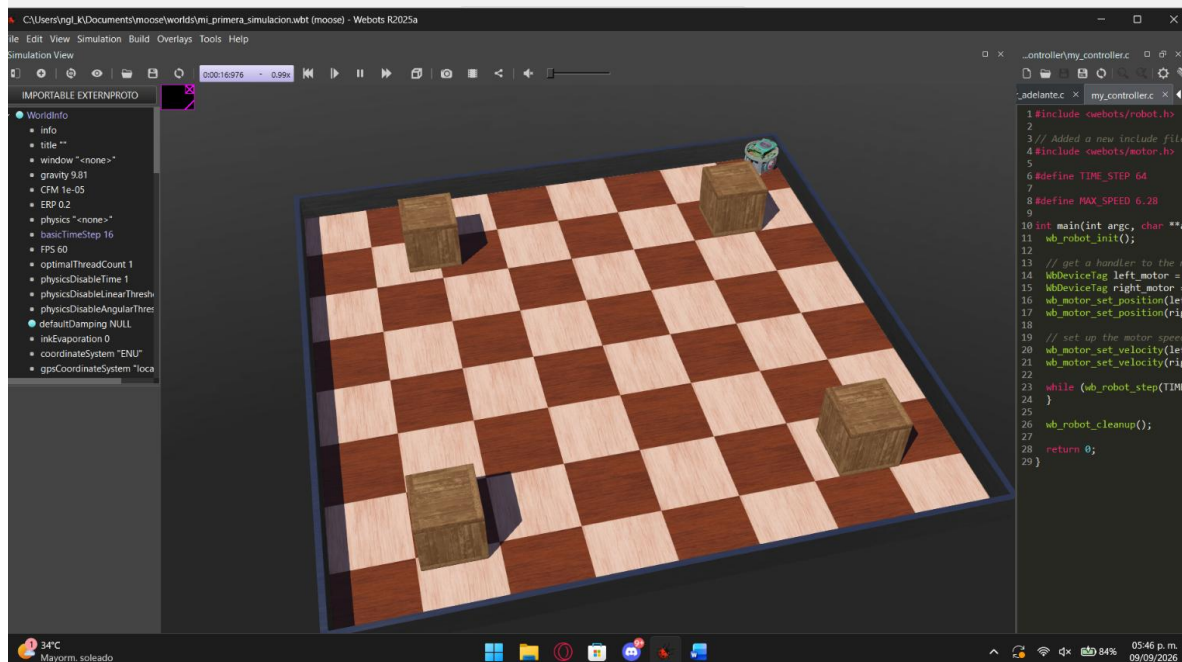


10.- Modificar código del controlador del robot

```

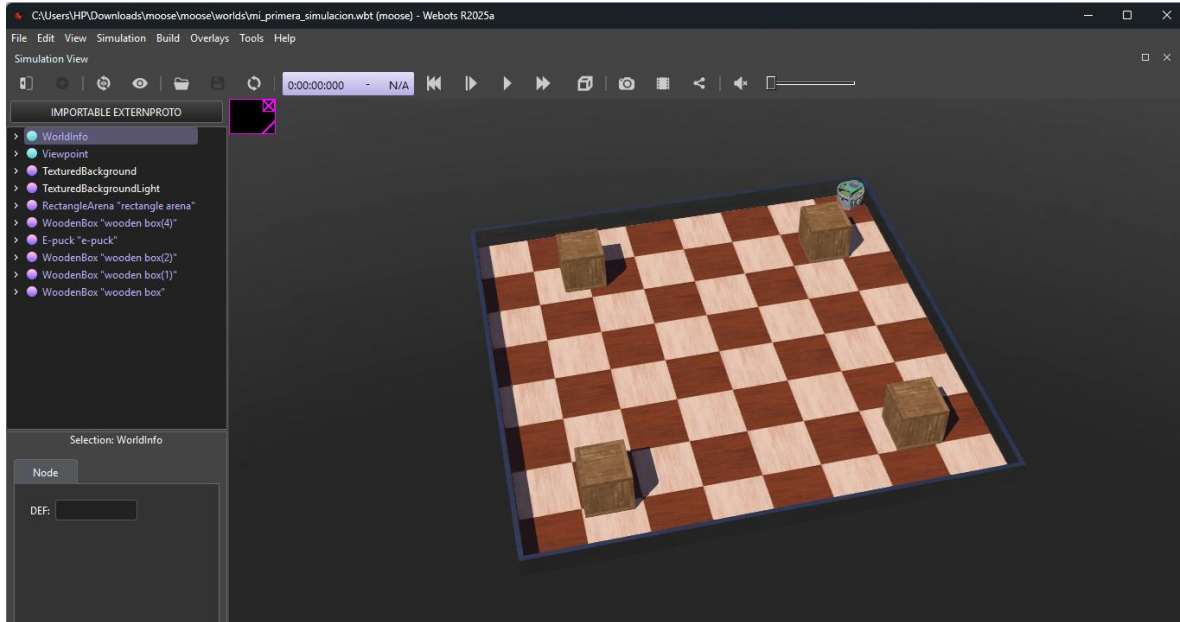
1 #include <roscpp.h>
2
3 // Added a new include file
4 #include <roscpp.h>
5
6 #define TIME_STEP 64
7
8 int main(int argc, char **argv) {
9     wb_robot_init();
10
11     // get the motor devices
12     WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
13     WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
14     // set the target position of the motors
15     wb_motor_set_position(left_motor, 10.0);
16     wb_motor_set_position(right_motor, 10.0);
17
18     while (wb_robot_step(TIME_STEP) != -1);
19
20     wb_robot_cleanup();
21
22     return 0;
23 }
    
```

Ejecución del tutorial

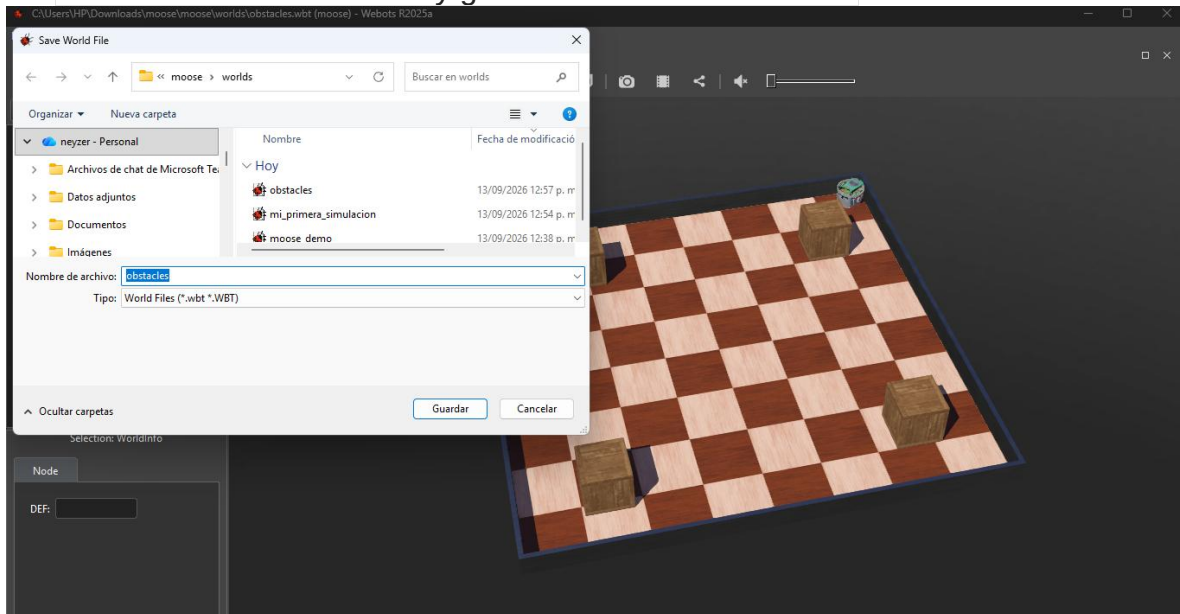


Tutorial #2

Práctica #1 Asegúrate de que my_first_simulation.wbt esté abierto
Verifica que la simulación esté pausada y en tiempo virtual 0:00:00:000 (usa el botón Reset si es necesario)



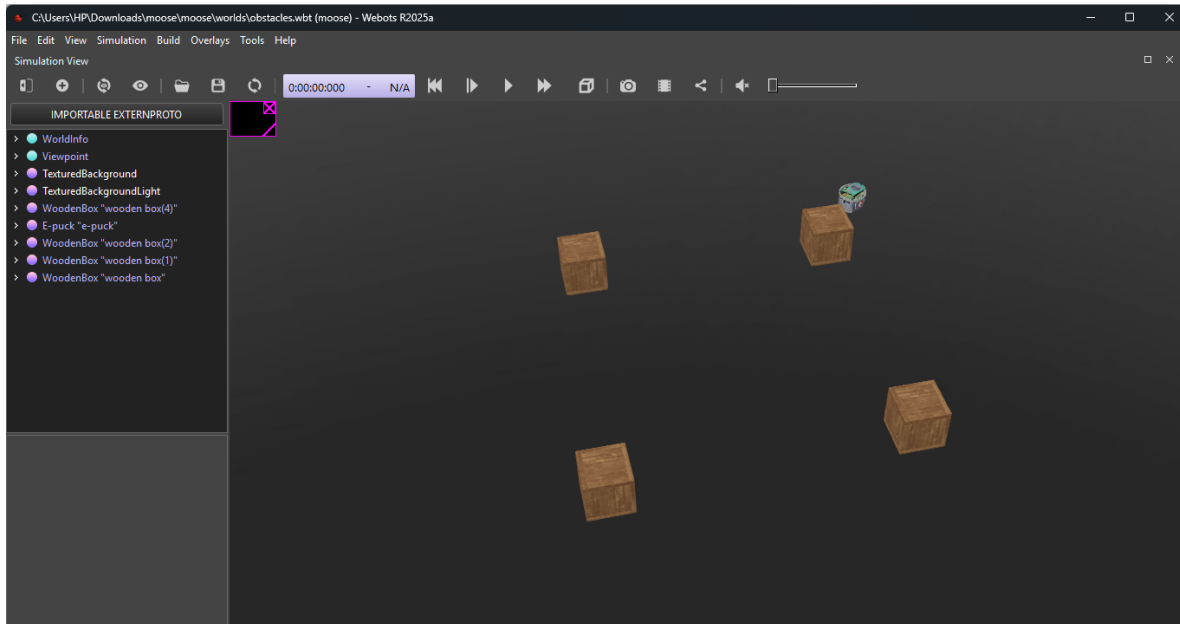
Ve a File > Save World As... y guarda como: **obstacles.wbt**



Práctica #2: Modificar el Suelo

Selecciona el nodo RectangleArena en el Scene Tree (árbol de escena)

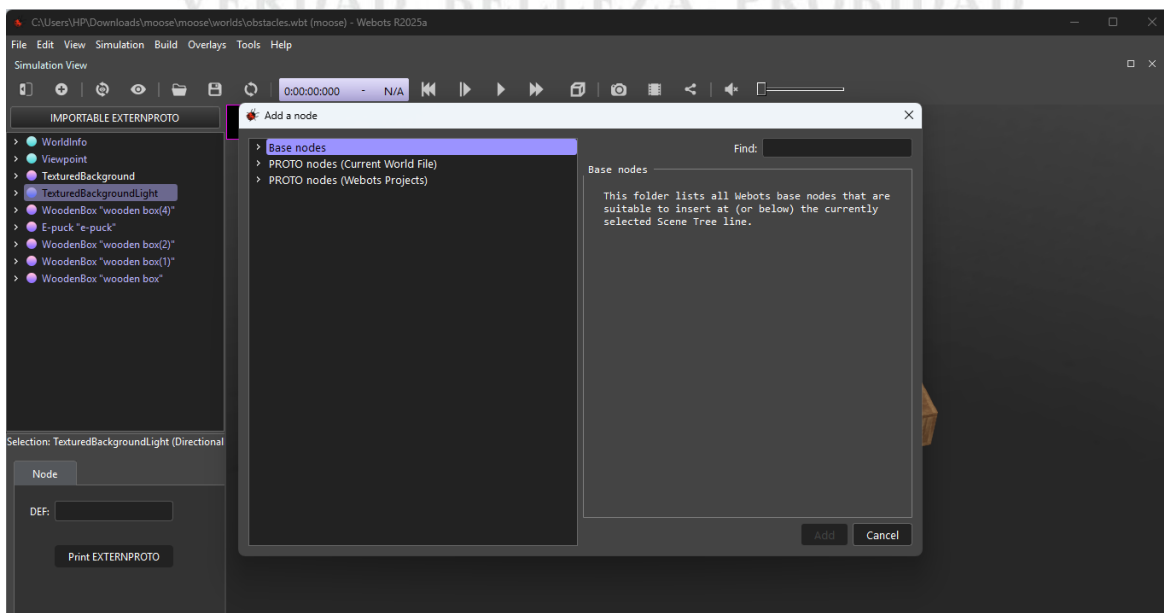
Presiona la tecla Delete en tu teclado



Añadir un Floor nuevo:

Selecciona el nodo TexturedBackgroundLight

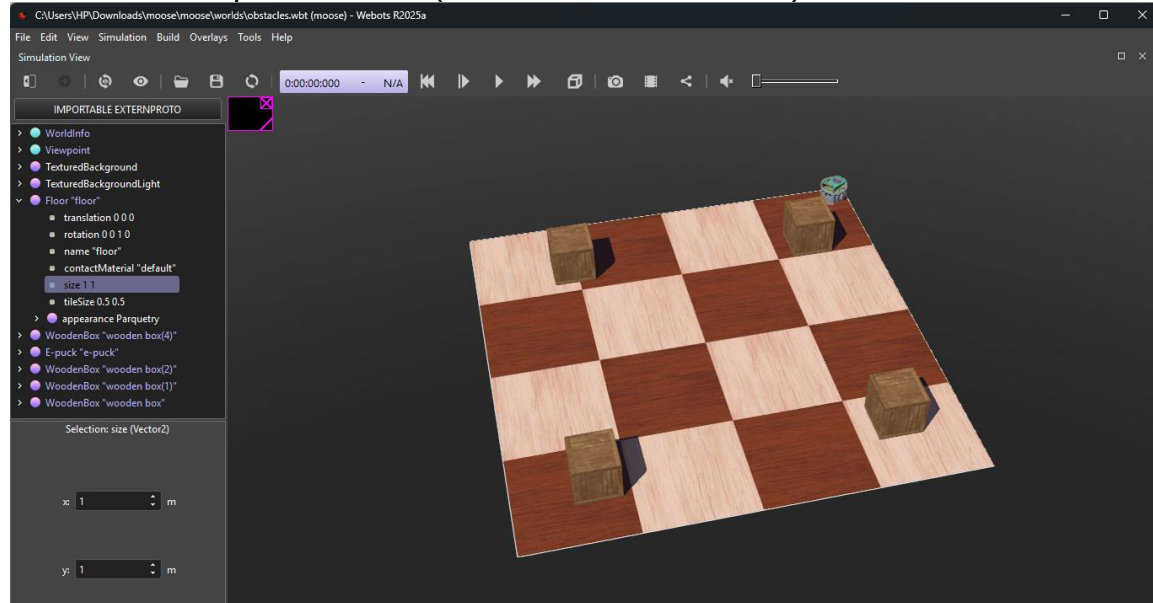
Haz clic en el botón Add (Añadir)



Práctica #3: Modificar tamaño del Floor

Expande el nodo Floor en el Scene Tree

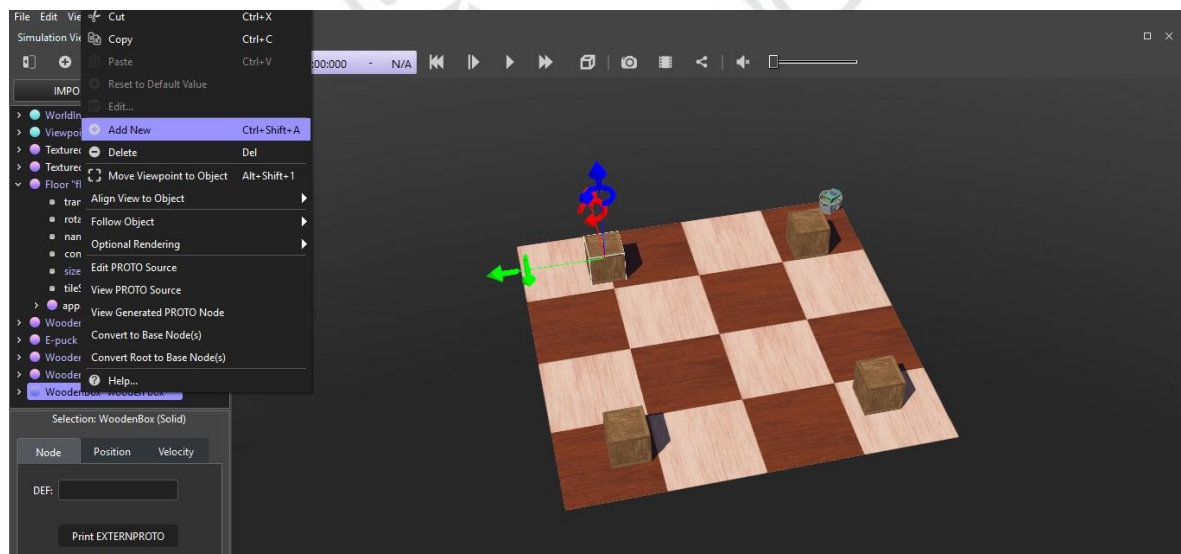
Modifica el campo size a: 1 1 (esto lo hace de 1m x 1m)



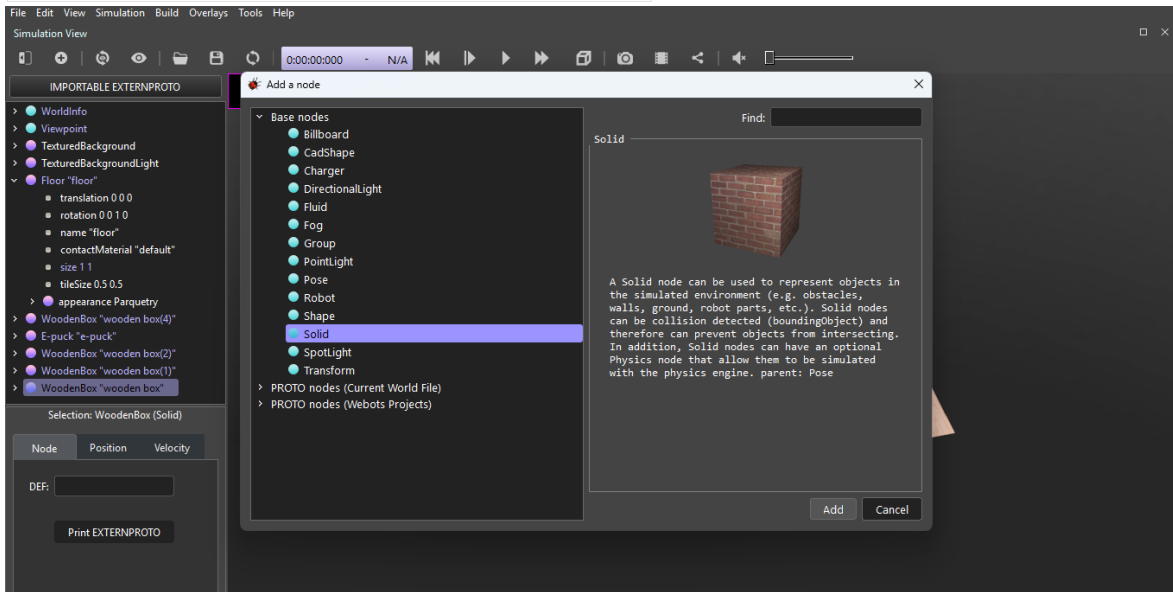
Práctica #4: Crear una Bola (Rigid Body)

Ahora crearemos una bola que interactuará con la física. Este es el concepto más importante: el **nodo Solid** (cuerpo rígido).

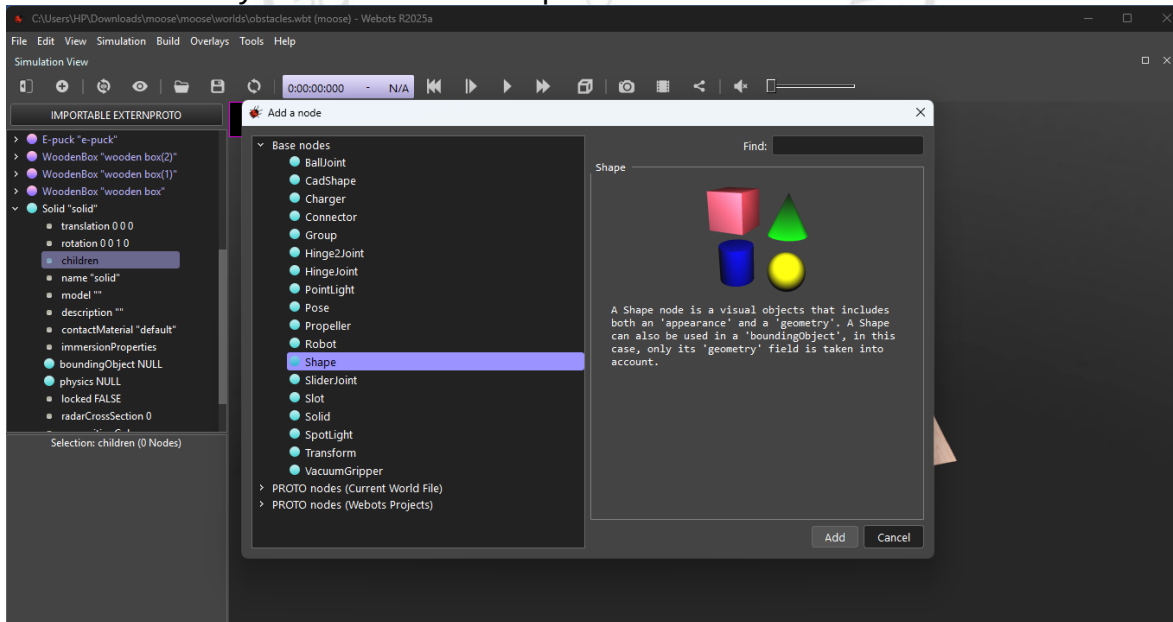
Selecciona el último nodo en el Scene Tree y presiona **Add**



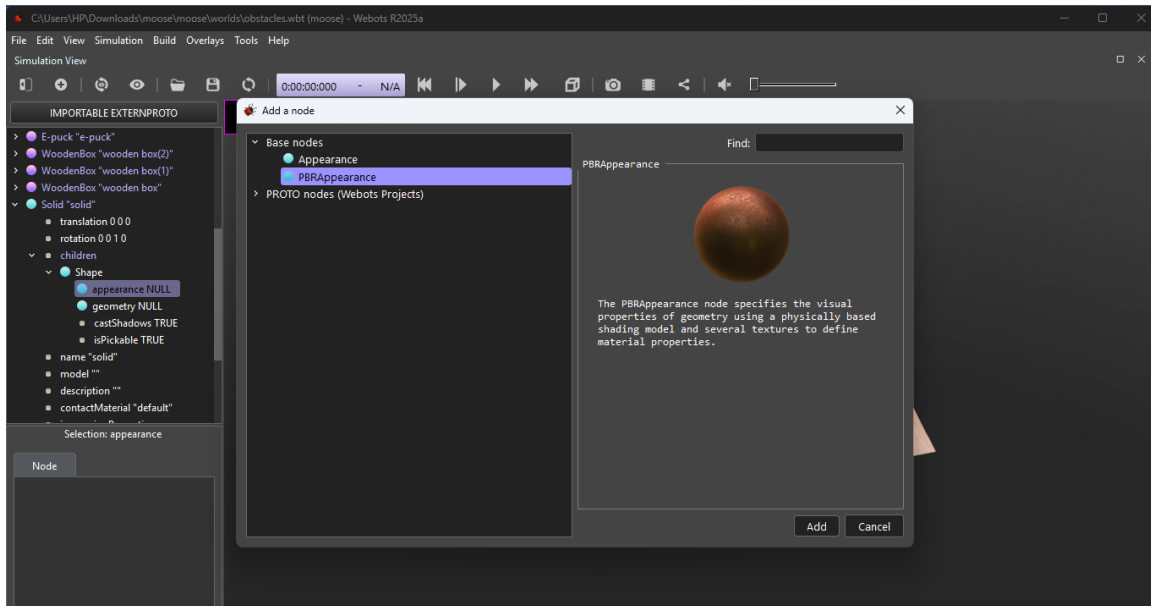
En la sección **Base nodes**, selecciona **Solid**



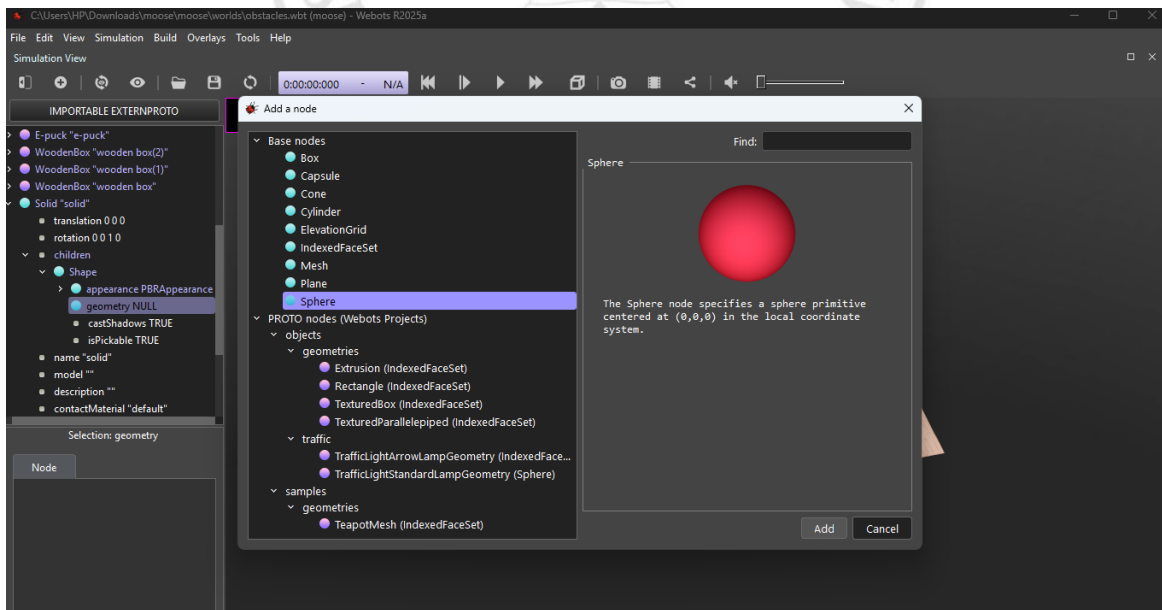
Expande el nodo Solid que acabas de crear y selecciona su campo children
Haz clic en Add y añade un nodo Shape



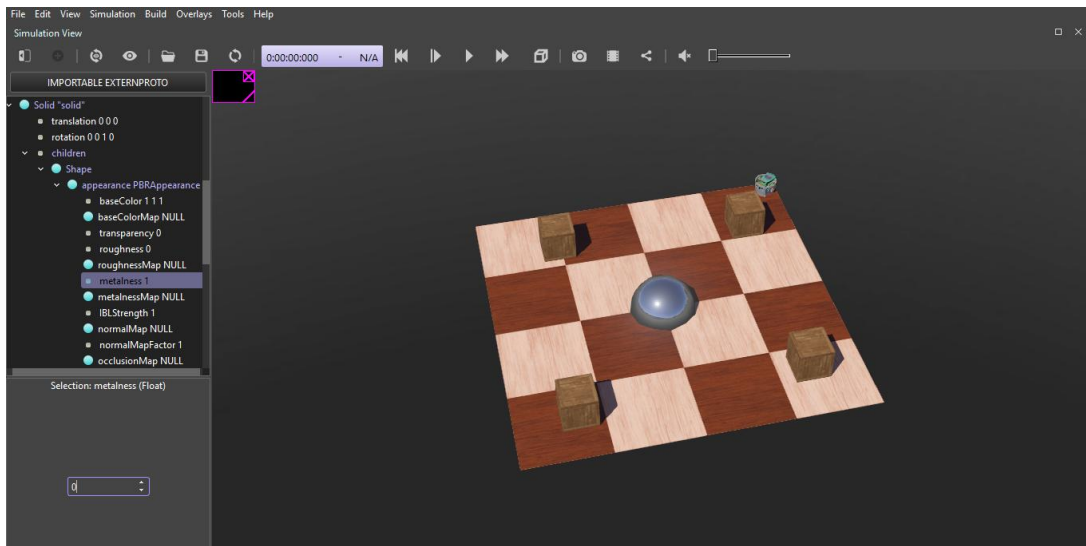
Selecciona el campo appearance del Shape y añade un nodo PBRAppearance



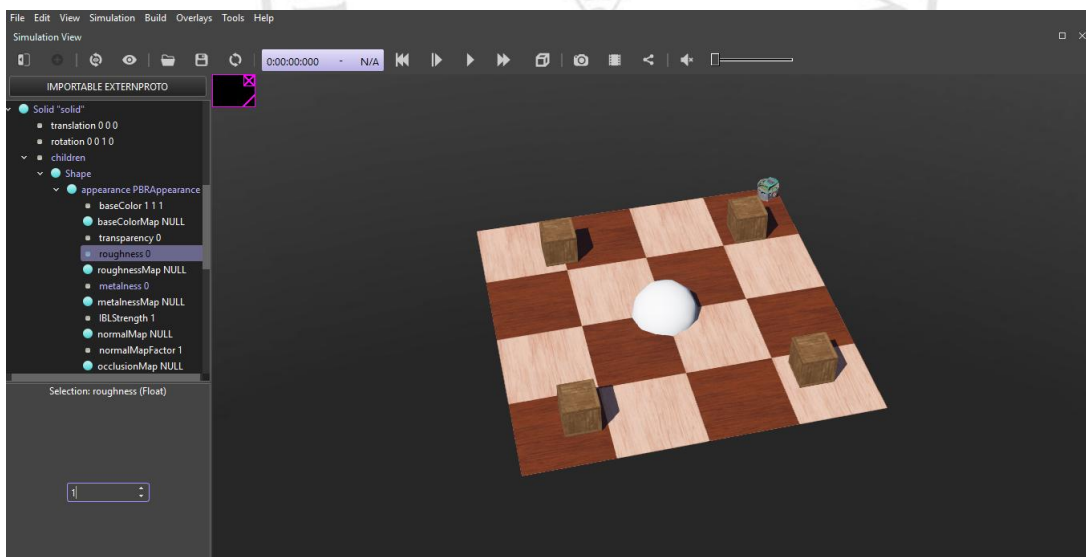
Ahora configura lo siguiente:
Añade un nodo Sphere al campo geometry del Shape



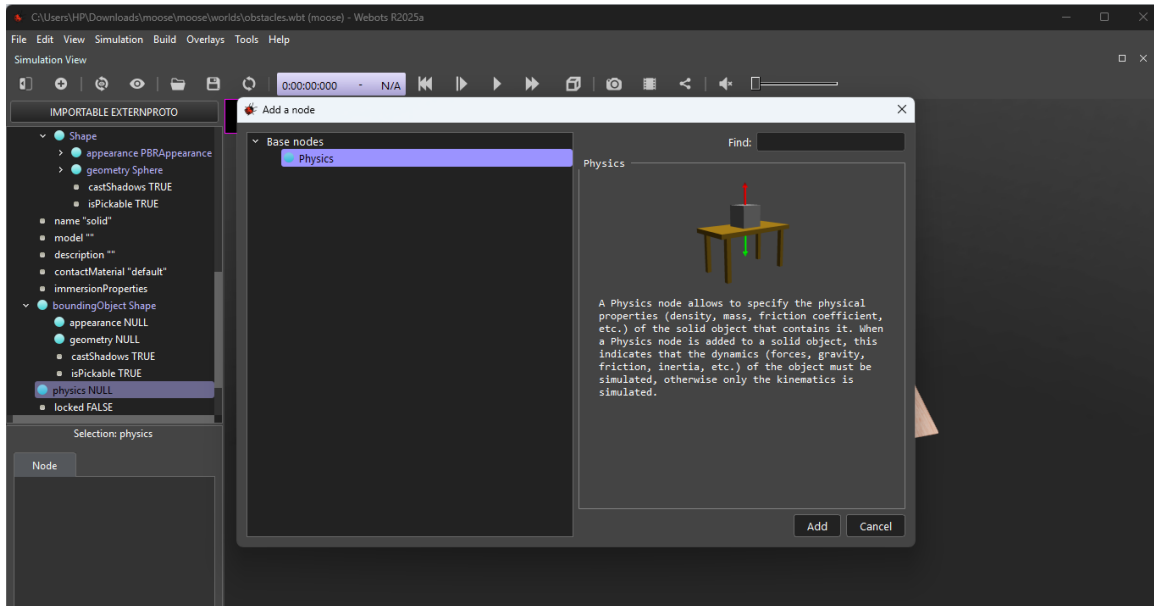
Expande el PBRAppearance y cambia:
metalness = 0



roughness =1

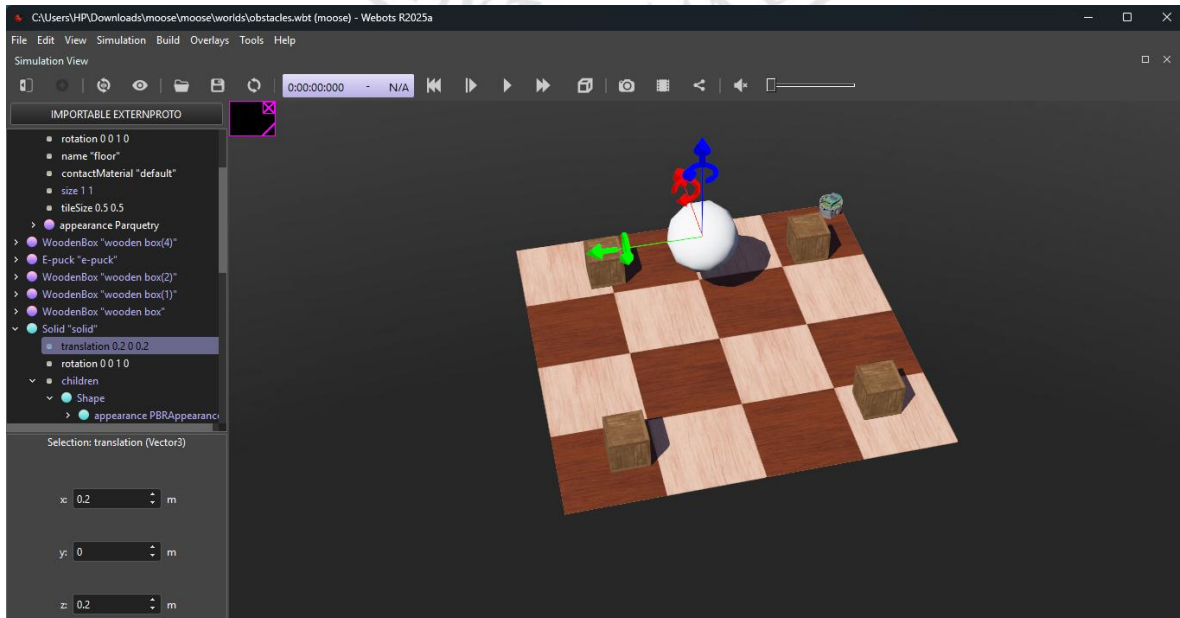


Añade otro nodo Sphere al campo boundingObject del Solid
Añade un nodo Physics al campo physics del Solid



Cambiar la posición de la bola

Busca el campo translation al inicio del nodo Solid (dice 0 0 0) y cambialos a 0.2, 0 0.2



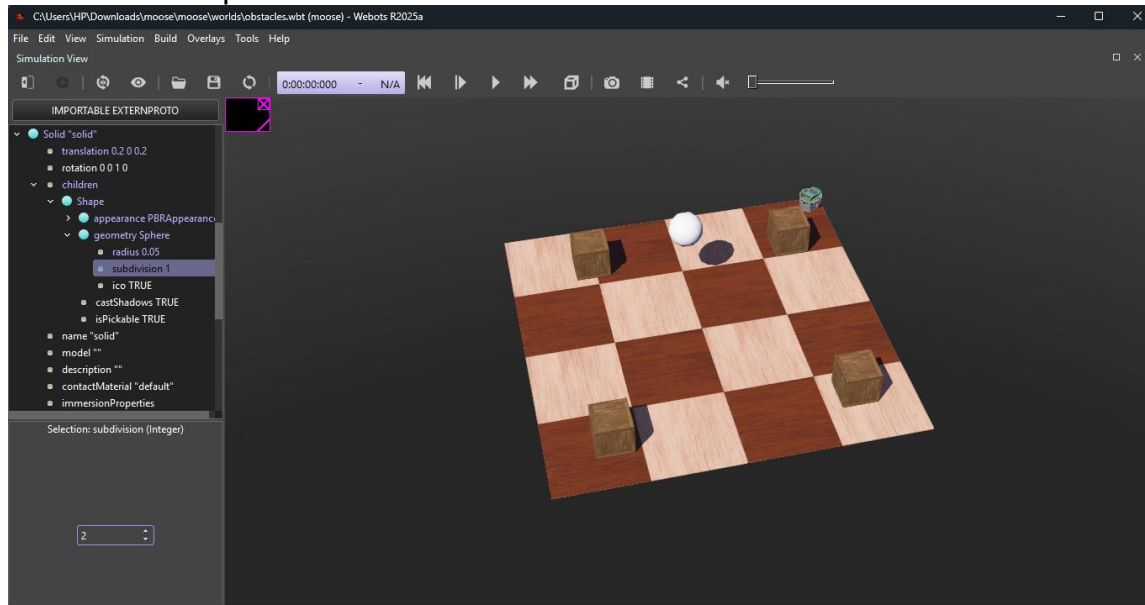
Práctica práctica #5:

Reducir el tamaño de la bola

Expande el nodo geometry Sphere (doble clic sobre él)

Cambia el campo radius a: 0.05

Cambia el campo subdivision a: 2

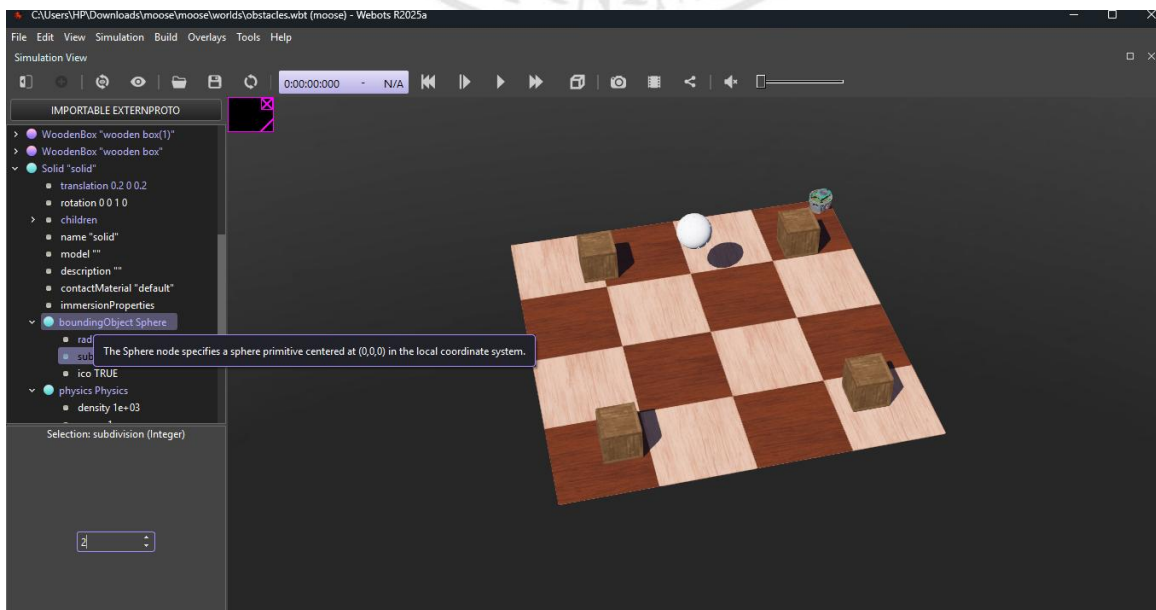


Lo mismo con el Sphere que está en boundingObject:

Expande el Sphere del boundingObject

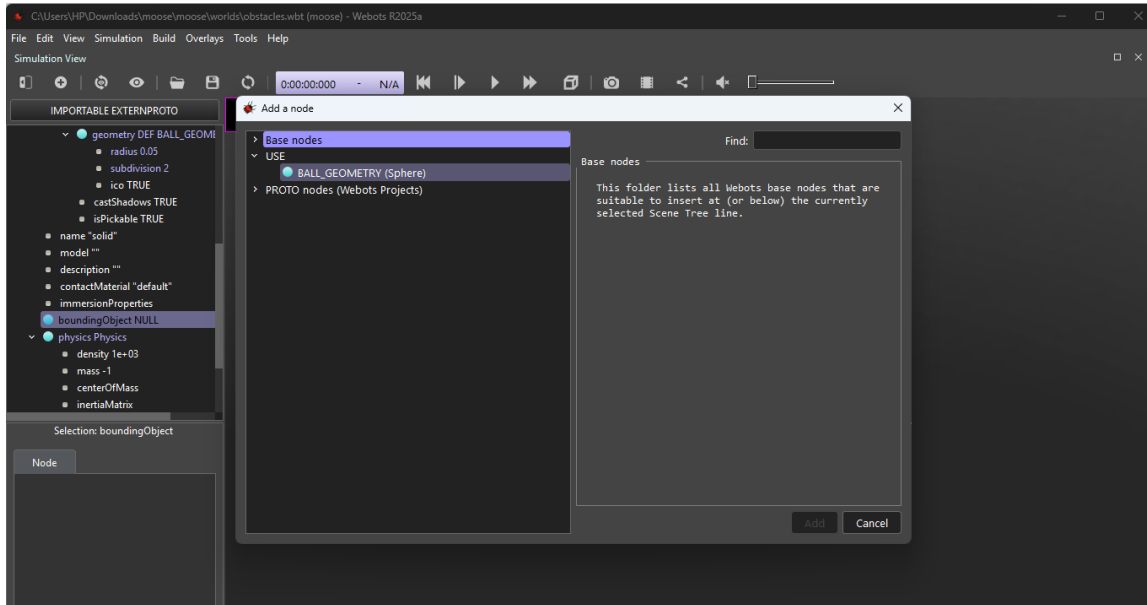
Cambia radius a: 0.05

Cambia subdivision a: 2

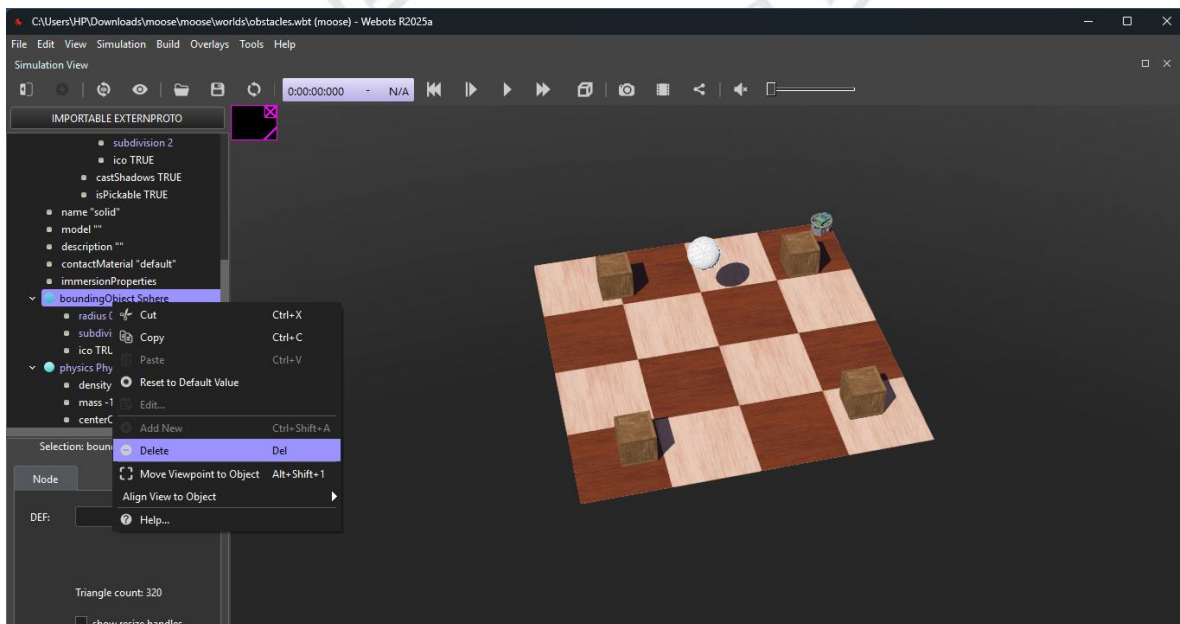


Práctica #6

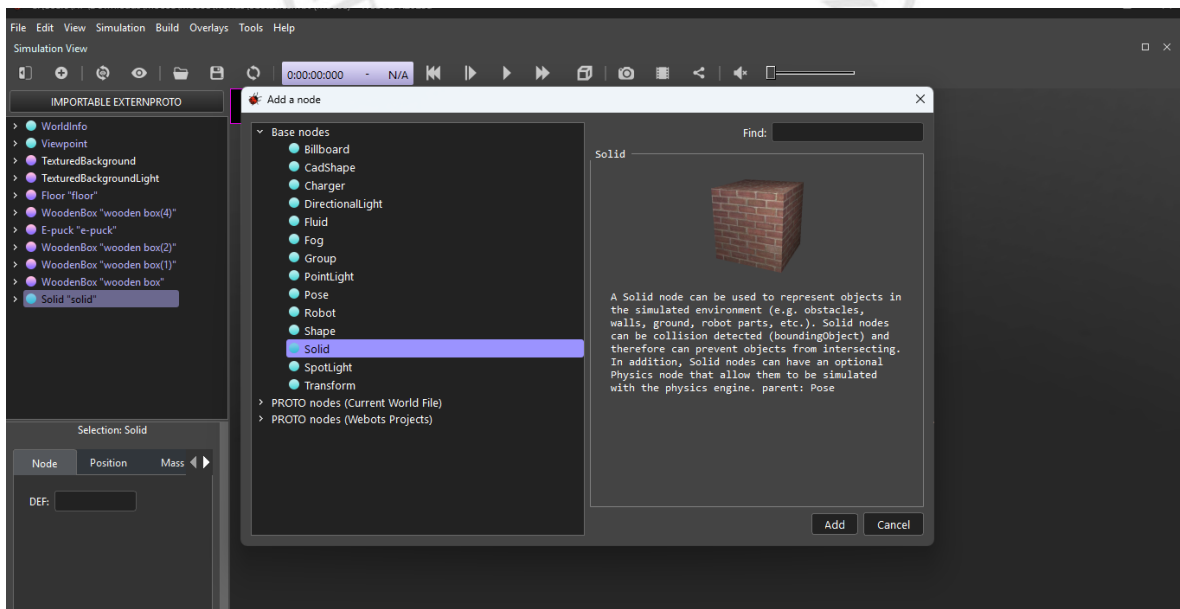
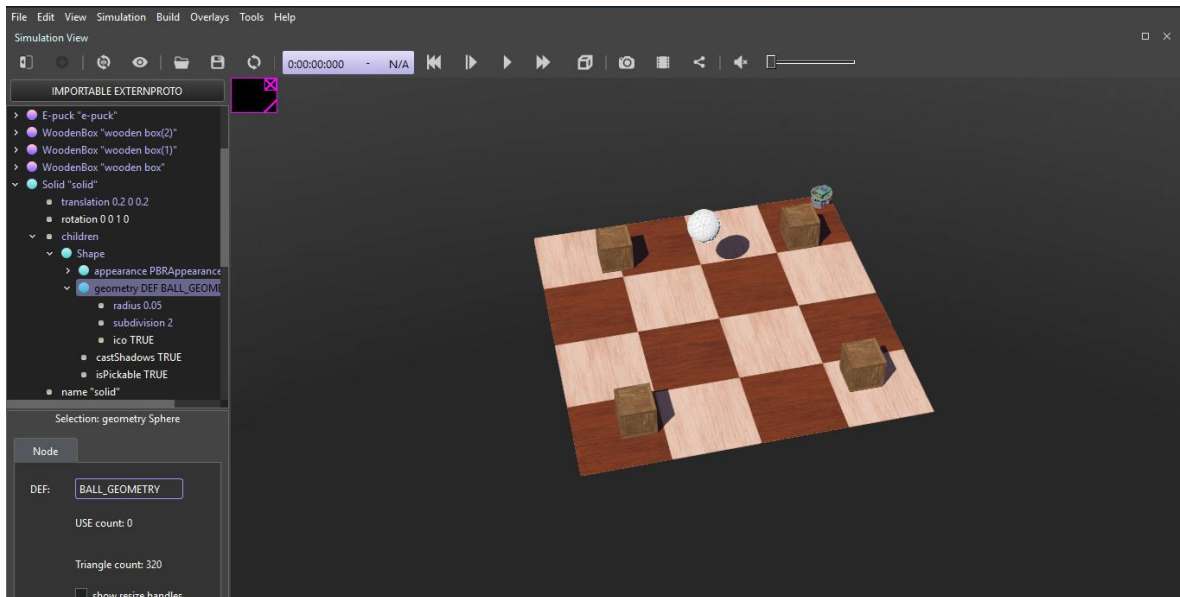
Con boundingObject seleccionado, haz clic en Add
Selecciona: USE > BALL_GEOMETRY



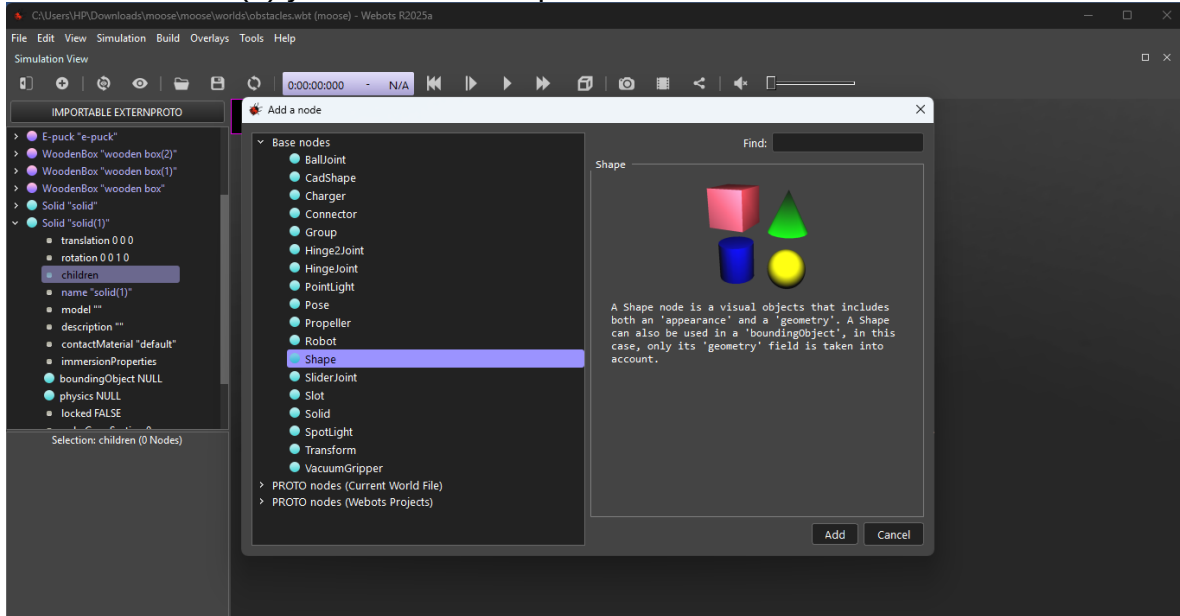
Eliminar el Sphere que está en boundingObject:
Clic derecho sobre boundingObject Sphere > Delete



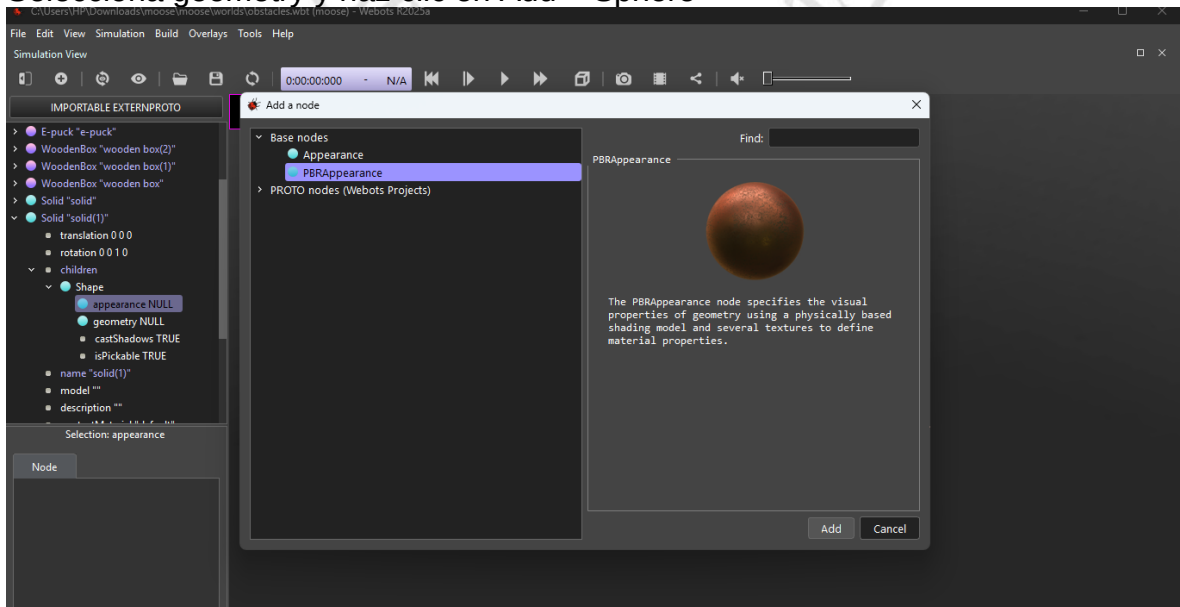
Práctica #7: Crear una segunda bola con los mismos parámetros pero usando el nodo [Shape](#) (en lugar del nodo [Sphere](#) directamente) para el mecanismo DEF-USE.



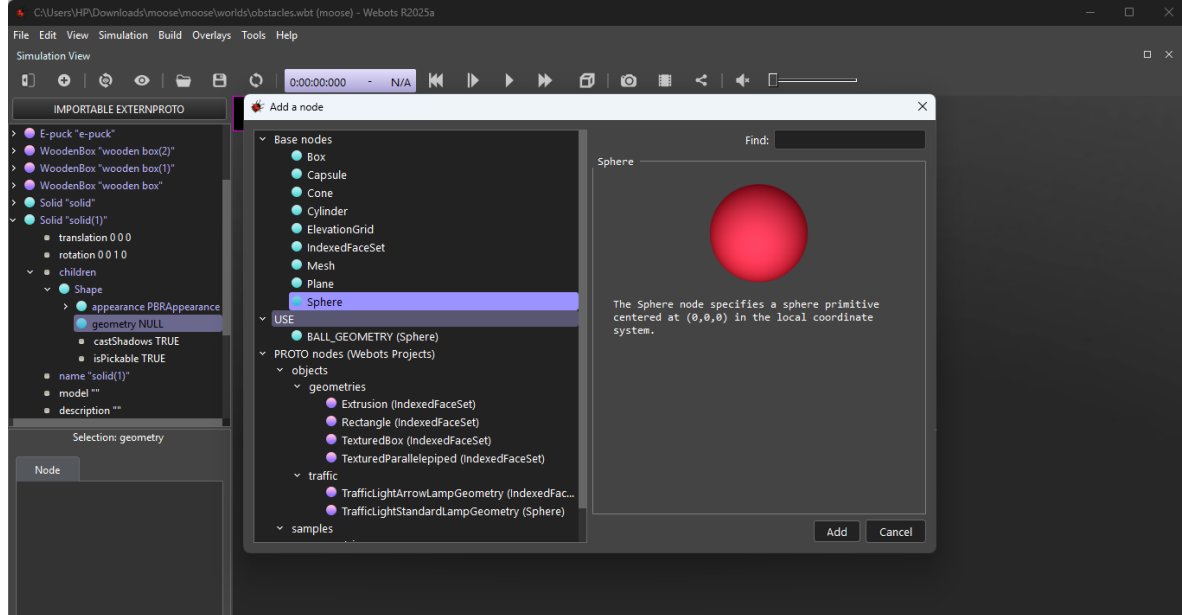
Selecciona el campo children
Haz clic en Add (+) y selecciona: Shape



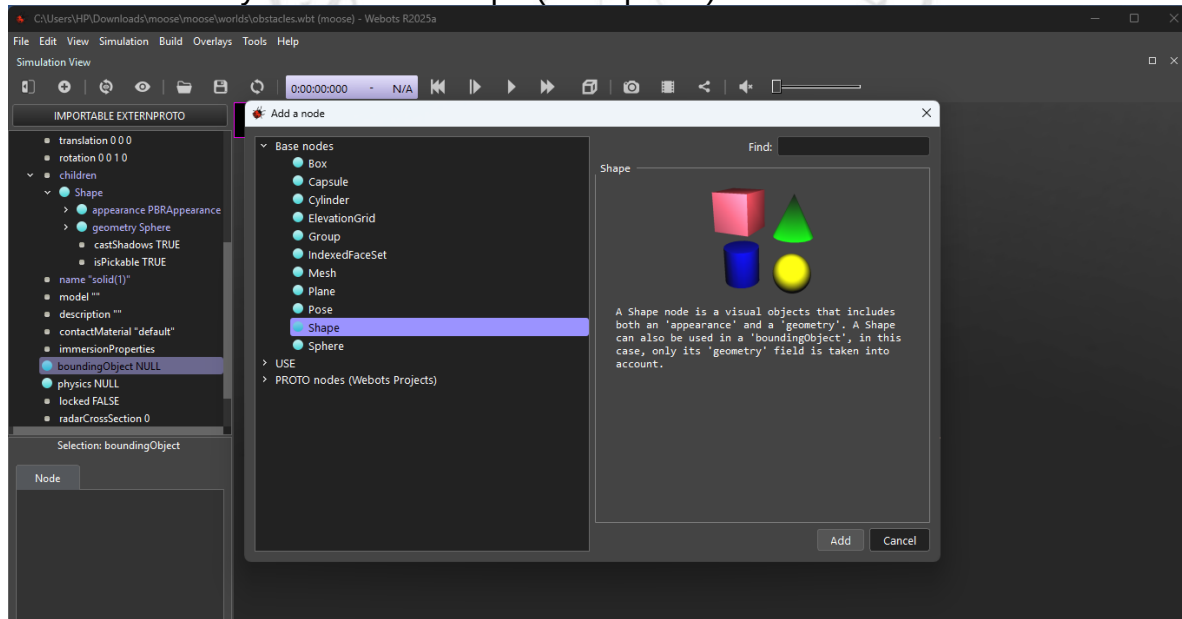
Expande ese Shape y selecciona appearance
Haz clic en Add y selecciona: PBRAppearance
Selecciona geometry y haz clic en Add > Sphere



Selecciona geometry y haz clic en Add > Sphere



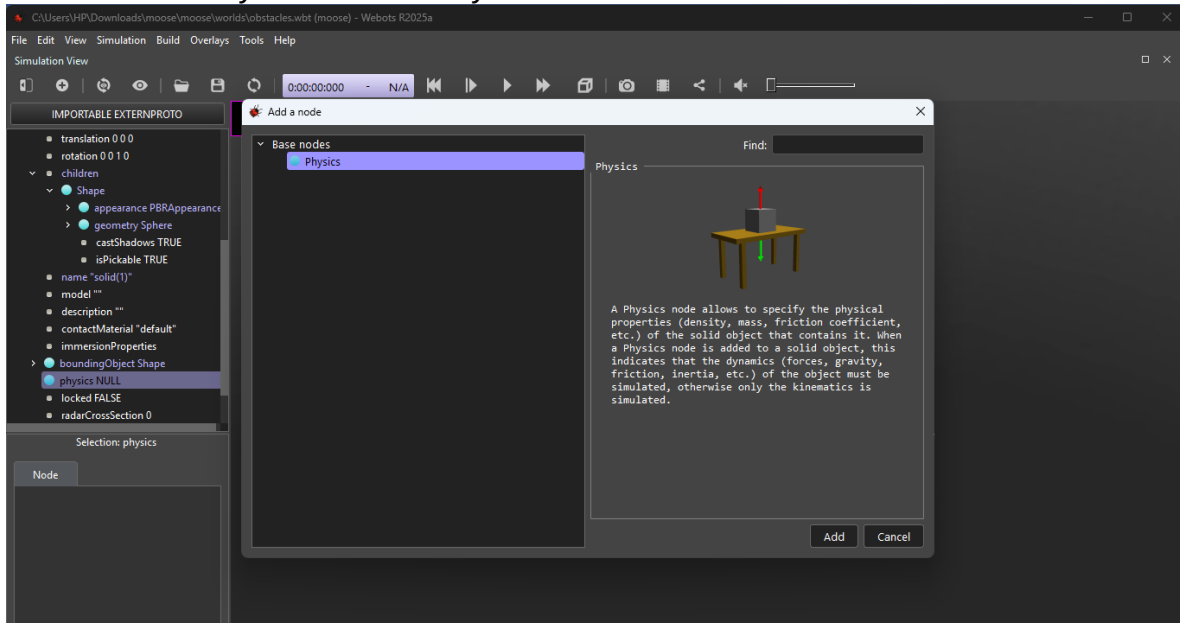
Busca el campo boundingObject (debería decir NULL) Haz clic en Add y selecciona: Shape (NO Sphere)



Añadir Physics

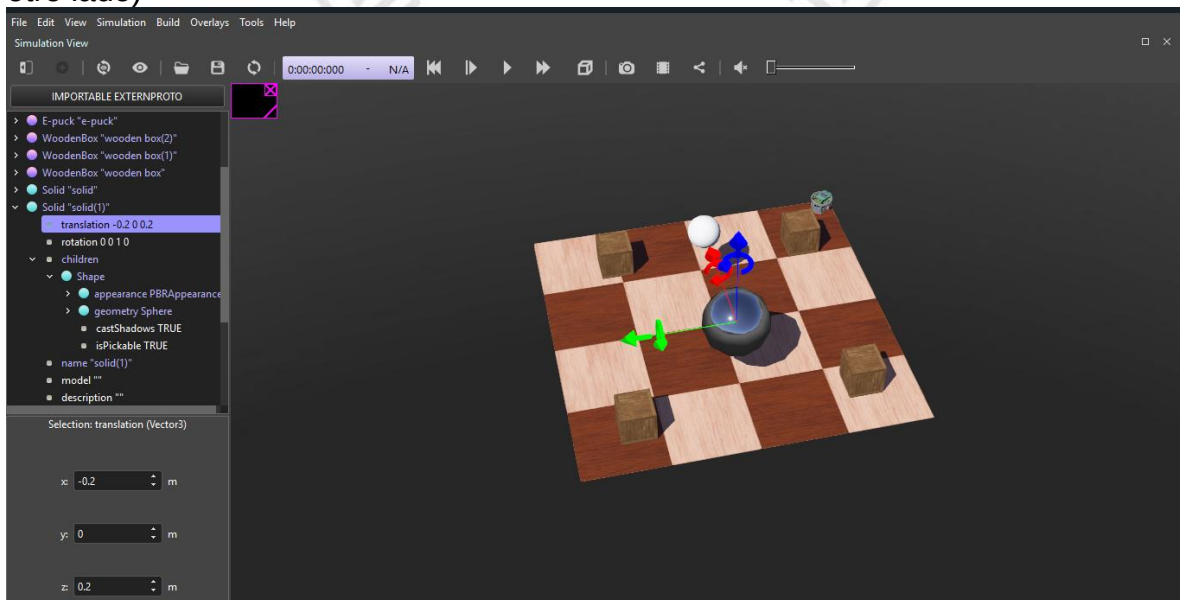
Busca el campo physics (debería decir NULL)

Haz clic en Add y selecciona: Physics



Busca el campo translation del segundo Solid

Cámbialo a una posición diferente, por ejemplo: -0.2 0 0.2 (para que quede del otro lado)

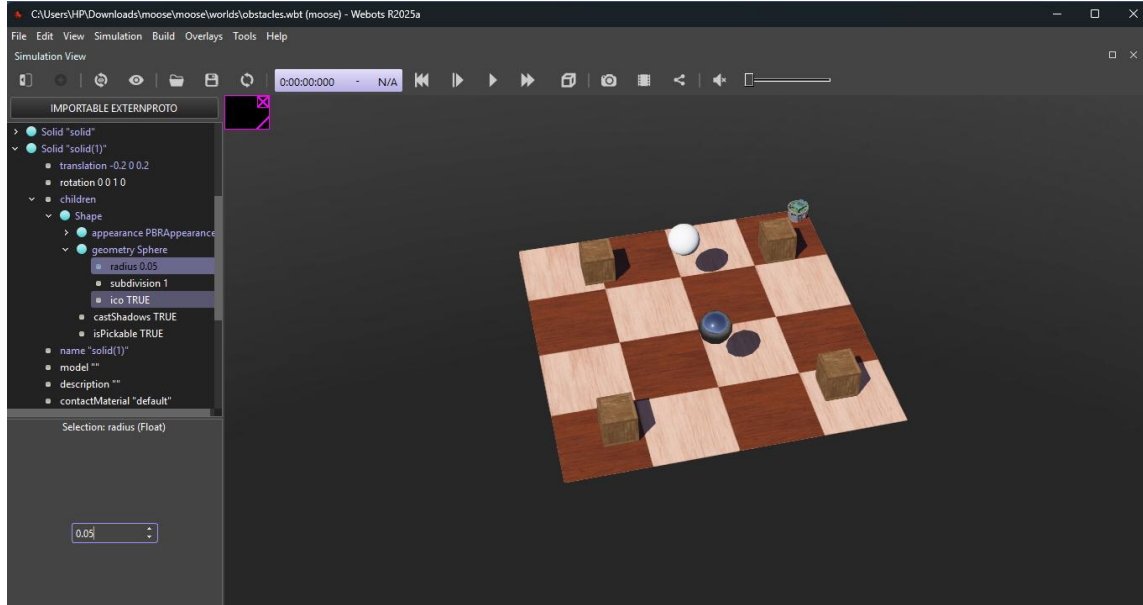


Configurar tamaños y apariencia

En ambos Sphere (el de geometry y el del boundingObject):

Cambia radius a: 0.05

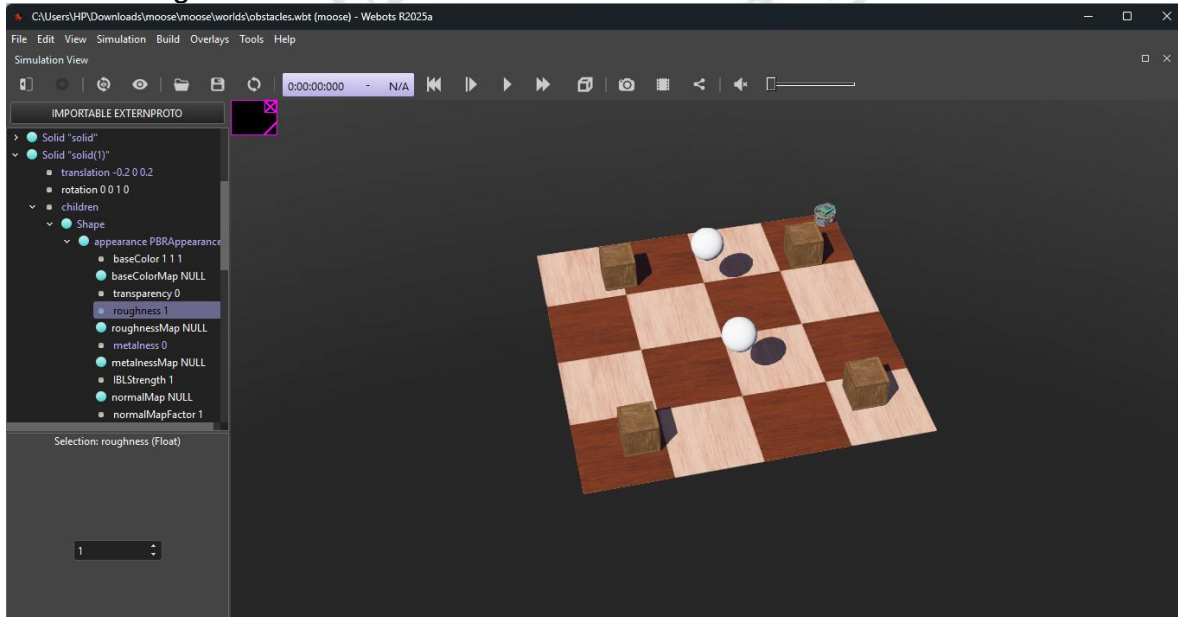
Cambia subdivision a: 2



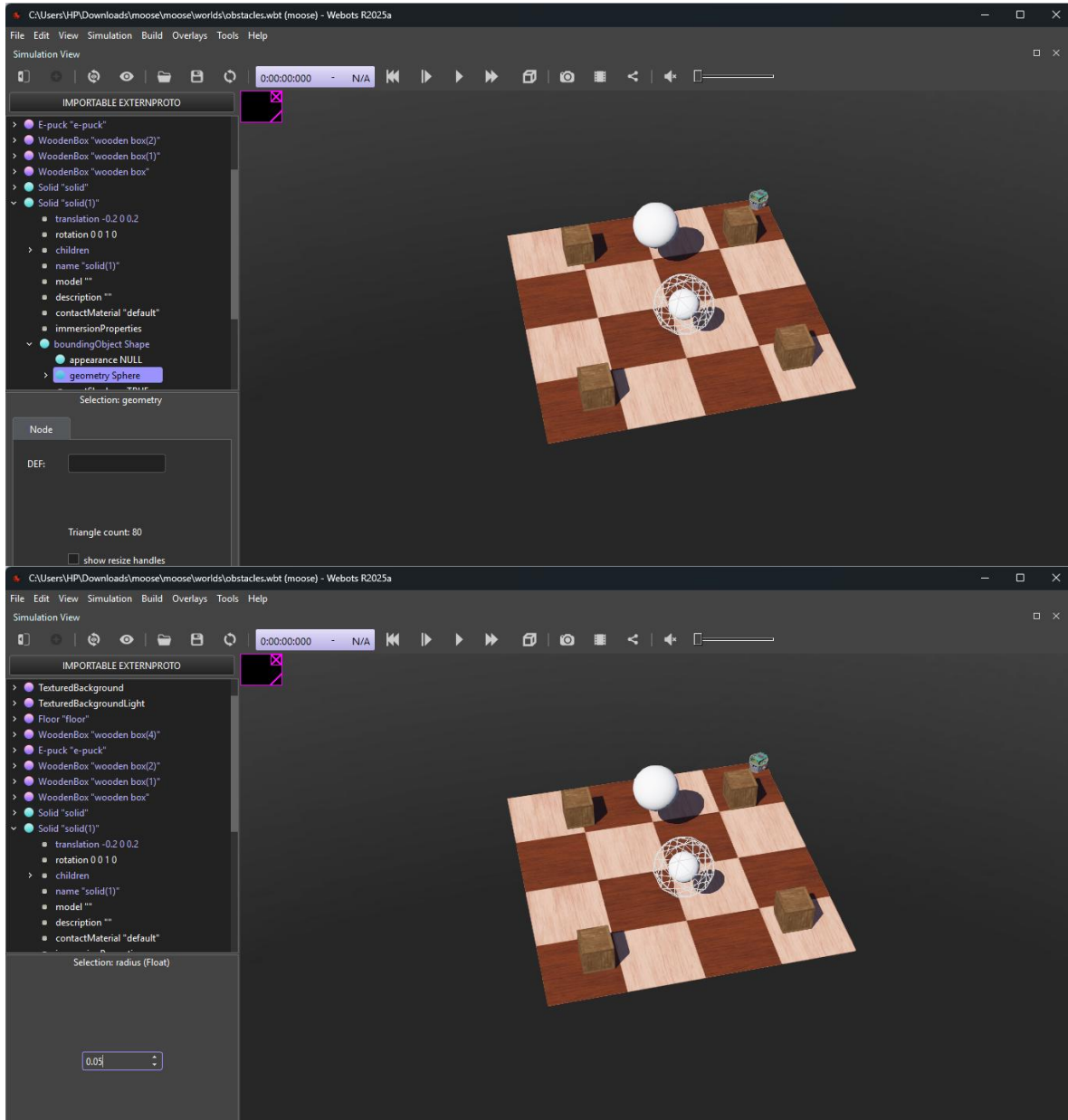
En el PBRAppearance de la segunda bola:

Cambia metalness a: 0

Cambia roughness a: 1



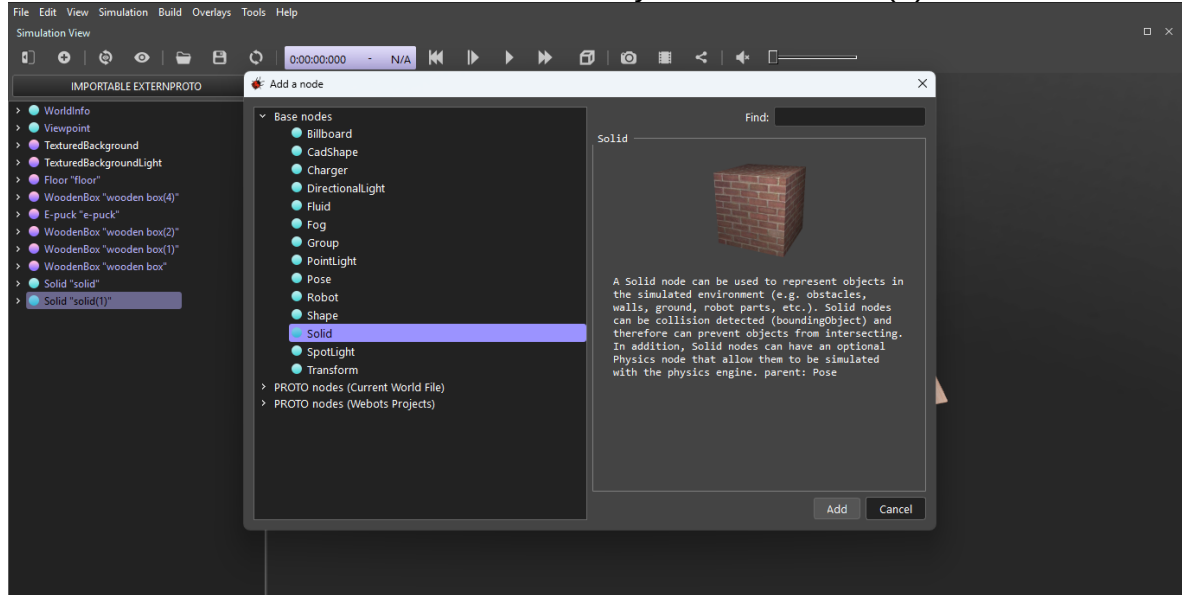
Expande ese Shape > selecciona geometry > Add > Box o Sphere
Si usas Sphere: radius = 0.05



Práctica #8

Crear la primera pared (Modelo)

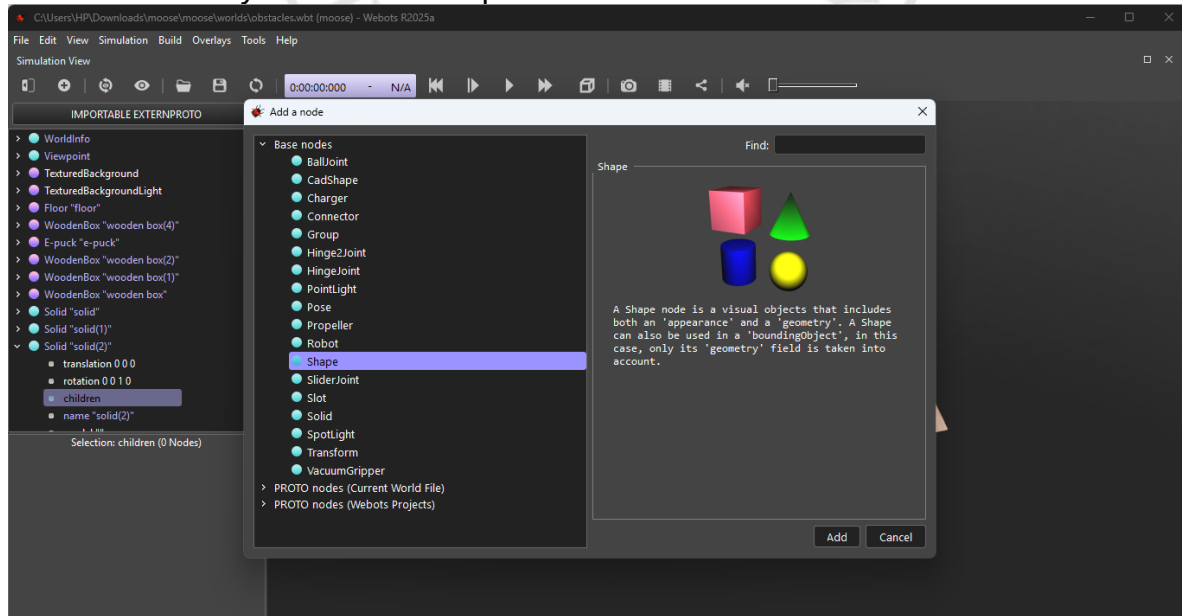
Selecciona el último nodo en el Scene Tree y haz clic en Add (+).



Expande ese nuevo Solid y selecciona el campo children.

Expande ese nuevo Solid y selecciona el campo children.

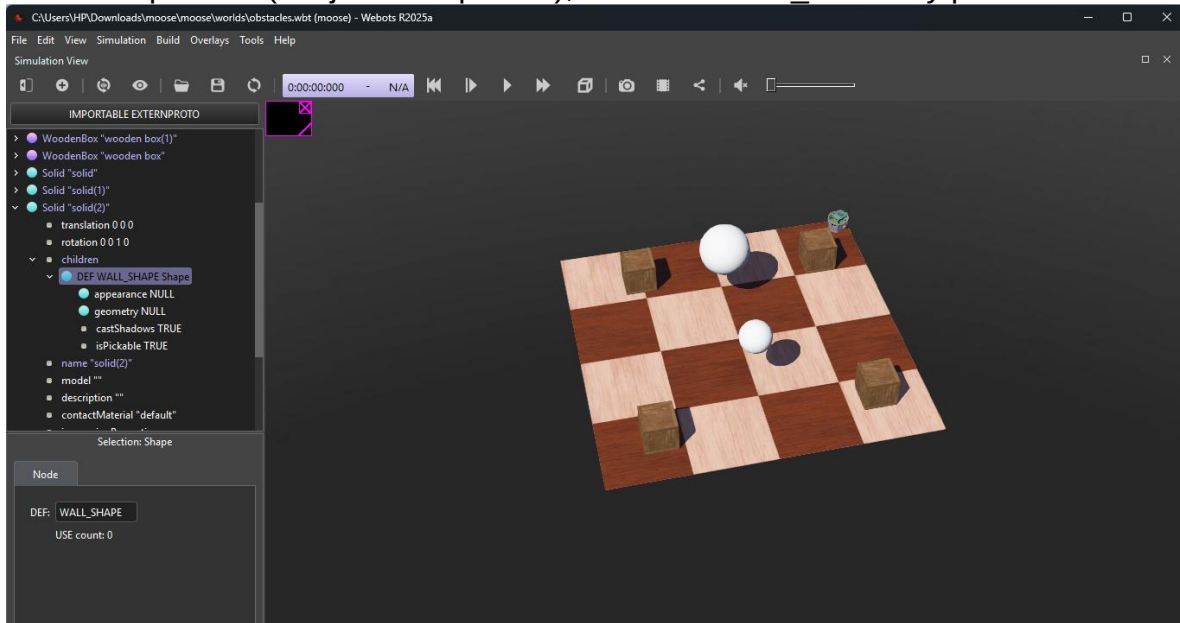
Haz clic en Add y selecciona Shape.



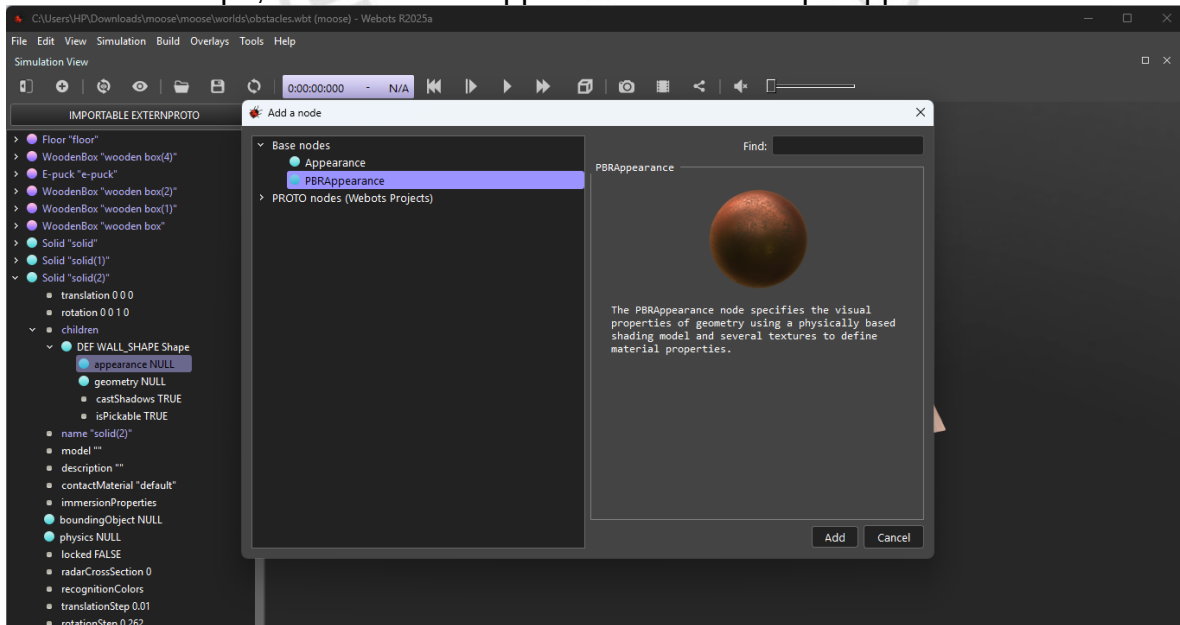
Configura el Shape (Aquí aplicamos DEF):

Selecciona el nodo Shape que acabas de crear.

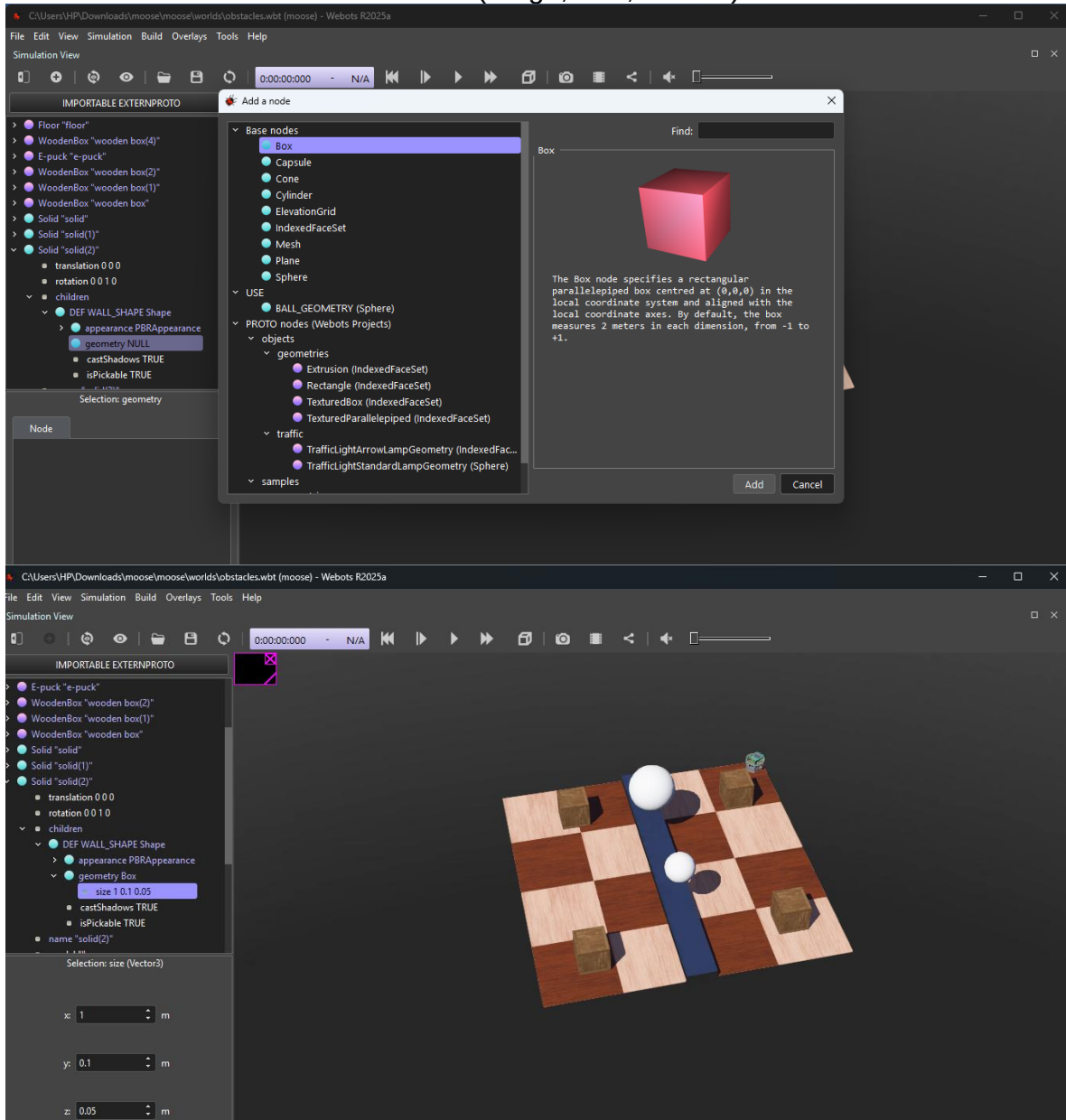
En el campo DEF (abajo a la izquierda), escribe: WALL_SHAPE y presiona Enter.



Dentro del Shape, añade un PBRAppearance en el campo appearance.



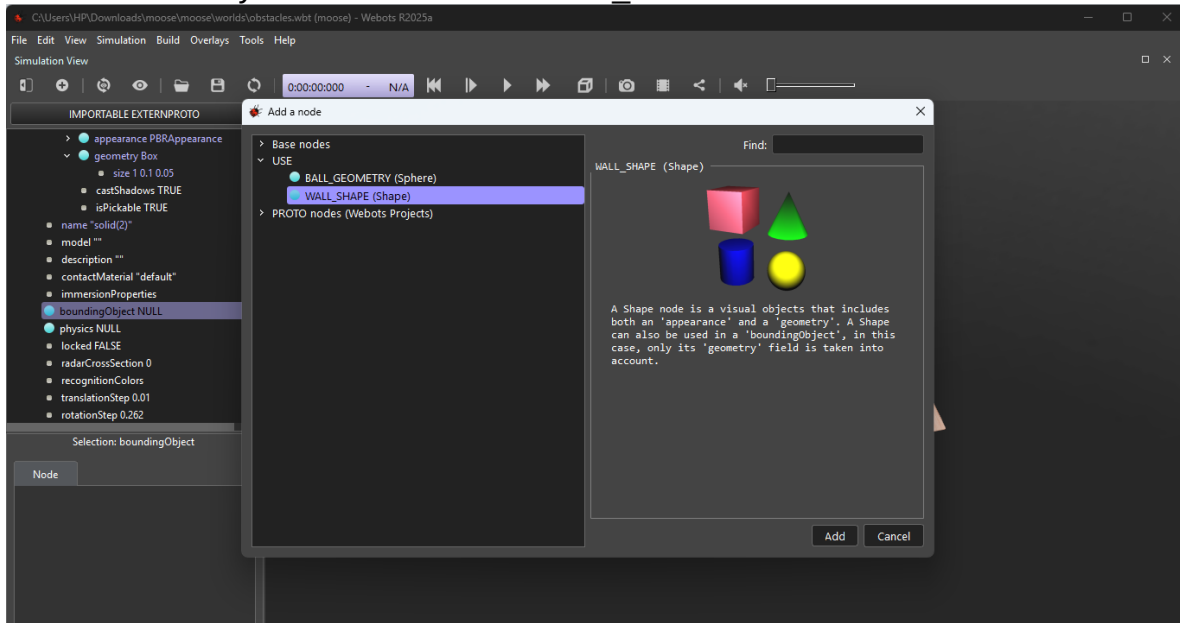
En el campo geometry, añade un Box.
Cambia el size del Box a: 1 0.1 0.05 (Largo, Alto, Grosor).



Configura el BoundingBox (Aquí aplicamos USE):

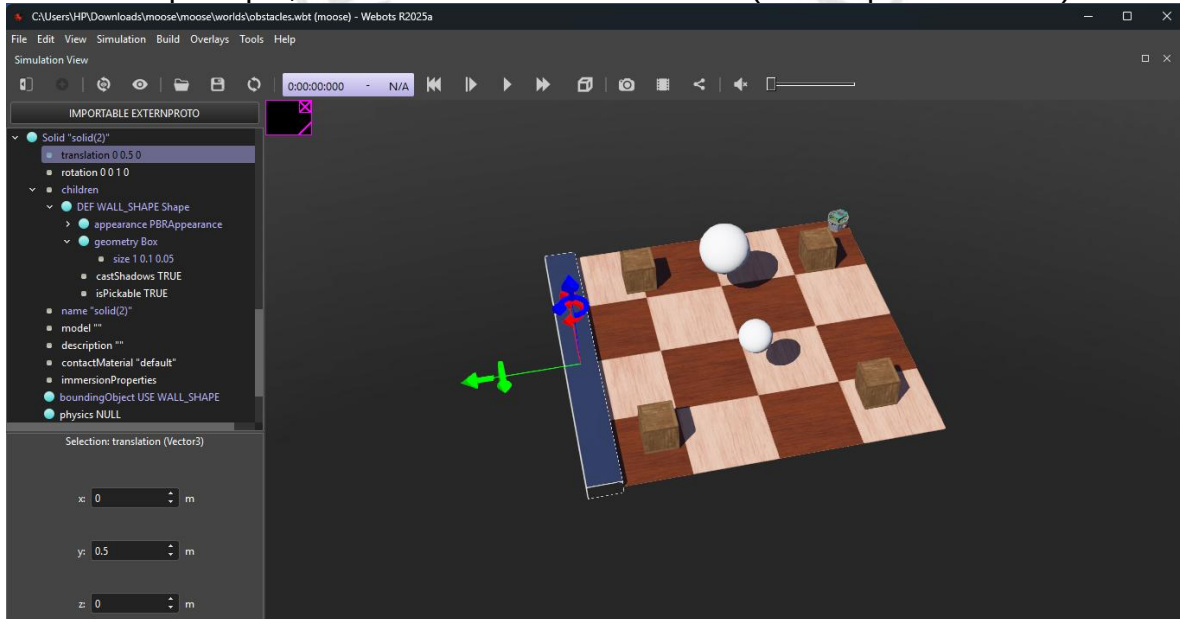
En el Solid principal, selecciona el campo boundingObject.

Haz clic en Add y selecciona USE > WALL_SHAPE.



Posiciona la pared:

En el Solid principal, cambia el translation a: 0 0.5 0 (Esto la pone al fondo).

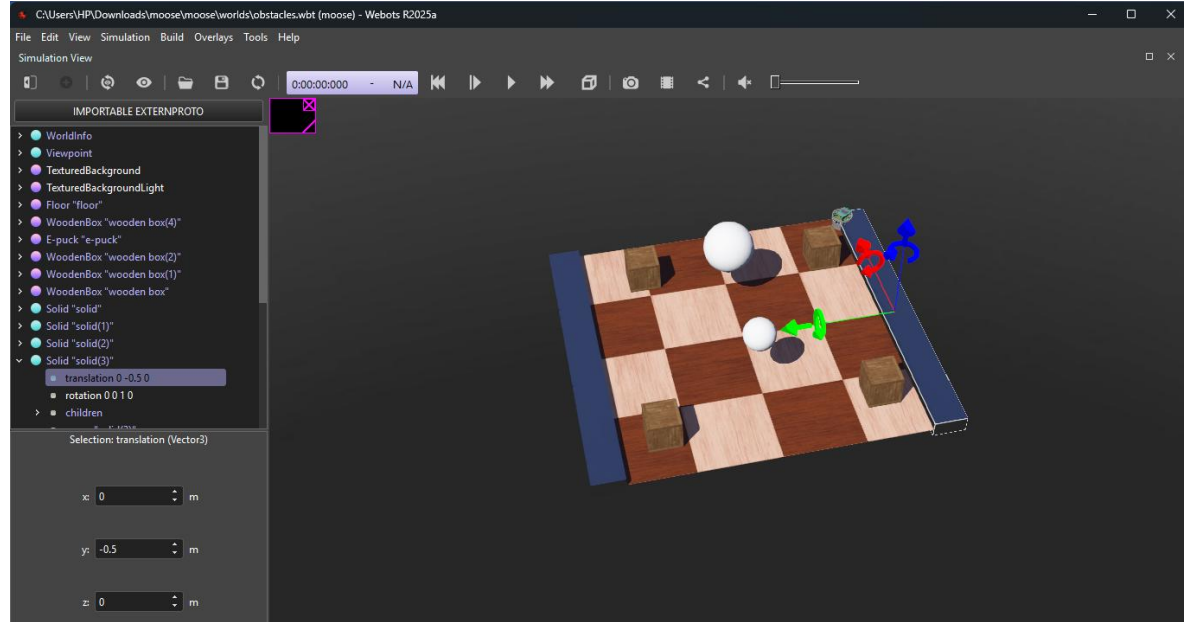


Crear las otras 3 paredes (Copiar y Pegar)

Selecciona el Solid de la pared que acabas de crear.

Presiona Ctrl + C y luego Ctrl + V

Cambia translation a: 0 -0.5 0.

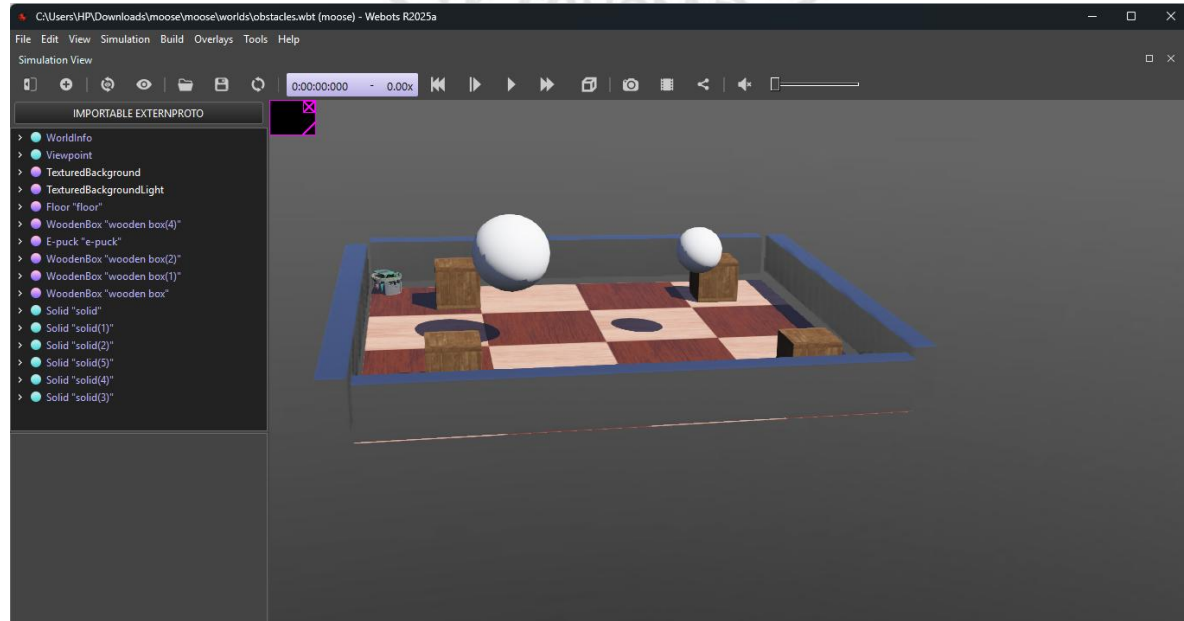


Pared Derecha (Este):

Copia y pega otra vez.

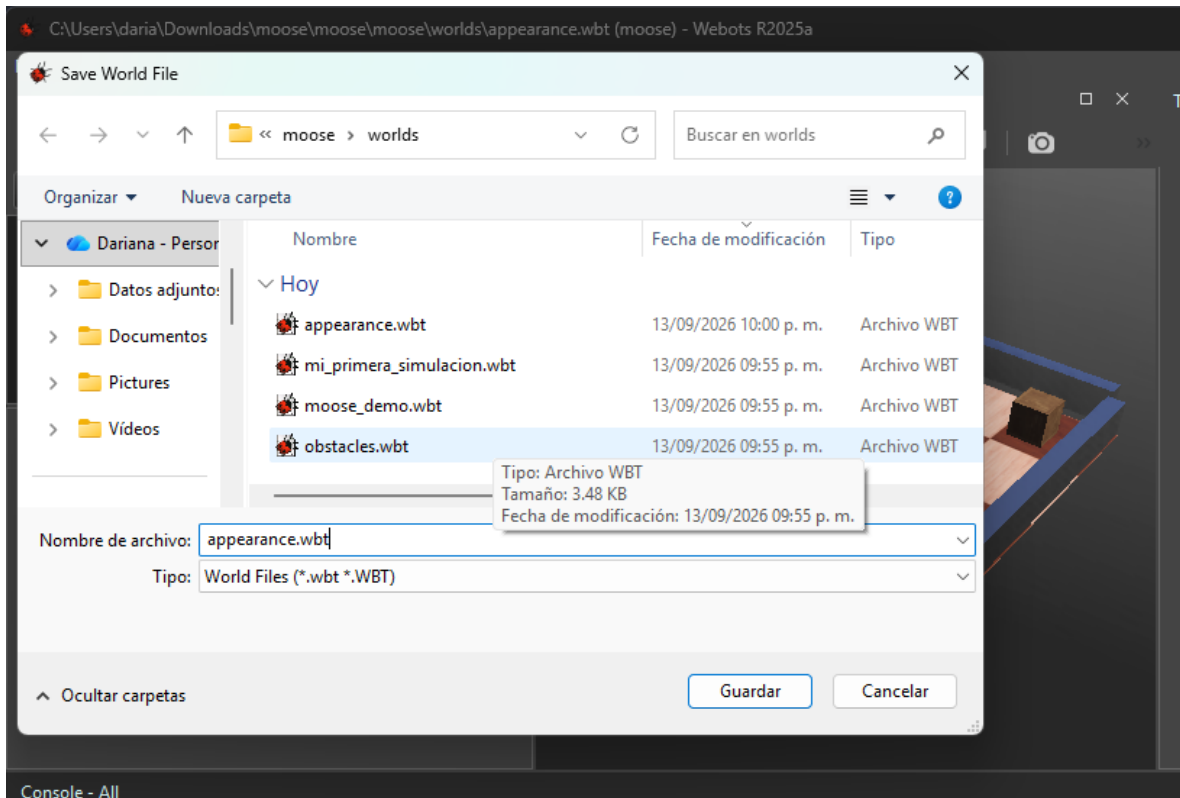
Cambia translation a: 0.5 0 0.

Cambia rotation a: 0 1 0 1.57

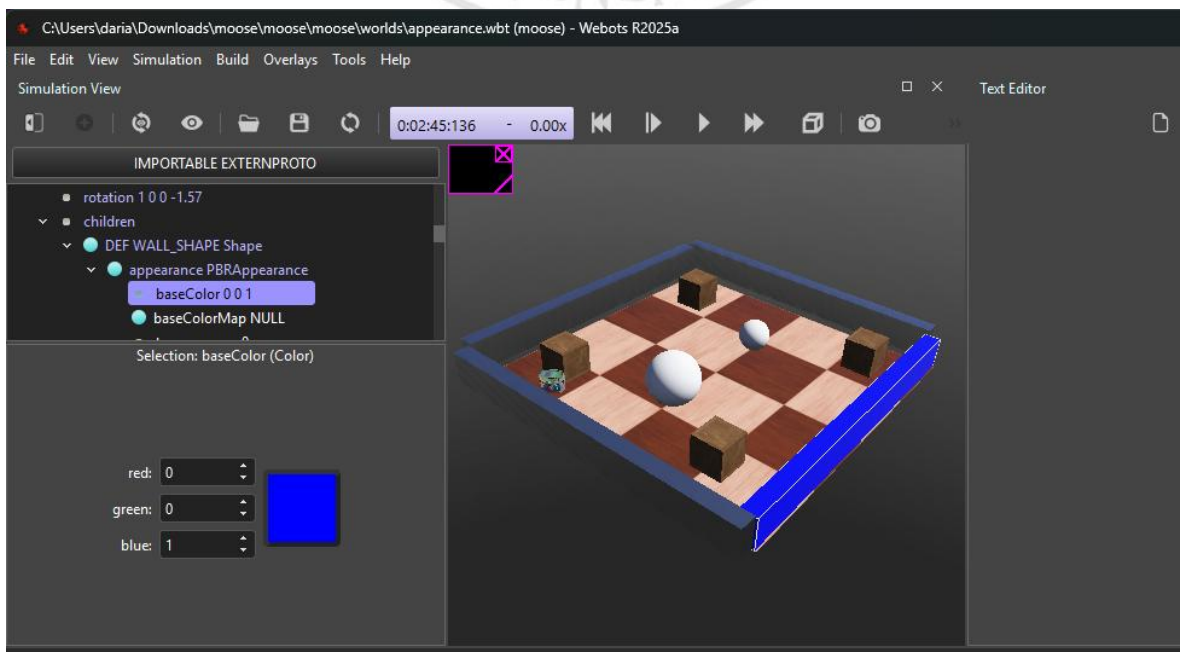


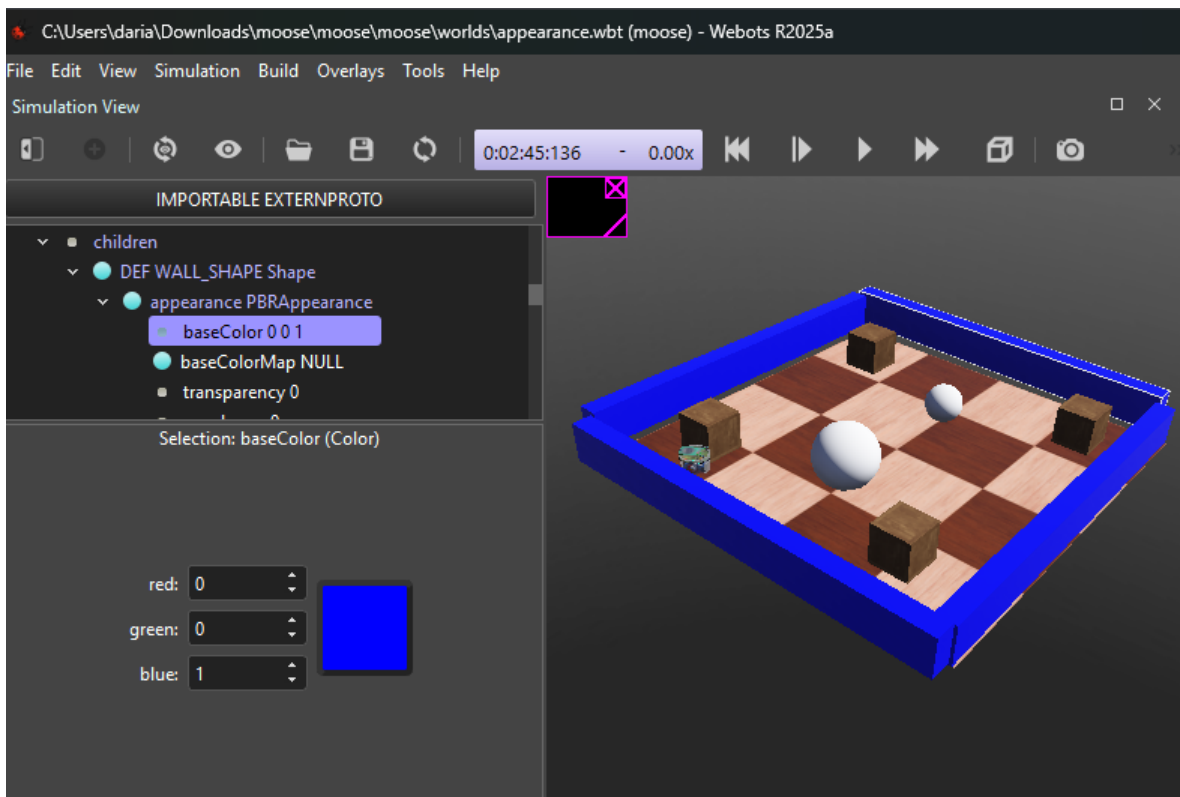
TUTORIAL 3

Hands-on #1: Guardar como appearance.wbt

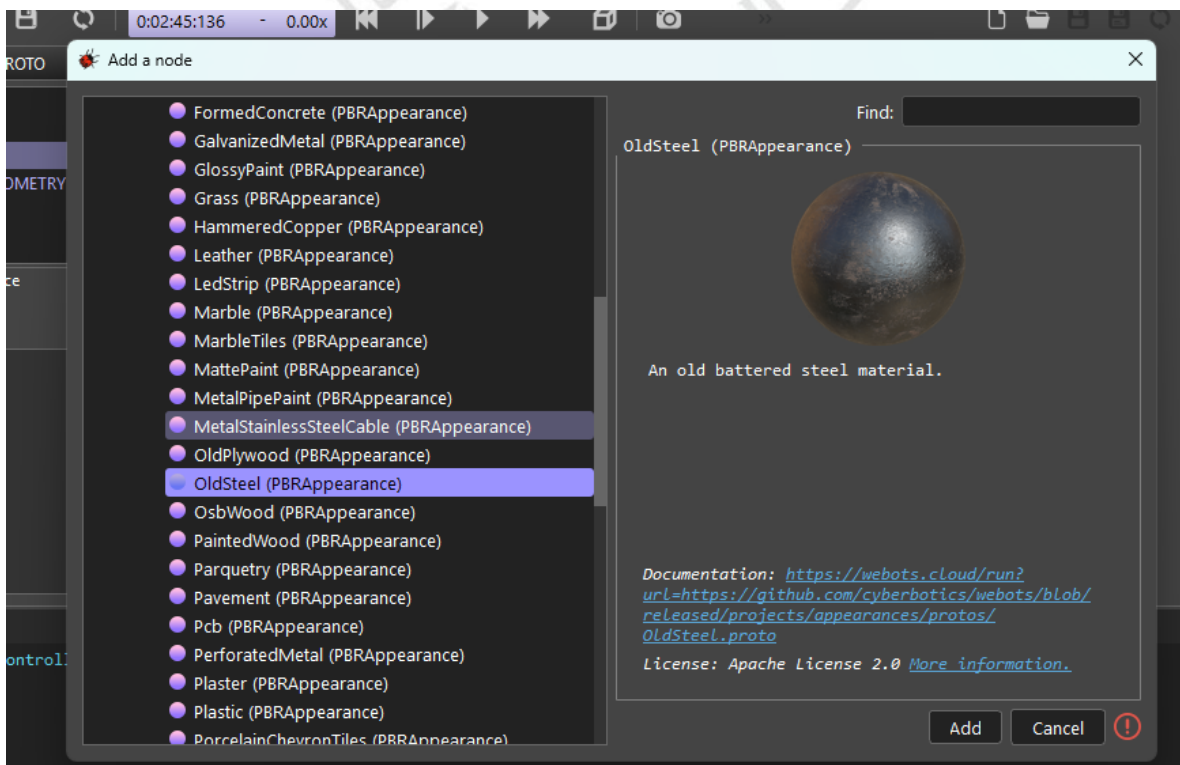


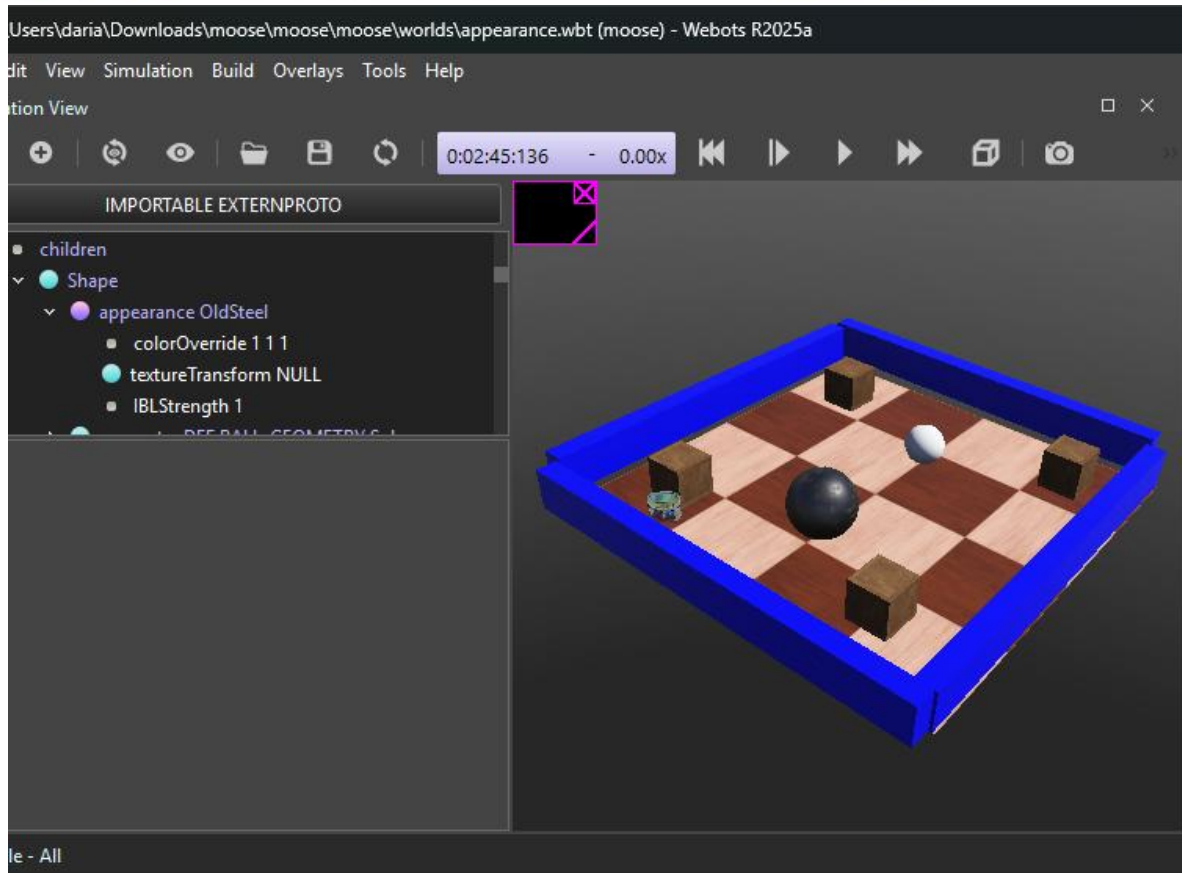
Hands-on #2: Colorear las paredes de azul



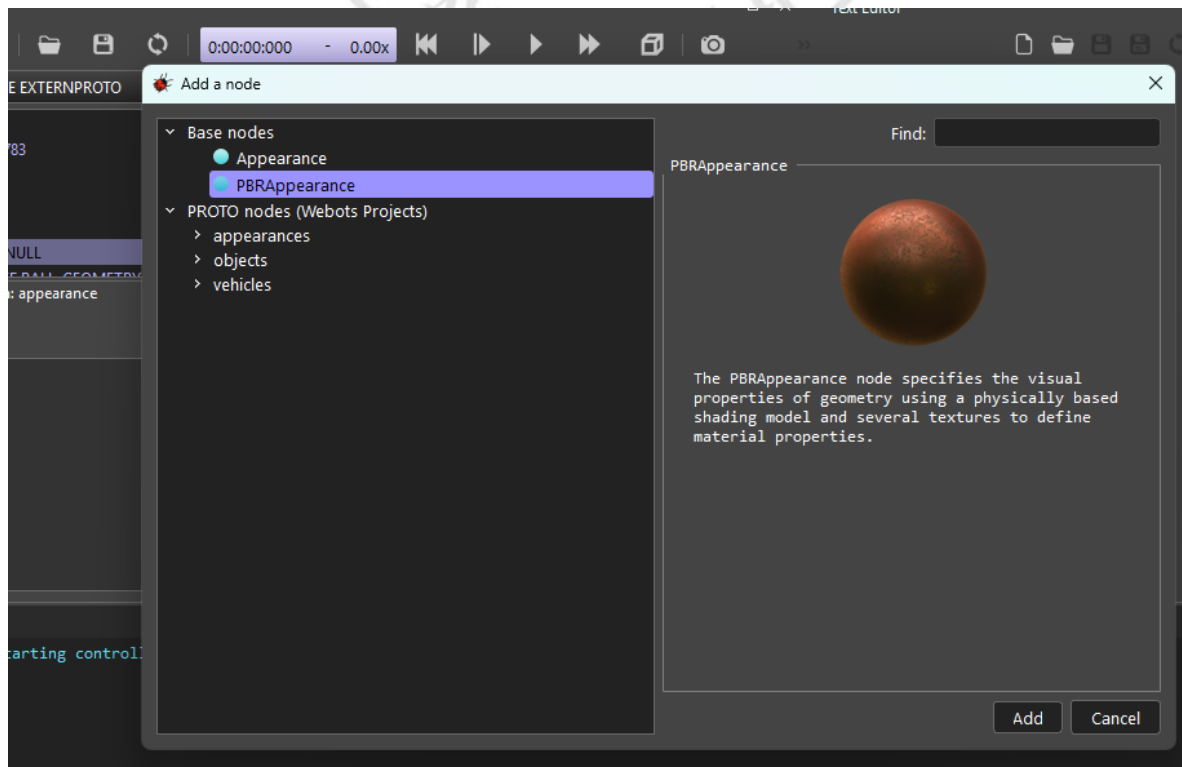


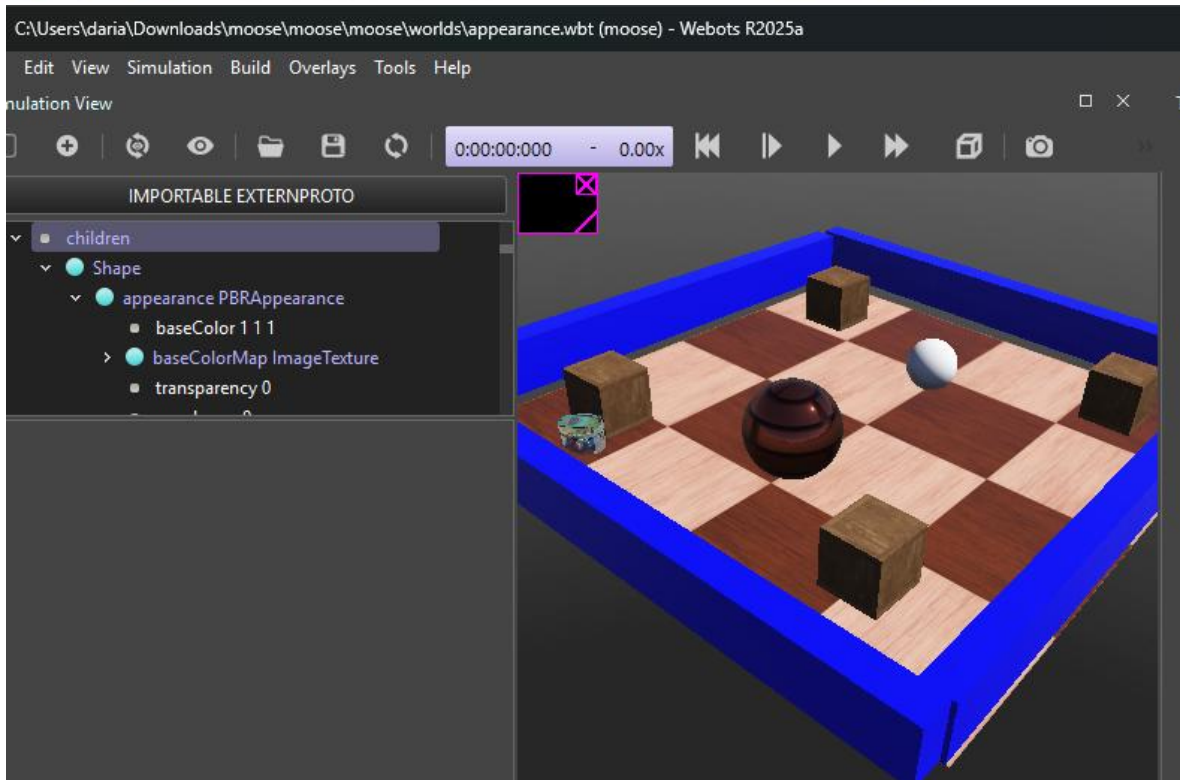
Hands-on #3: Aplicar OldSteel a la pelota



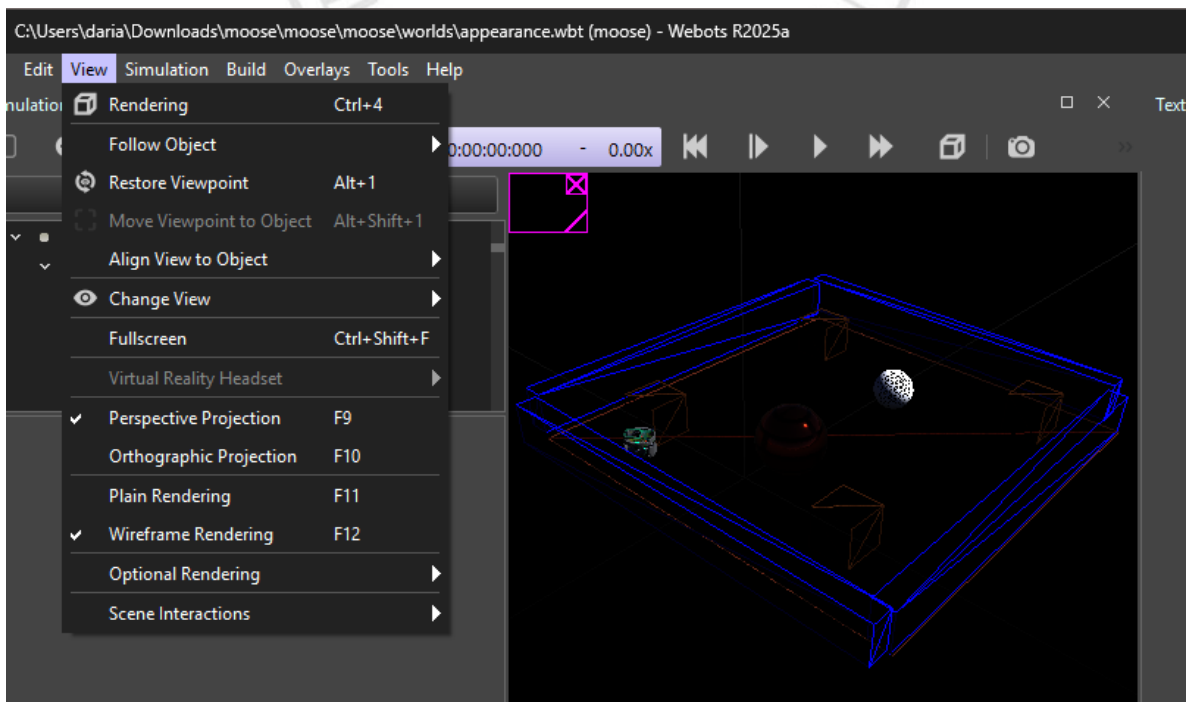


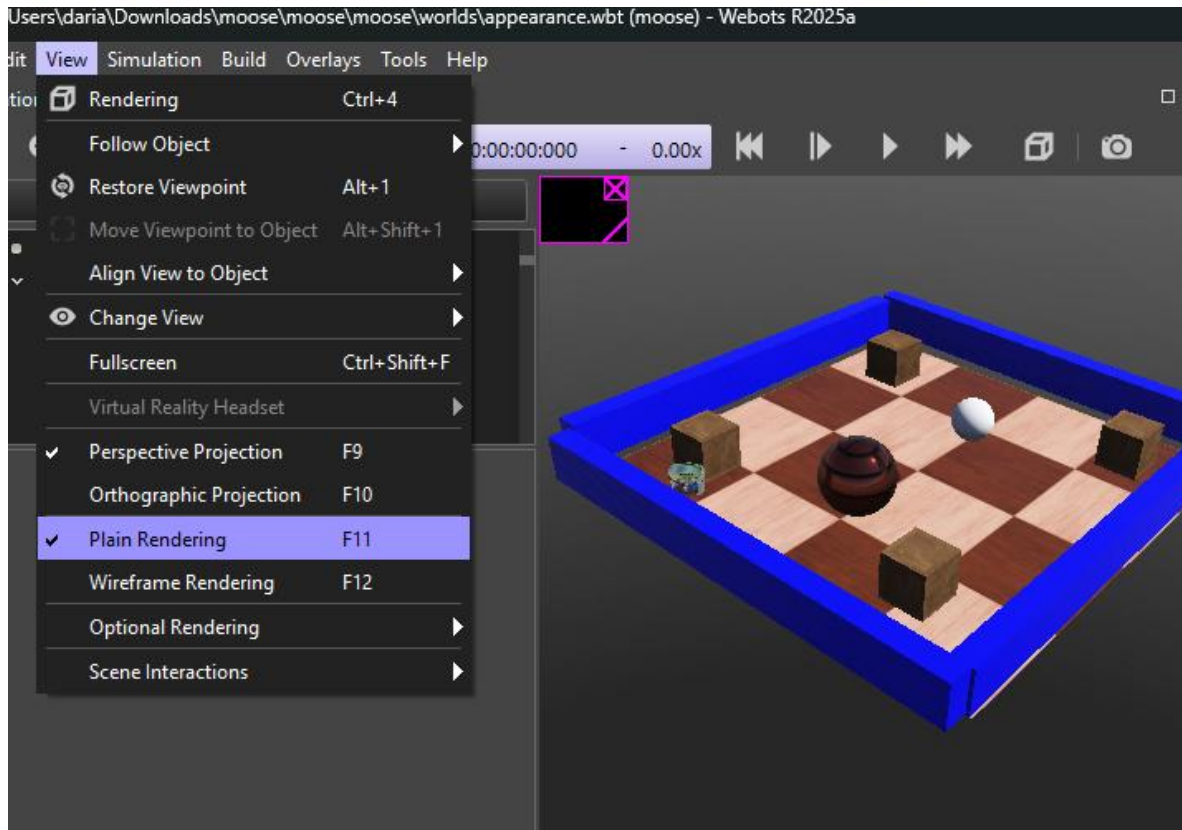
Hands-on #4: Aplicar textura desde disco (red_brick_wall.jpg)





Hands-on #5: Modos de renderizado

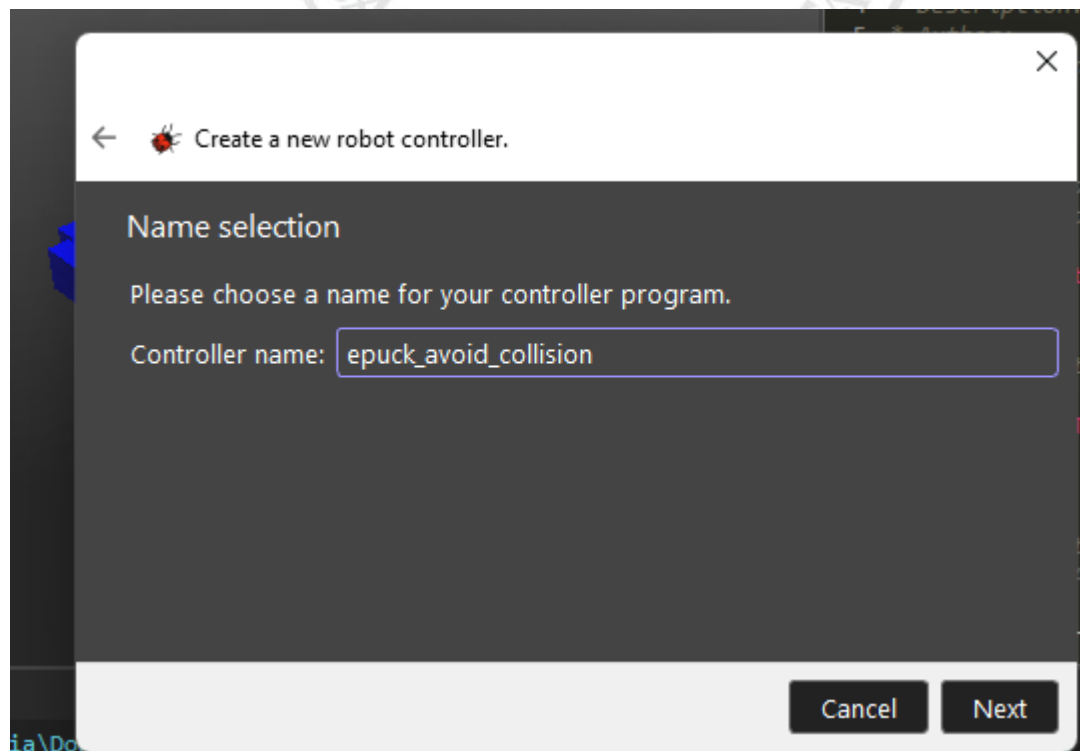
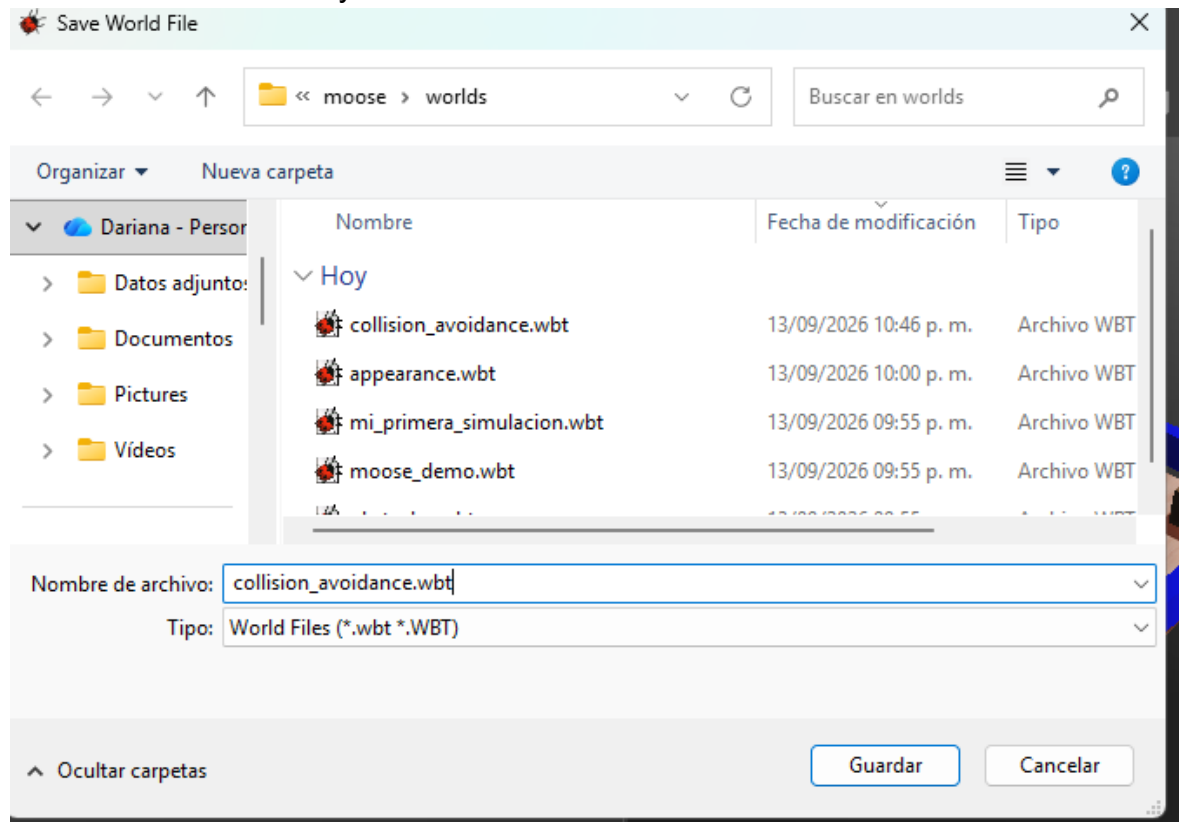


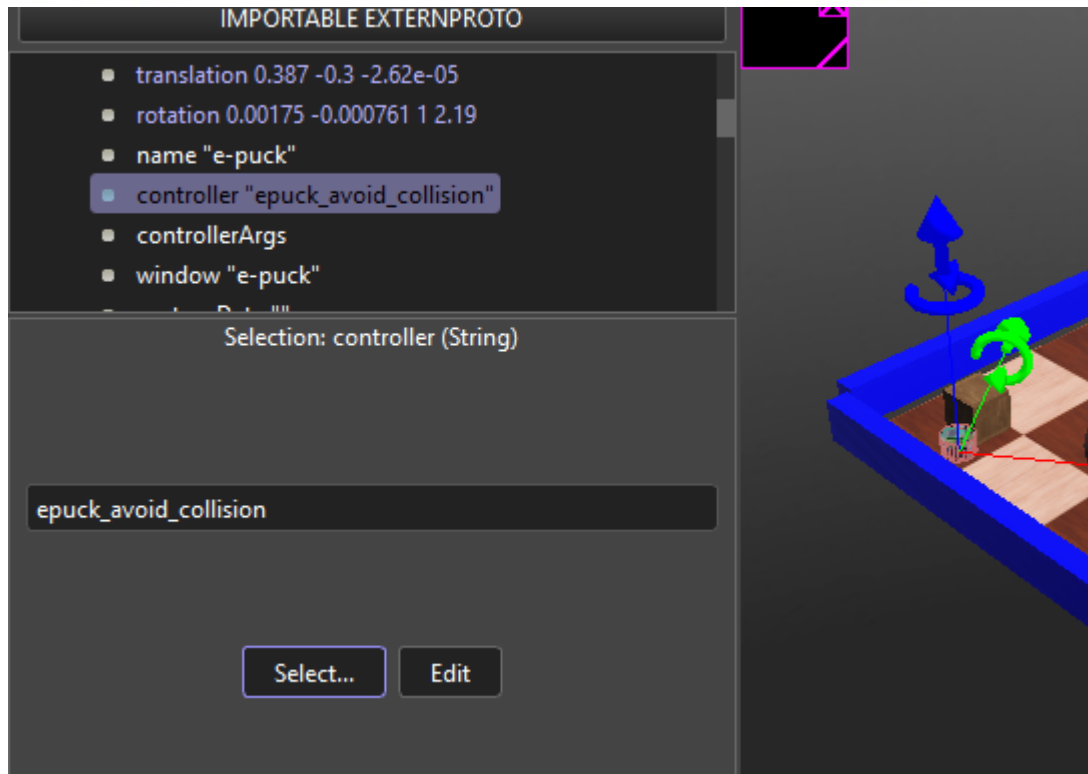


VERDAD, BELLEZA, PROBIDAD

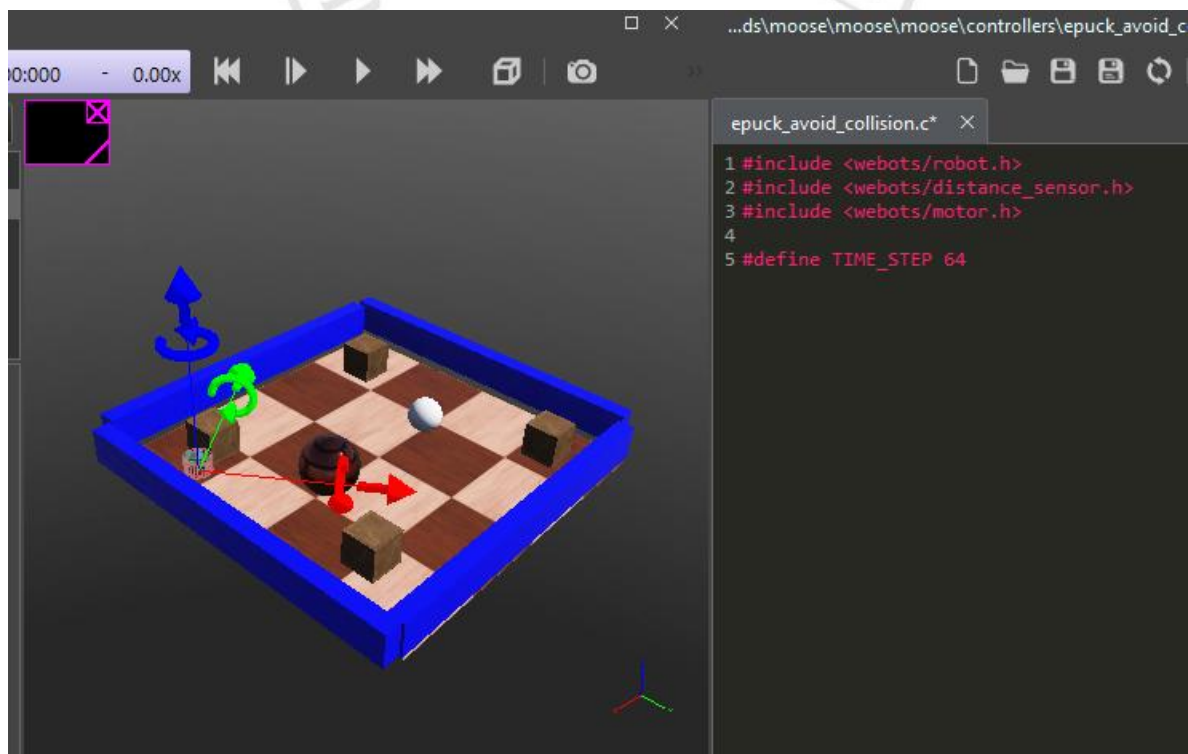
TUTORIAL 4

Hands-on #1: Guardar y crear controlador

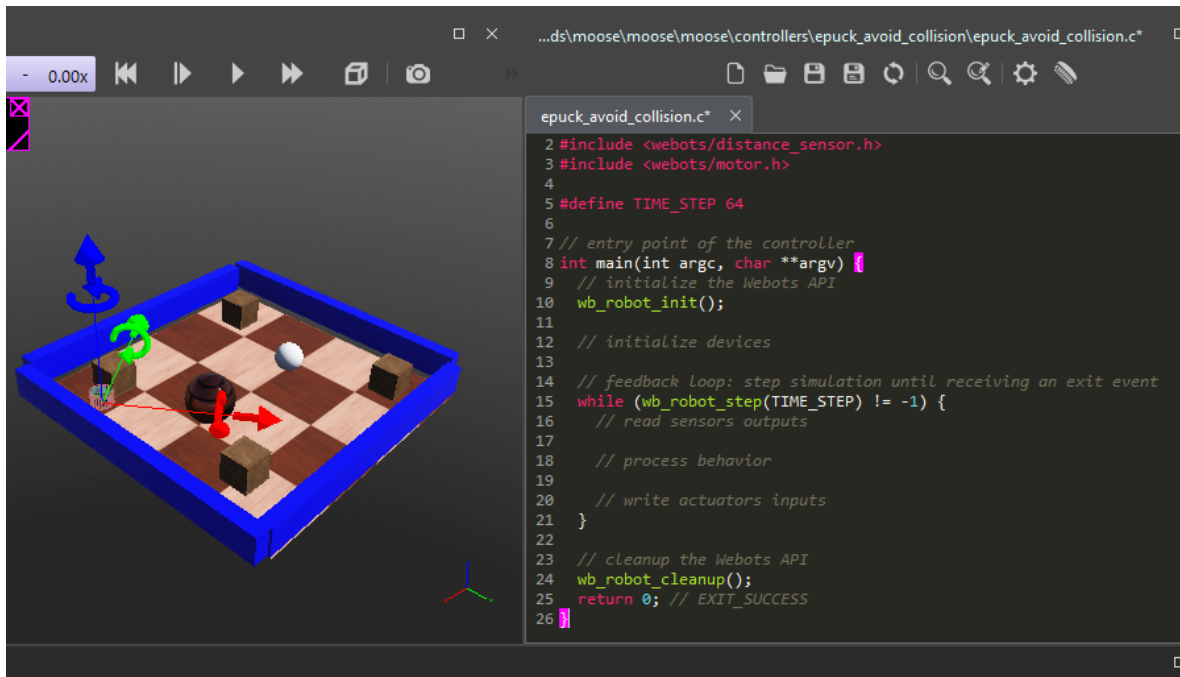




Hands-on #2: Includes y macros



Hands-on #3: Prototipo de main



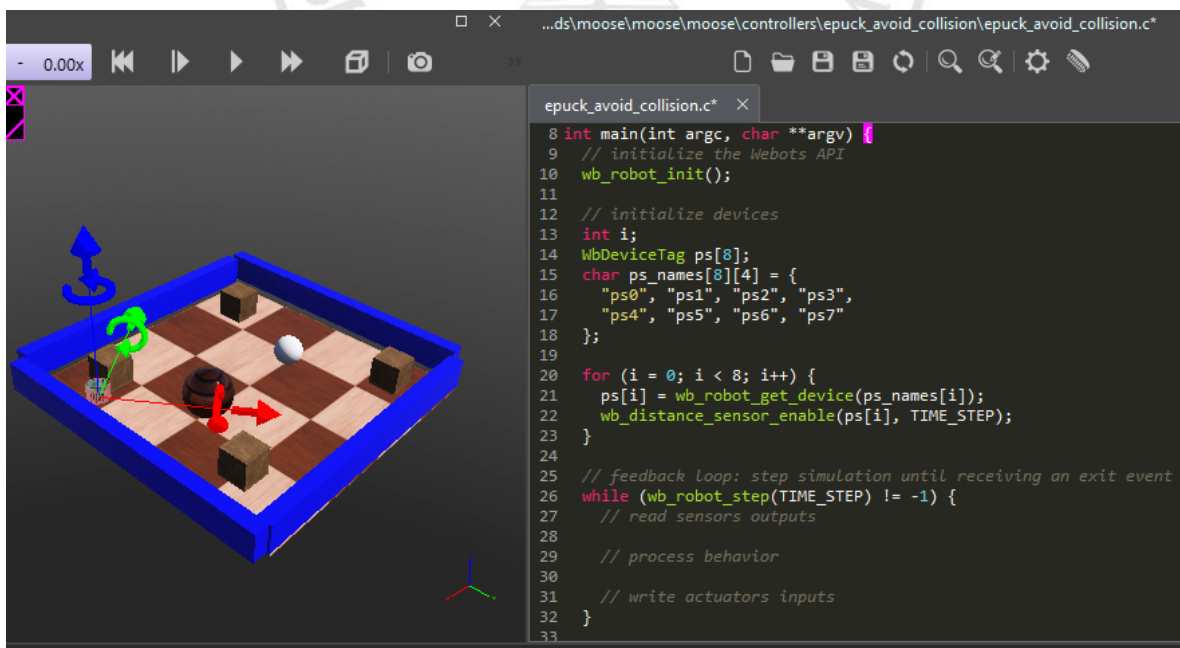
The screenshot shows a 3D simulation of a robot (a small black sphere) on a checkered floor within a blue rectangular arena. There are several brown blocks representing obstacles. A red arrow indicates the robot's movement direction. To the right, a code editor displays the following C++ code:

```

1  ...ds\moose\moose\moose\controllers\epuck_avoid_collision\epuck_avoid_collision.c*
2  #include <webots/distance_sensor.h>
3  #include <webots/motor.h>
4
5  #define TIME_STEP 64
6
7  // entry point of the controller
8  int main(int argc, char **argv) {
9      // initialize the Webots API
10     wb_robot_init();
11
12     // initialize devices
13
14     // feedback loop: step simulation until receiving an exit event
15     while (wb_robot_step(TIME_STEP) != -1) {
16         // read sensors outputs
17
18         // process behavior
19
20         // write actuators inputs
21     }
22
23     // cleanup the Webots API
24     wb_robot_cleanup();
25     return 0; // EXIT_SUCCESS
26 }

```

Hands-on #4: Inicializar dispositivos

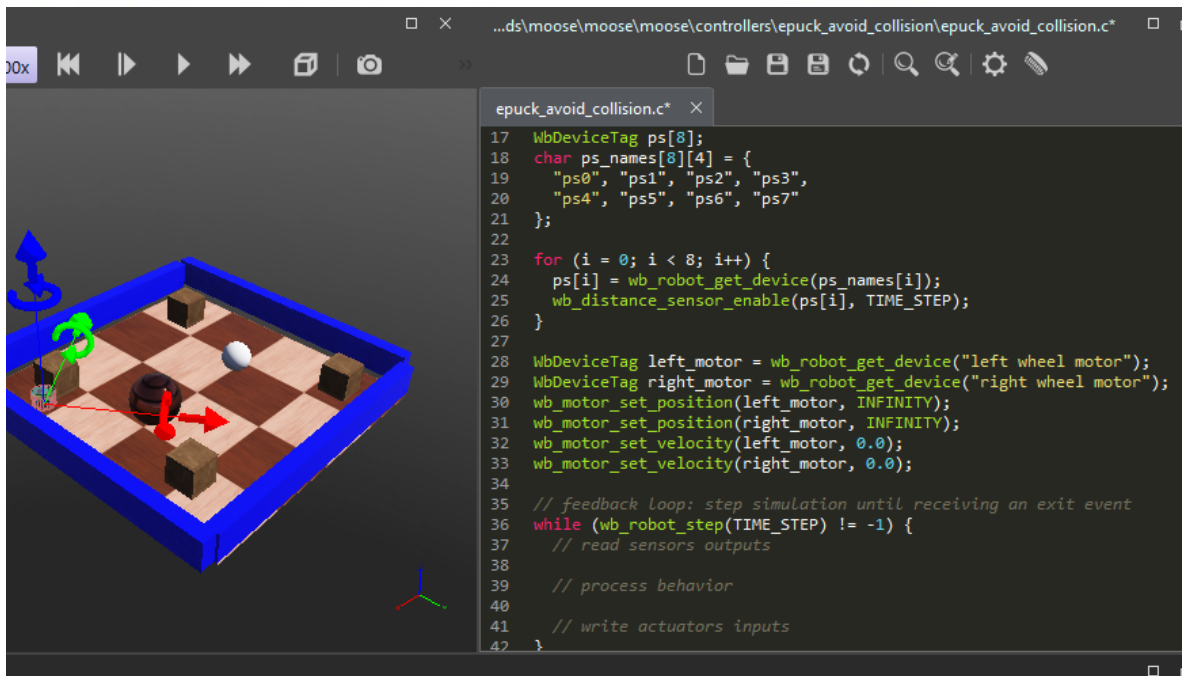


The screenshot shows the same 3D simulation environment. The code editor now displays the following C++ code, which includes the initialization of distance sensors:

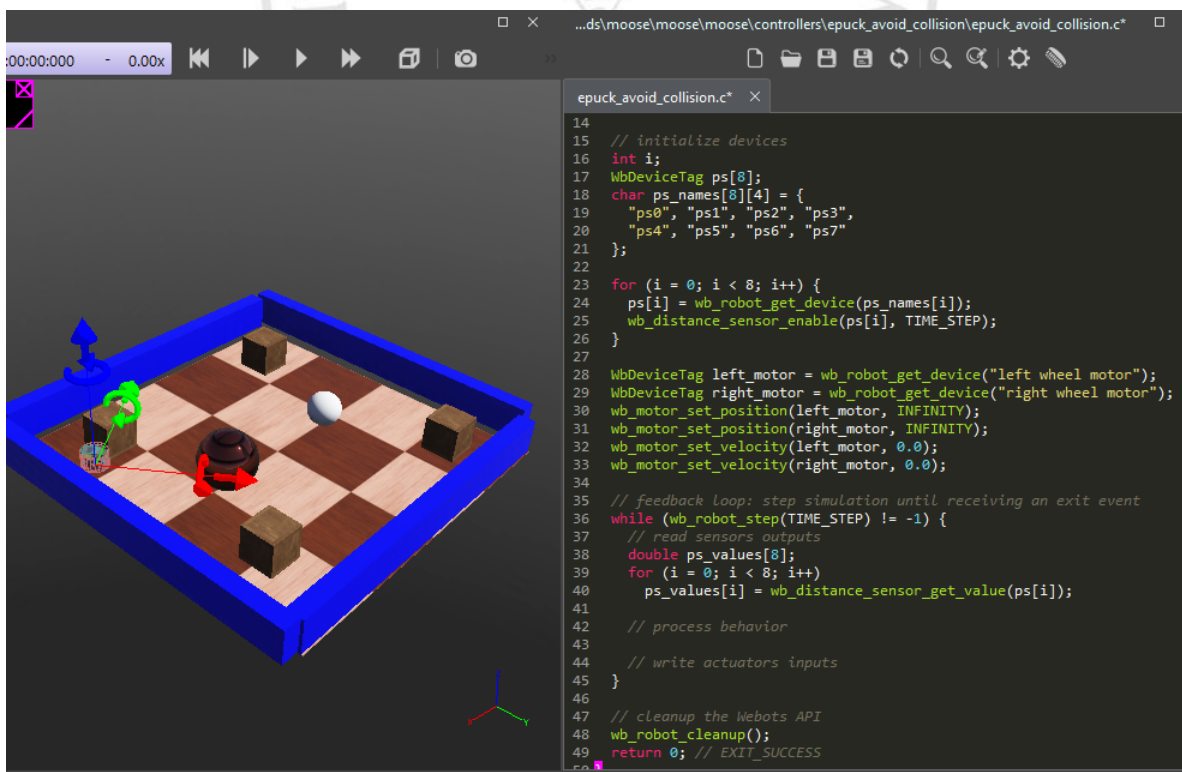
```

1  ...ds\moose\moose\moose\controllers\epuck_avoid_collision\epuck_avoid_collision.c*
2
3  int main(int argc, char **argv) {
4      // initialize the Webots API
5      wb_robot_init();
6
7      // initialize devices
8      int i;
9      WbDeviceTag ps[8];
10     char ps_names[8][4] = {
11         "ps0", "ps1", "ps2", "ps3",
12         "ps4", "ps5", "ps6", "ps7"
13     };
14
15     for (i = 0; i < 8; i++) {
16         ps[i] = wb_robot_get_device(ps_names[i]);
17         wb_distance_sensor_enable(ps[i], TIME_STEP);
18     }
19
20     // feedback loop: step simulation until receiving an exit event
21     while (wb_robot_step(TIME_STEP) != -1) {
22         // read sensors outputs
23
24         // process behavior
25
26         // write actuators inputs
27     }
28 }

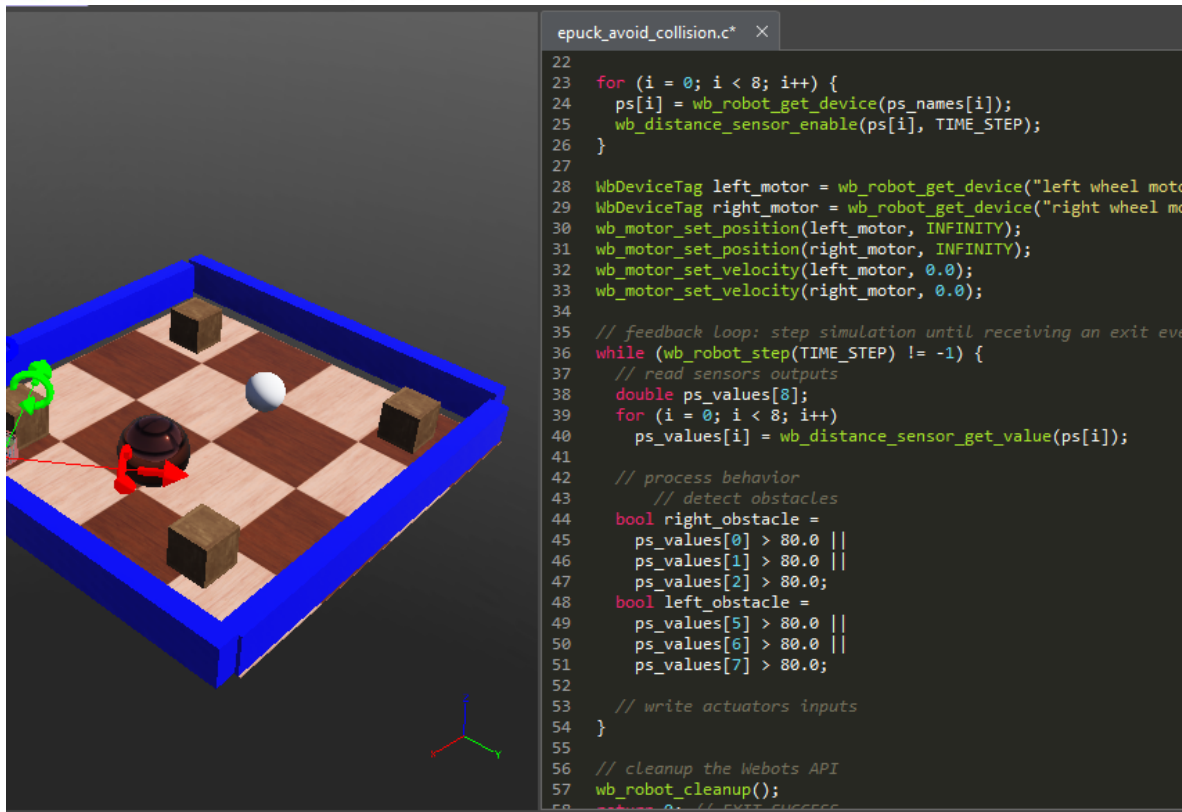
```



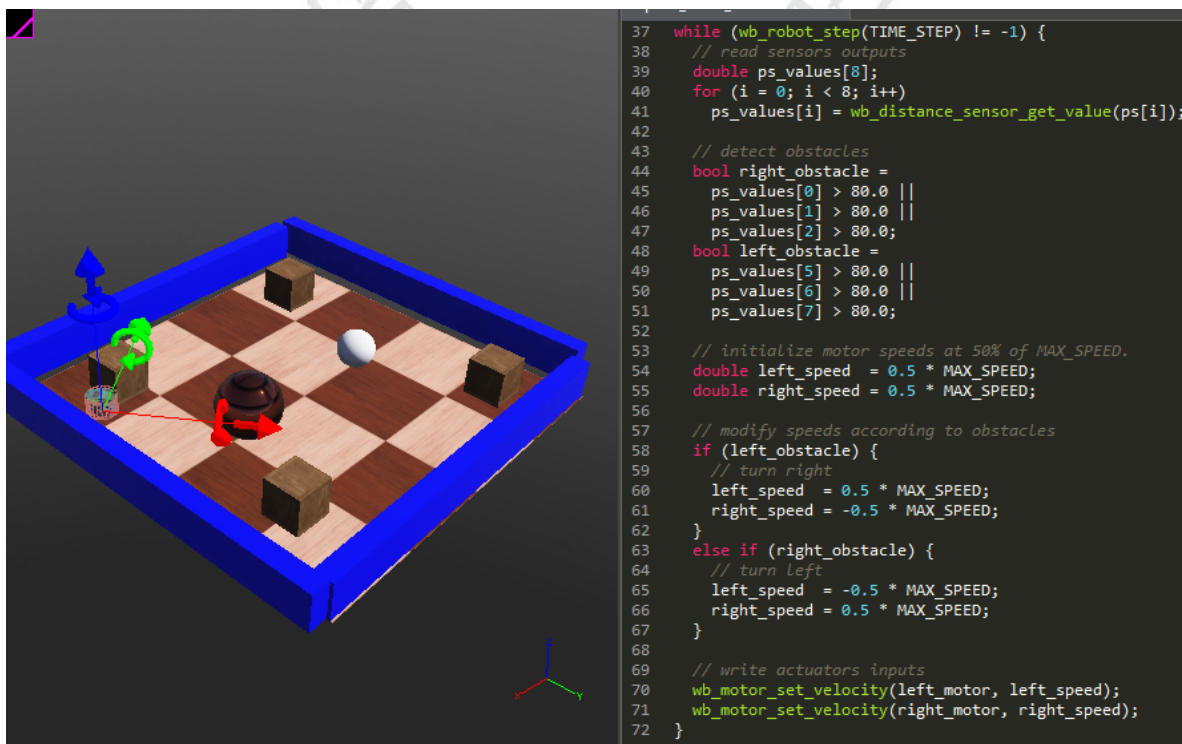
Hands-on #5: Leer sensores en el bucle



Hands-on #6: Detectar obstáculos



Hands-on #7: Controlar los motores



Compilar y Ejecutar

CAUsers\dania\Downloads\moose\moose\moose\worlds\collision_avoidance.wbt (moose) - Webots R2025a

File Edit View Simulation Build Overlays Tools Help

Simulation View

0:00:37.472 - 0.98x

IMPORTABLE EXTERNPROTO

- translation 0.198 0.319 -3.2e-05
- rotation 0.000534 -0.000349 -1 -1.99
- name "e-puck"
- controller "epuck_avoid_collision"
- controllerArgs
- window "e-puck"
- customData ""
- superior FAL SF

epuck_avoid_collision.c

```

7
8 #define MAX_SPEED 6.28
9
10 // entry point of the controller
11 int main(int argc, char **argv)
12 // initialize the Webots API
13 wb_robot_init();
14
15 // internal variables
16 int i;
17 WbDeviceTag ps[8];
18 char ps_names[8][4] = {
19     "ps0", "ps1", "ps2", "ps3",
20     "ps4", "ps5", "ps6", "ps7"
21 };
22
23 // initialize devices
24 for (i = 0; i < 8; i++) {
25     ps[i] = wb_robot_get_device(ps_names[i]);
26     wb_distance_sensor_enable(ps[i], TIME_STEP);
27 }
28
29 WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
30 WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
31 wb_motor_set_position(left_motor, INFINITY);
32 wb_motor_set_position(right_motor, INFINITY);
33 wb_motor_set_velocity(left_motor, 0.0);
    
```

Console - All

```

make -j 4
# updating epuck_avoid_collision.d
# compiling epuck_avoid_collision.c
# linking epuck_avoid_collision.exe
# copying to epuck_avoid_collision.exe
Build finished.
make -j 4
make: Nothing to be done for 'all'.
Nothing to be done for build targets.
    
```

CAUsers\dania\Downloads\moose\moose\moose\worlds\collision_avoidance.wbt (moose) - Webots R2025a

File Edit View Simulation Build Overlays Tools Help

Simulation View

0:00:53.792 - 0.97x

IMPORTABLE EXTERNPROTO

- translation -0.415 -0.00889 -3.19e-05
- rotation -0.00113 -0.00168 -1 1.19
- name "e-puck"
- controller "epuck_avoid_collision"
- controllerArgs
- window "e-puck"
- customData ""
- superior FAL SF

epuck_avoid_collision.c

```

7
8 #define MAX_SPEED 6.28
9
10 // entry point of the controller
11 int main(int argc, char **argv)
12 // initialize the Webots API
13 wb_robot_init();
14
15 // internal variables
16 int i;
17 WbDeviceTag ps[8];
18 char ps_names[8][4] = {
19     "ps0", "ps1", "ps2", "ps3",
20     "ps4", "ps5", "ps6", "ps7"
21 };
22
23 // initialize devices
24 for (i = 0; i < 8; i++) {
25     ps[i] = wb_robot_get_device(ps_names[i]);
26     wb_distance_sensor_enable(ps[i], TIME_STEP);
27 }
28
29 WbDeviceTag left_motor = wb_robot_get_device("left wheel motor");
30 WbDeviceTag right_motor = wb_robot_get_device("right wheel motor");
31 wb_motor_set_position(left_motor, INFINITY);
32 wb_motor_set_position(right_motor, INFINITY);
33 wb_motor_set_velocity(left_motor, 0.0);
    
```

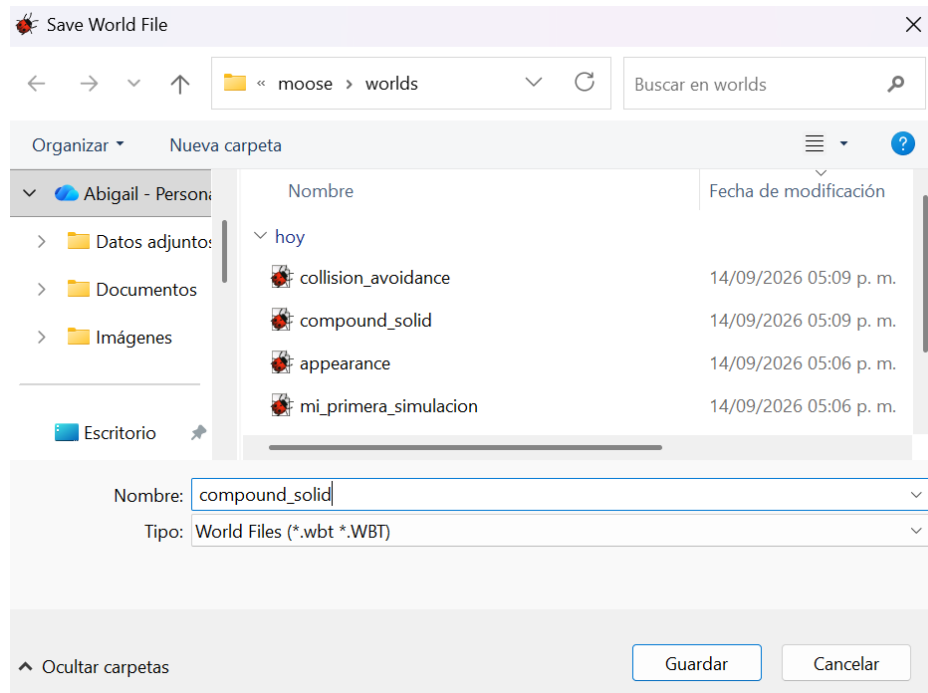
Console - All

```

make -j 4
# updating epuck_avoid_collision.d
# compiling epuck_avoid_collision.c
# linking epuck_avoid_collision.exe
# copying to epuck_avoid_collision.exe
Build finished.
make -j 4
make: Nothing to be done for 'all'.
Nothing to be done for build targets.
    
```

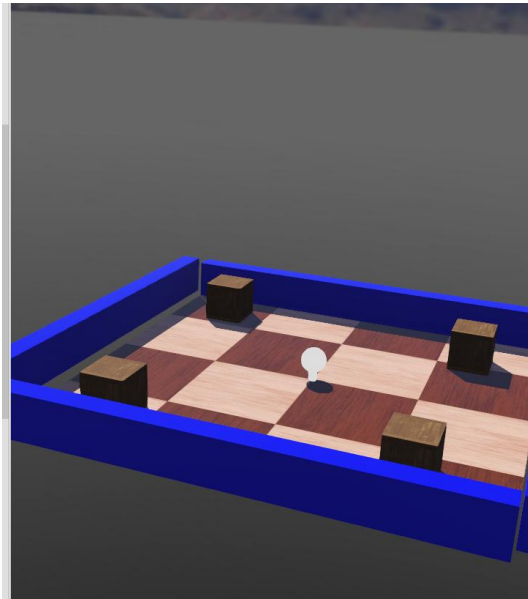
Tutorial #5

1 Creación de una nueva simulación a partir del tutorial anterior. Llamado compound_solid.wbt

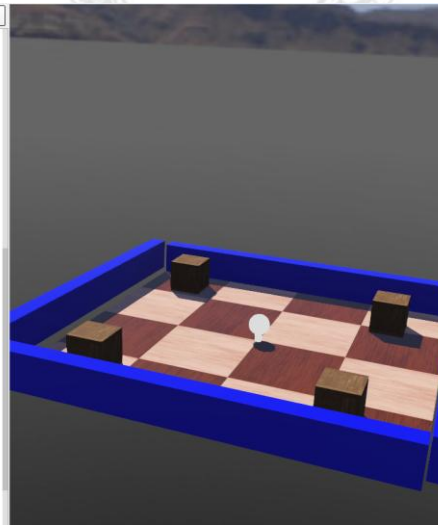


2: Crea la mancuerna siguiendo la figura. Crea primero el mango sin colocarlo en un nodo de pose (así el eje del mango tendrá la misma dirección que el eje z del sólido). El mango debe tener una longitud de 0,1 m y un radio de 0,01 m. Los pesos deben tener un radio de 0,03 m y una subdivisión de 2.

- Solid "dumbbell"
 - translation 0 0 0
 - rotation 0 0 1 0
- children
 - Pose
 - translation 0 0 -0.05
 - rotation 0 0 1 0
 - children
 - Shape
 - appearance NULL
 - geometry Sphere
 - radius 0.03
 - subdivision 2
 - ico TRUE
 - castShadows TRUE
 - isPickable TRUE
 - translationStep 0.01
 - rotationStep 0.262
- Pose
 - translation 0 0 0.05
 - rotation 0 0 1 0
- children
 - Shape
 - appearance NULL
 - geometry Sphere
 - radius 0.03
 - subdivision 2
 - ico TRUE
 - castShadows TRUE



- IMPORTABLE EXTERNPROTO
- isPickable TRUE
 - translationStep 0.01
 - rotationStep 0.262
 - Pose
 - translation 0 0 0.05
 - rotation 0 0 1 0
 - children
 - Shape
 - appearance NULL
 - geometry Sphere
 - radius 0.03
 - subdivision 2
 - ico TRUE
 - castShadows TRUE
 - isPickable TRUE
 - translationStep 0.01
 - rotationStep 0.262
 - Shape
 - appearance NULL
 - geometry Cylinder
 - bottom TRUE
 - height 0.1
 - radius 0.01
 - side TRUE
 - top TRUE
 - subdivision 36
 - castShadows TRUE
 - isPickable TRUE
 - name "dumbbell"



3. Configuración de la masa de la mancuerna a 2 kg.

- boundingObject NULL
- physics Physics
 - density 2
 - mass -1
 - centerOfMass

4. Modificación del centro de masa de la mancuerna sobre el eje Z.

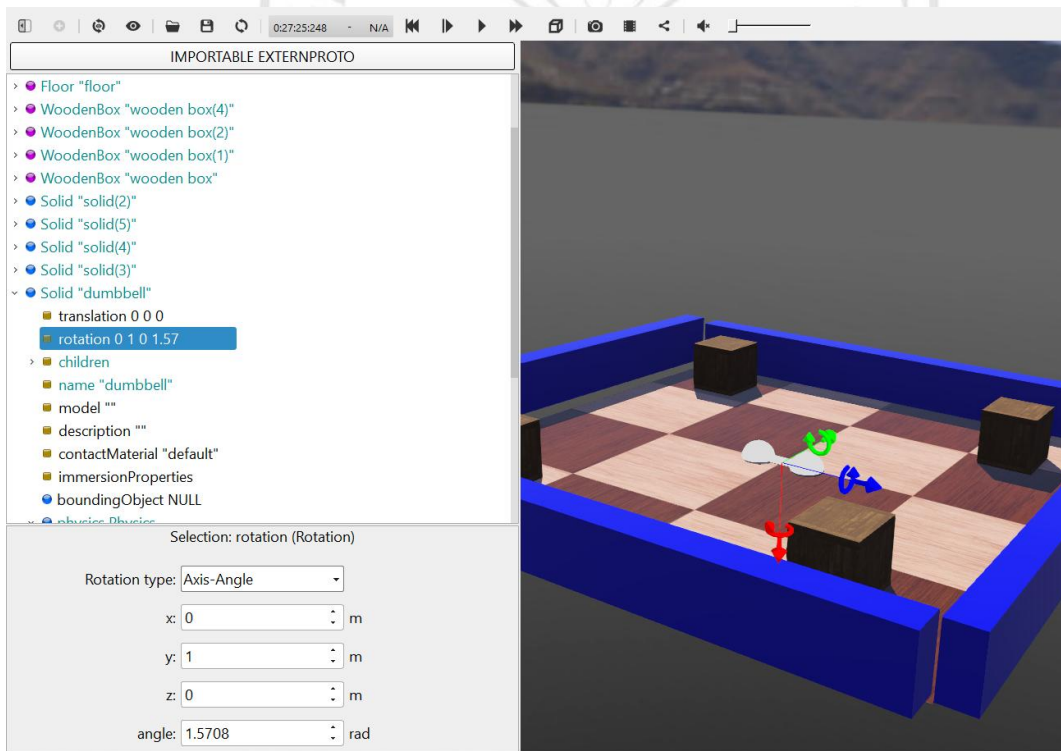
Selection: centerOfMass (Vector3 #1)

x: 0 m

y: 0 m

z: 0.01 m

5. Rotación de la mancuerna para colocar el eje del mango paralelo al suelo.



6. Configuración de las propiedades de contacto para modificar la fricción de la mancuerna.

contactProperties

• ContactProperties

■ material1 "dumbbell"

■ material2 "default"

▼ ■ coulombFriction

0

■ frictionRotation 0 0

■ rollingFriction 0 0 0

■ bounce 0.5

File Edit View Simulation Build Overlays Tools Help

Simulation View

0:27:25:248 N/A

IMPORTABLE EXTERNPROTO

■ translation 0 0 -0.05

■ rotation 0 0 1 0

▼ ■ children

▼ • Shape

• appearance NULL

• geometry DEF G2 Sph

■ castShadows TRUE

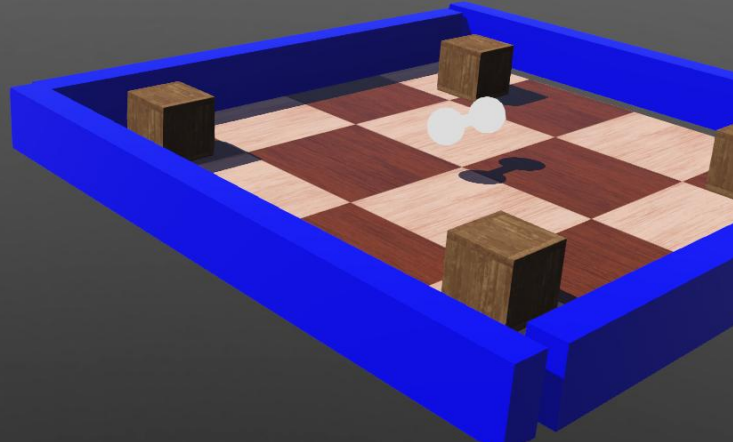
■ isPickable TRUE

■ translationStep 0.01

■ rotationStep 0.262

▼ • Pose

■ translation 0 0 0.05



7. Configuración de parámetros adicionales de física del sólido.

Learning Object Group

physics Physics

density -1

mass 2

> centerOfMass

inertiaMatrix

▼ damping Damping

linear 0.2

angular 0.2

▼ physics Physics

density -1

mass 2

> centerOfMass

▼ inertiaMatrix

1 1 1

0 0 0

▼ WorldInfo

info

title ""

window "<none>"

gravity 9.81

CFM 1e-05

ERP 0.2

physics "<none>"

basicTimeStep 16

FPS 60

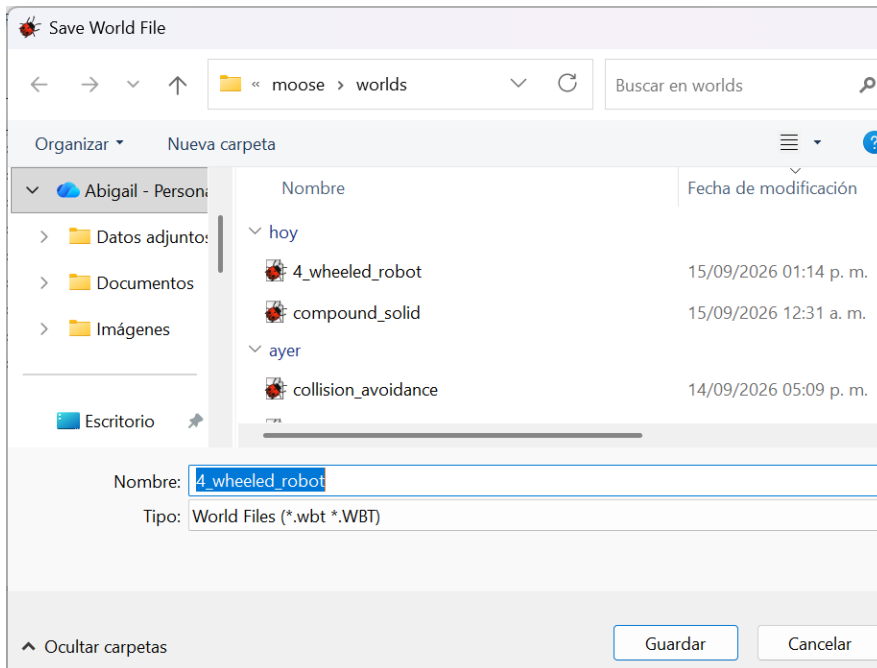
optimalThreadCount 1

physicsDisableTime 1

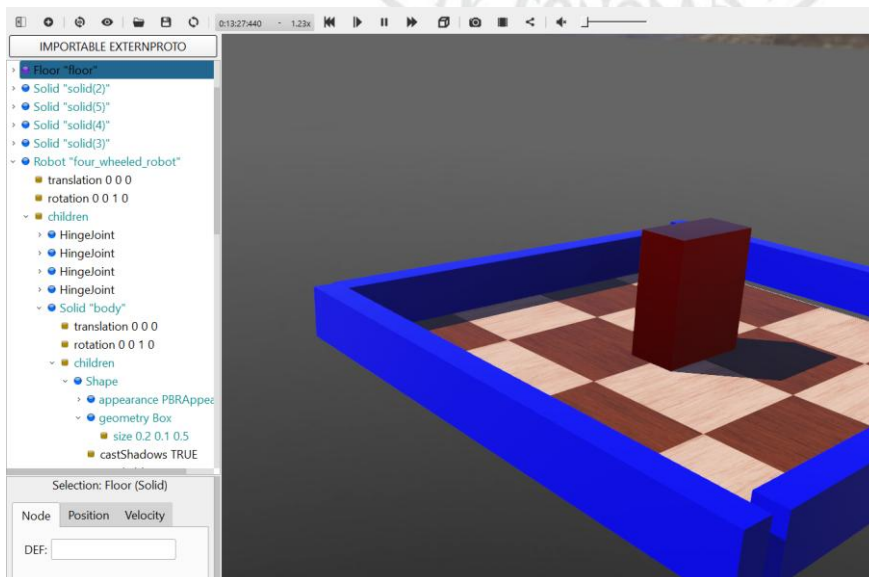
VERDAD, BELLEZA, PROBABIDAD

Tutorial #6

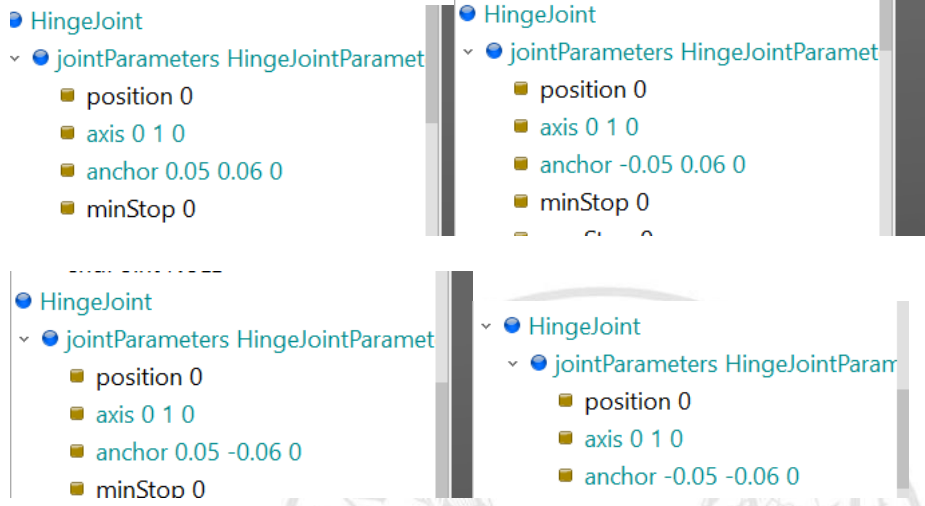
1 Creación del archivo 4_wheeled_robot



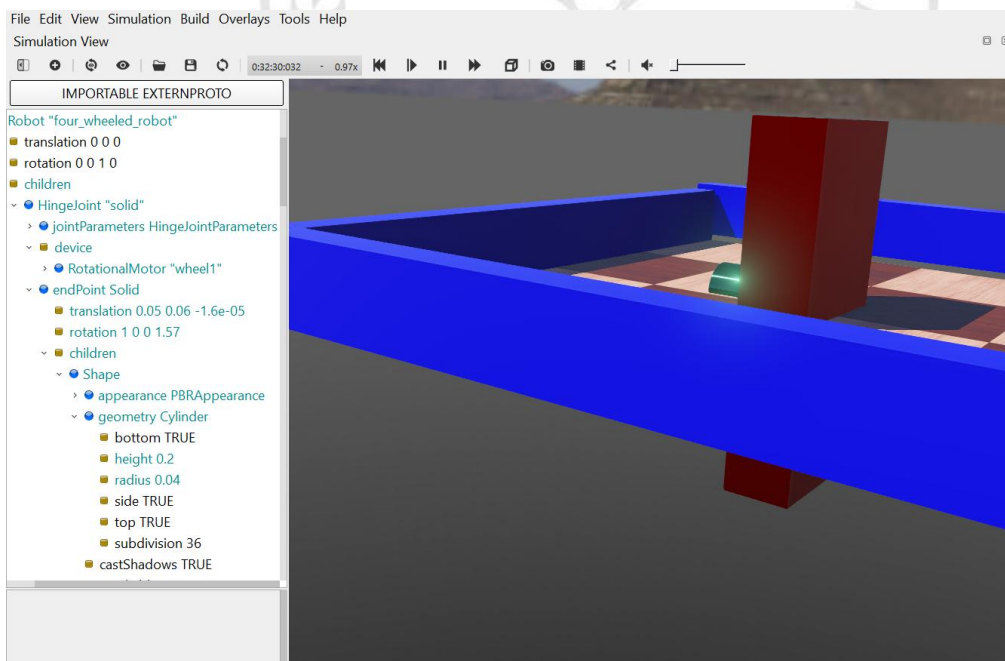
2. Creación de la estructura principal del robot de cuatro ruedas.



3. Configuración de la articulación de las ruedas y sus ejes de rotación.

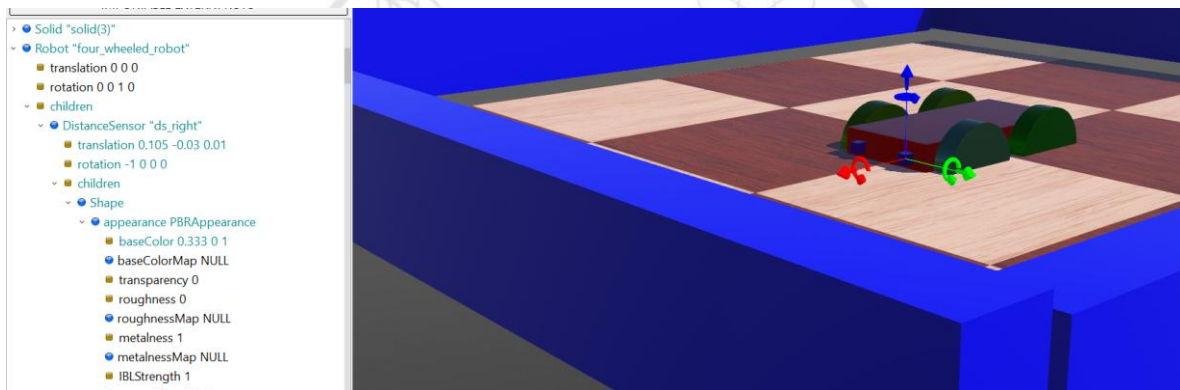


4. Construcción de las cuatro ruedas, motores y elementos físicos del robot.



5. Incorporación y orientación de los dos sensores de distancia en la parte frontal del robot.

- ▼ HingeJoint
 - ▼ jointParameters HingeJointParam
 - position 0
 - axis 0 1 0
 - anchor -0.05 -0.06 0

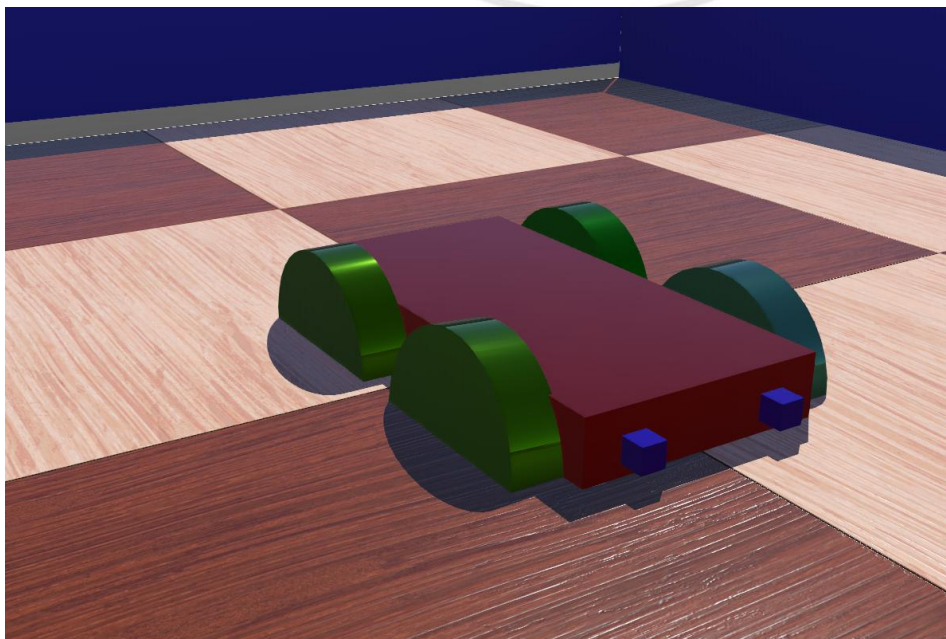


VERDAD, BELLEZA, PROBIDAD

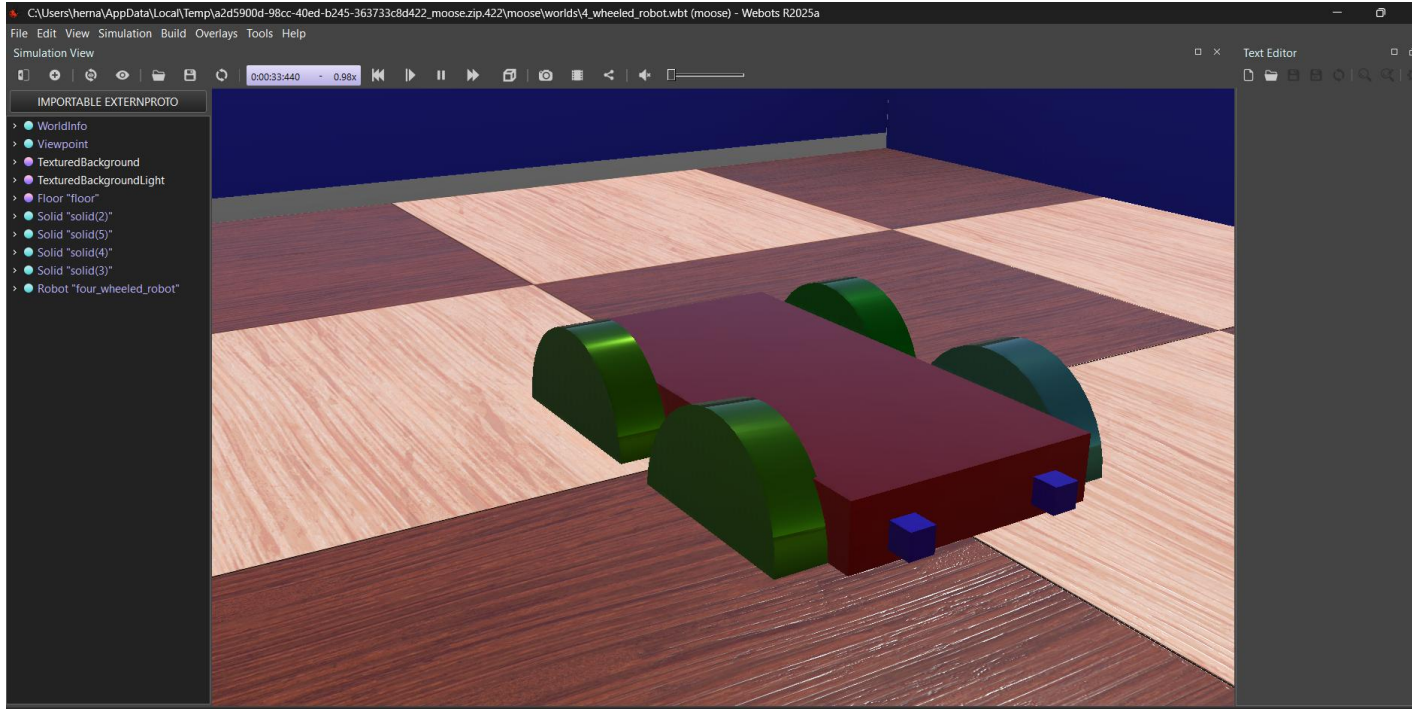
6. Implementación del controlador para avanzar y evitar obstáculos mediante sensores de distancia.

```
File Edit View Simulation Build Overlays Tools Help
C:\Users\abyhd\Downloads\moose\controllers\four_wheeled_collision_avoidance\four_wheeled_collision_avoidance.c
four_wheeled_collision_avoidance.c
1#include <webots/distance_sensor.h>
2#include <webots/motor.h>
3#include <webots/robot.h>
4
5#define TIME_STEP 64
6
7int main(int argc, char **argv) {
8    wb_robot_init();
9    int i;
10    bool avoid_obstacle_counter = 0;
11    WbDeviceTag ds[2];
12    char ds_names[2][10] = {"ds_left", "ds_right"};
13    for (i = 0; i < 2; i++) {
14        ds[i] = wb_robot_get_device(ds_names[i]);
15        wb_distance_sensor_enable(ds[i], TIME_STEP);
16    }
17    WbDeviceTag wheels[4];
18    char wheels_names[4][8] = {"wheel1", "wheel2", "wheel3", "wheel4"};
19    for (i = 0; i < 4; i++) {
20        wheels[i] = wb_robot_get_device(wheels_names[i]);
21        wb_motor_set_position(wheels[i], INFINITY);
22    }
23    while (wb_robot_step(TIME_STEP) != -1) {
24        double left_speed = 1.0;
25        double right_speed = 1.0;
26        if (avoid_obstacle_counter > 0) {
27            avoid_obstacle_counter--;
28            left_speed = 1.0;
29            right_speed = -1.0;
30        } else { // read sensors
31            double ds_values[2];
32            for (i = 0; i < 2; i++)
33                ds_values[i] = wb_distance_sensor_get_value(ds[i]);
34            if (ds_values[0] < 950.0 || ds_values[1] < 950.0)
35                avoid_obstacle_counter = 100;
36        }
37    }
38}
```

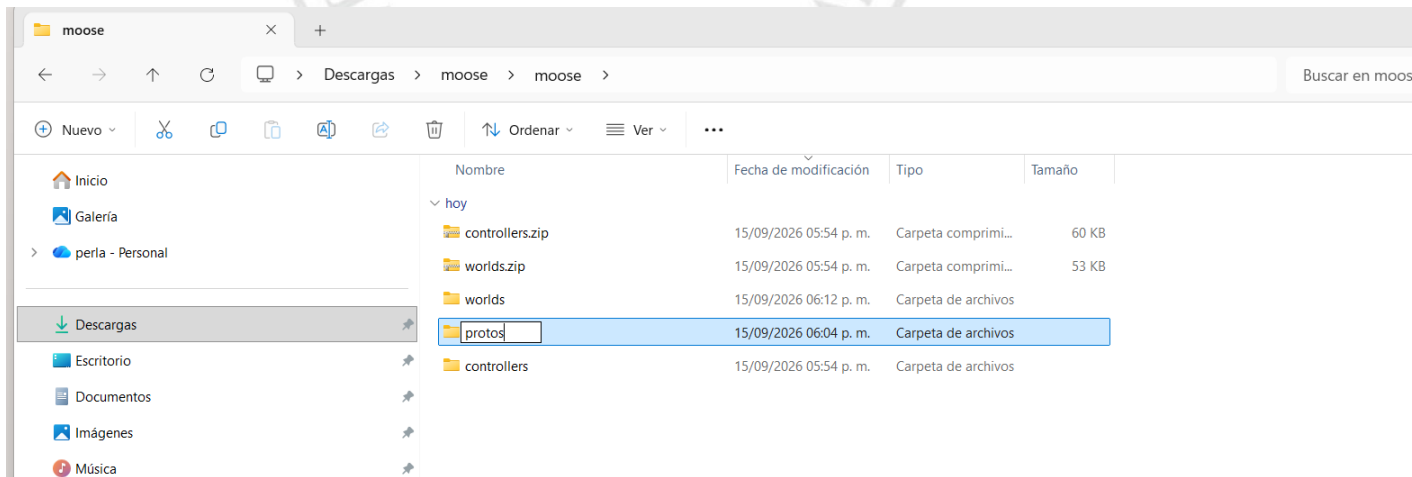
Vista 3D del robot de las 4 ruedas.

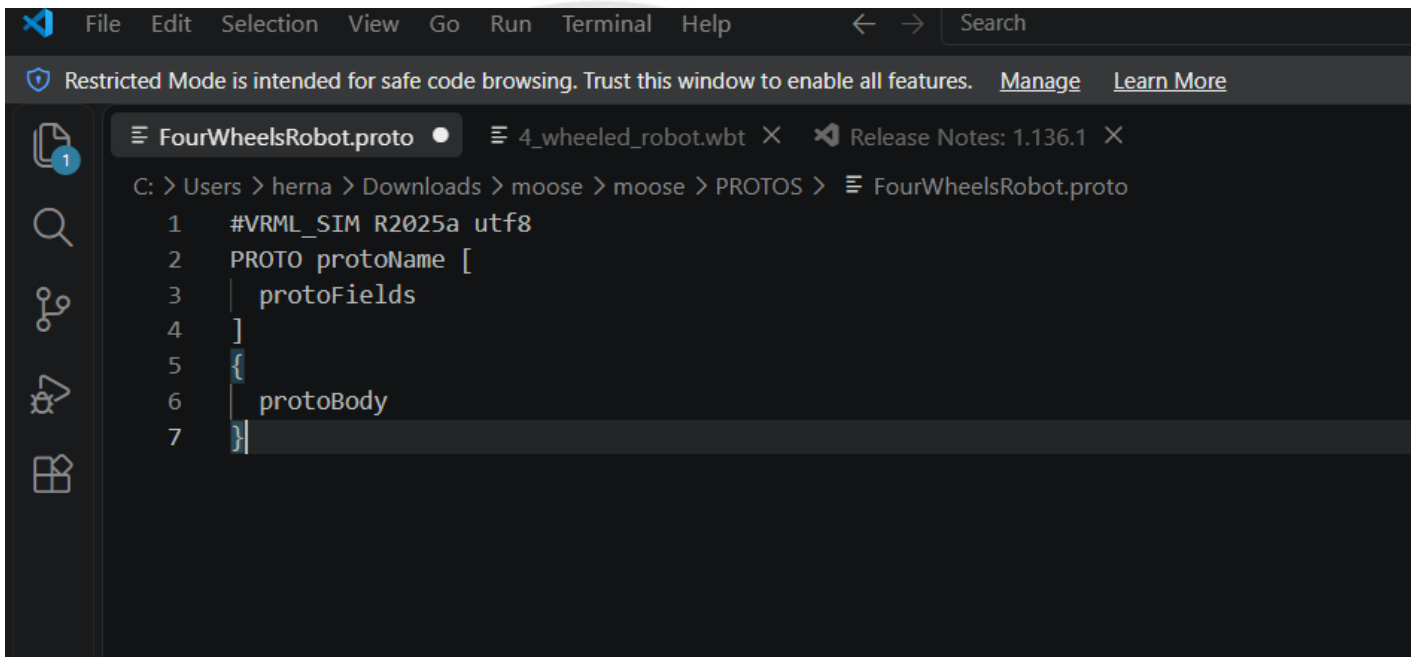
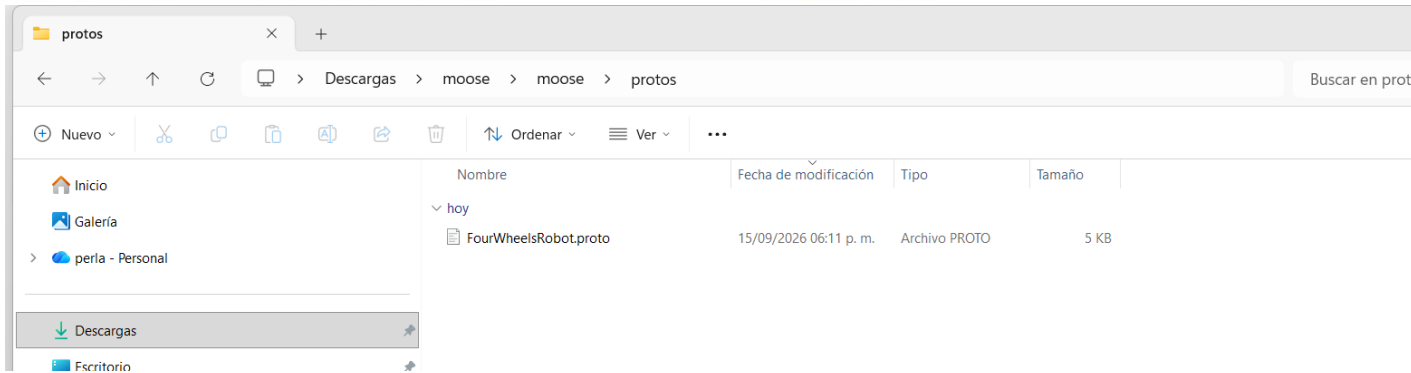


Tutorial #7



PRACTICA1:





PRACTICA2

VERDAD, BELLEZA, PROBIDAD

```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

C:\> Users > herna > Downloads > moose > moose > worlds > 4_wheeled_robot.wbt

1 |VRML_SIM R2025a utf8
2
3 |EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/backgrounds/protos/TexturedBackground.proto"
4 |EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/backgrounds/protos/TexturedBackgroundLight.proto"
5 |EXTERNPROTO "https://raw.githubusercontent.com/cyberbotics/webots/R2025a/projects/objects/floors/protos/Floor.proto"
6
7 |WorldInfo {
8 |  basicTimeStep 16
9 |  contactProperties [
10 |    contactProperties {
11 |      material1 "dumbbell"
12 |      coulombFriction [
13 |        0
14 |      ]
15 |    }
16 |  ]
17 |}
18 |Viewpoint {
19 |  orientation -0.18063673915593556 0.05119943019511437 0.9822163645627201 2.5985474338324095
20 |  position 0.3594462891104058 -0.2540275738946651 0.18249976765107534
21 |}
22 |TexturedBackground {
23 |}
24 |TexturedBackgroundLight {
25 |}
26 |Floor {
27 |  size 1 1
28 |}
29 |Solid {
30 |  translation 0 0.47 0.05
31 |  rotation 1 0 0 -1.5707953071795862
32 |  children [
33 |    DEF WALL_SHAPE Shape {
34 |      appearance PBRAppearance {
35 |        baseColor 0 0 1
```

```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

C:\> Users > herna > Downloads > moose > moose > worlds > 4_wheeled_robot.wbt

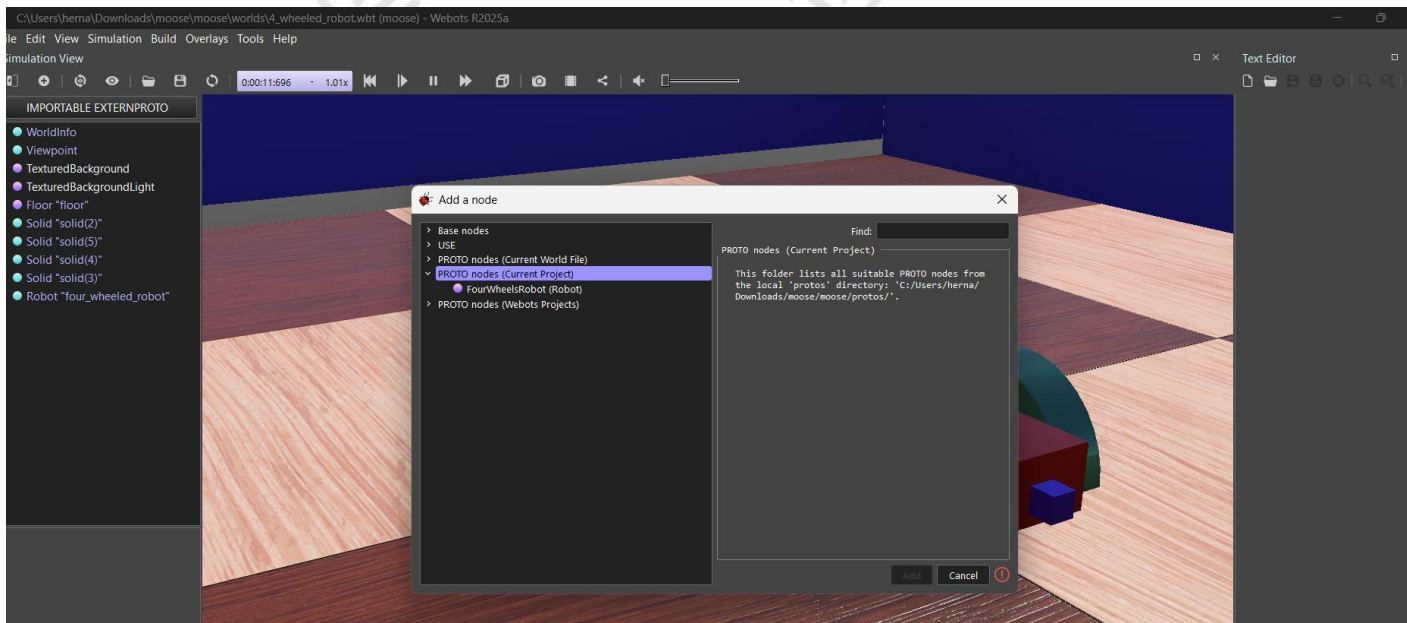
80 |Solid {
94 |  name "solid(3)"
95 |  boundingObject USE WALL_SHAPE
96 |}
97 |Robot {
98 |  children [
99 |    DistanceSensor {
100 |      translation 0.105 -0.03 0.01
101 |      rotation -1 0 0 0
102 |      children [
103 |        Shape {
104 |          appearance PBRAppearance {
105 |            baseColor 0.333333 0 1
106 |          }
107 |          geometry Box {
108 |            size 0.01 0.01 0.01
109 |          }
110 |        }
111 |      ]
112 |      name "ds_right"
113 |    }
114 |    DistanceSensor {
115 |      translation 0.105 0.03 0.01
116 |      rotation 1 0 0 0
117 |      children [
118 |        Shape {
119 |          appearance PBRAppearance {
120 |            baseColor 0.333333 0 1
121 |          }
122 |          geometry Box {
123 |            size 0.01 0.01 0.01
124 |          }
125 |        }
126 |      ]
127 |      name "ds_left"
```

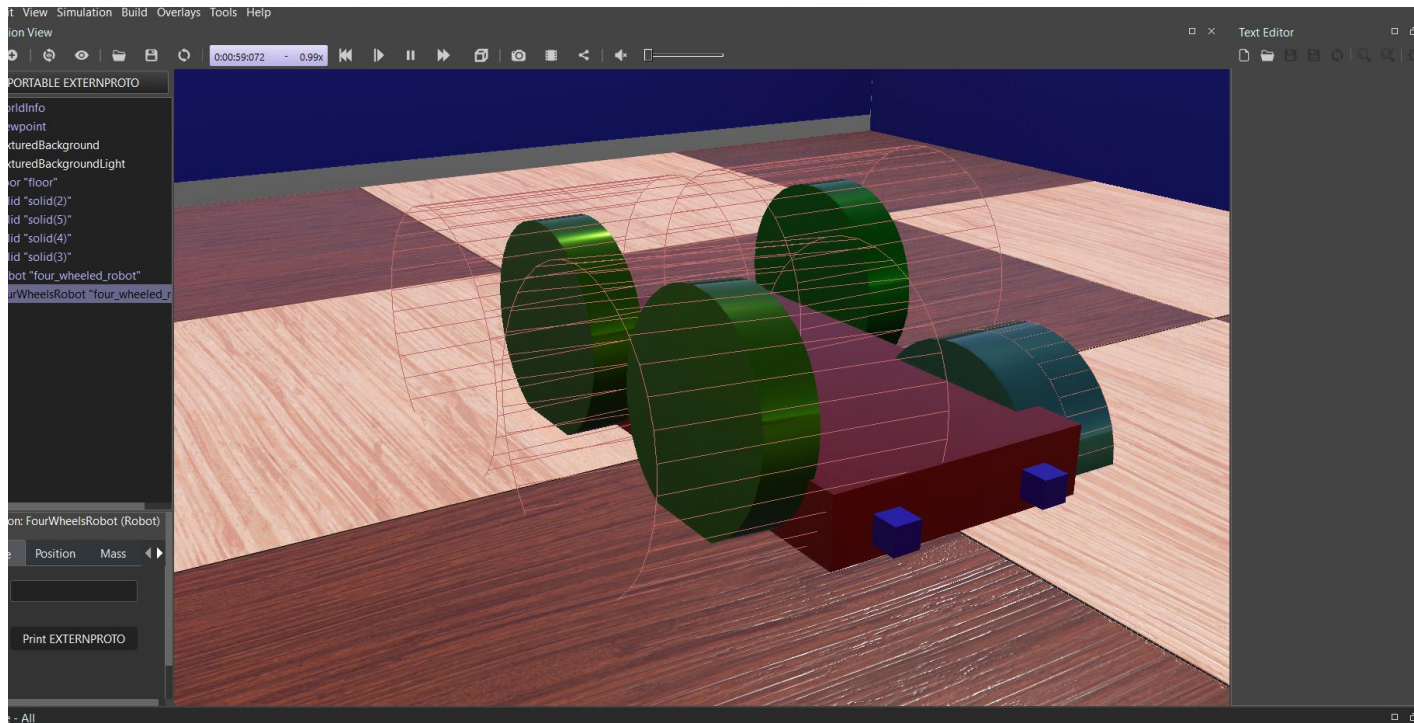
```

1  C:\Users\herna\Downloads\moose\moose\worlds\4_wheeled_robot.wbt - Webots R2025a
2  File Edit Selection View Go Run Terminal Help
3  Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
4  FourWheelsRobot.proto 4_wheeled_robot.wbt Release Notes: 1.136.1
5  C:\Users\herna\Downloads\moose\moose\worlds\4_wheeled_robot.wbt - Webots R2025a
6  Robot {
7    children [
141    HingeJoint {
151      endPoint Solid {
170        physics Physics {
171        }
172        linearVelocity 0 0 -9.430941613671781e-11
173        angularVelocity -3.644184555016256e-15 0 0
174      }
175    }
176    Solid {
177      children [
178      Shape {
179        appearance PBRAppearance {
180          baseColor 1 0 0
181        }
182        geometry Box {
183          size 0.2 0.1 0.05
184        }
185      }
186    ]
187    name "body"
188  }
189  ]
190  name "four_wheeled_robot"
191  controller "four_wheeled_collision_avoidance"
192  controllerArgs [
193  ""
194  ]
195  }
196  }

```

PRACTICA3





PRACTICA4

```
File Edit Selection View Go Run Terminal Help
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

4_wheeled_robot.wbt x FourWheelsRobot.proto x Release Notes: 1.136.1 x
C: > Users > herna > Downloads > moose > protos > FourWheelsRobot.proto
1 #VRML_SIM R2025a utf8
2 PROTO FourWheelsRobot [
3   field SFVec3f translation 0 0 0
4   field SFRotation rotation 0 0 1 0
5   field SFFloat bodyMass 1
6 ]
7 {
8   Robot {
9     children [
10      DistanceSensor {
11        translation 0.105 -0.03 0.01
12        rotation -1 0 0 0
13        children [
14          Shape {
15            appearance PBRAppearance {
16              baseColor 0.333333 0 1
17            }
18            geometry Box {
```

#VRML_SIM R2025a utf8

PROTO FourWheelsRobot [

field SFVec3f translation 0 0 0

field SFRotation rotation 0 0 1 0

field SFFloat bodyMass 1

]

{

Robot {

translation IS translation

rotation IS rotation

children [

DistanceSensor {

translation 0.105 -0.03 0.01

rotation -1 0 0 0

children [

Shape {

appearance PBRAppearance {

baseColor 0.333333 0 1

}

geometry Box {

size 0.01 0.01 0.01

}

}

]

name "ds_right"

}

DistanceSensor {

translation 0.105 0.03 0.01

rotation 1 0 0 0

children [

Shape {

appearance PBRAppearance {

```

baseColor 0.333333 0 1
}
geometry Box {
    size 0.01 0.01 0.01
}
}
]
name "ds_left"
}
HingeJoint {
    jointParameters HingeJointParameters {
        axis 0 1 0
        anchor 0.05 0.06 0
    }
    device [
        RotationalMotor {
            name "wheel1"
        }
    ]
    endPoint DEF WHEEL1 Solid {
        translation 0.05 0.06 0
        rotation 1 0 0 1.5708
        children [
            Shape {
                appearance PBRAppearance {
                    baseColor 0.333333 0.666667 0.498039
                }
            }
        ]
    }
}

```

```

    geometry Cylinder {
      height 0.02
      radius 0.04
    }
  }
]
boundingObject Cylinder {
  height 0.02
  radius 0.04
}
physics Physics {
}
}
}
HingeJoint {
  jointParameters HingeJointParameters {
    axis 0 1 0
    anchor -0.05 0.06 0
  }
  device [
    RotationalMotor {
      name "wheel2"
    }
  ]
}
endPoint Solid {
  translation -0.05 0.06 0
  rotation 1 0 0 1.5708
}

```

children [

Shape {

appearance PBRAppearance {

baseColor 0 0.666667 0

}

geometry Cylinder {

height 0.02

radius 0.04

}

}

]

name "WHEEL2"

boundingObject Cylinder {

height 0.1

radius 0.05

}

physics Physics {

}

}

}

HingeJoint {

jointParameters HingeJointParameters {

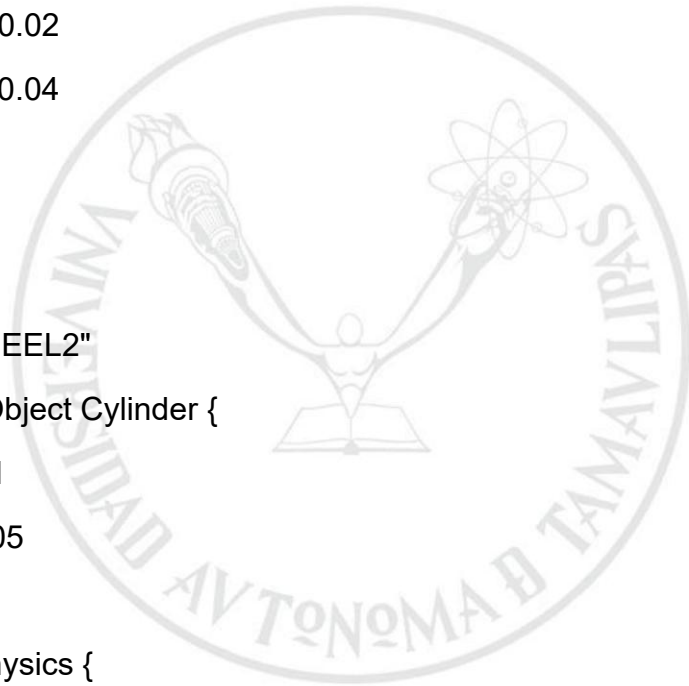
axis 0 1 0

anchor 0.05 -0.06 0

}

device [

RotationalMotor {



```

name "wheel3"

}

]

endPoint Solid {

translation 0.05 -0.06 0

rotation 1 0 0 1.5078

children [

Shape {

appearance PBRAppearance {

baseColor 0.333333 0.666667 0

}

geometry Cylinder {

height 0.02

radius 0.04

}

}

]

name "WHEEL3"

boundingObject Cylinder {

height 0.1

radius 0.05

}

physics Physics {

}

}

}

HingeJoint {

```

```
jointParameters HingeJointParameters {
```

```
axis 0 1 0
```

```
anchor -0.05 -0.06 0
```

```
}
```

```
device [
```

```
DEF wheel4 RotationalMotor {
```

```
name "wheel4"
```

```
}
```

```
]
```

```
endPoint Solid {
```

```
translation -0.05 -0.06 0
```

```
rotation 1 0 0 1.5708
```

```
children [
```

```
Shape {
```

```
appearance PBRAppearance {
```

```
baseColor 0.333333 0.666667 0
```

```
}
```

```
geometry Cylinder {
```

```
height 0.02
```

```
radius 0.04
```

```
}
```

```
}
```

```
]
```

```
name "WHEEL4"
```

```
boundingObject Cylinder {
```

```
height 0.1
```

```
radius 0.05
```

```

    }
    physics Physics {
    }
}
Solid {
    children [
        Shape {
            appearance PBRAppearance {
                baseColor 1 0 0
            }
            geometry Box {
                size 0.2 0.1 0.05
            }
        }
    ]
    name "BODY"
}
    VERDAD, BELLEZA, PROBIDAD
]
boundingObject USE BODY
physics Physics {
    density -1
    mass IS bodyMass
}
name "four_wheeled_robot"
controller "four_wheeled_collision_avoidance"
controllerArgs [

```

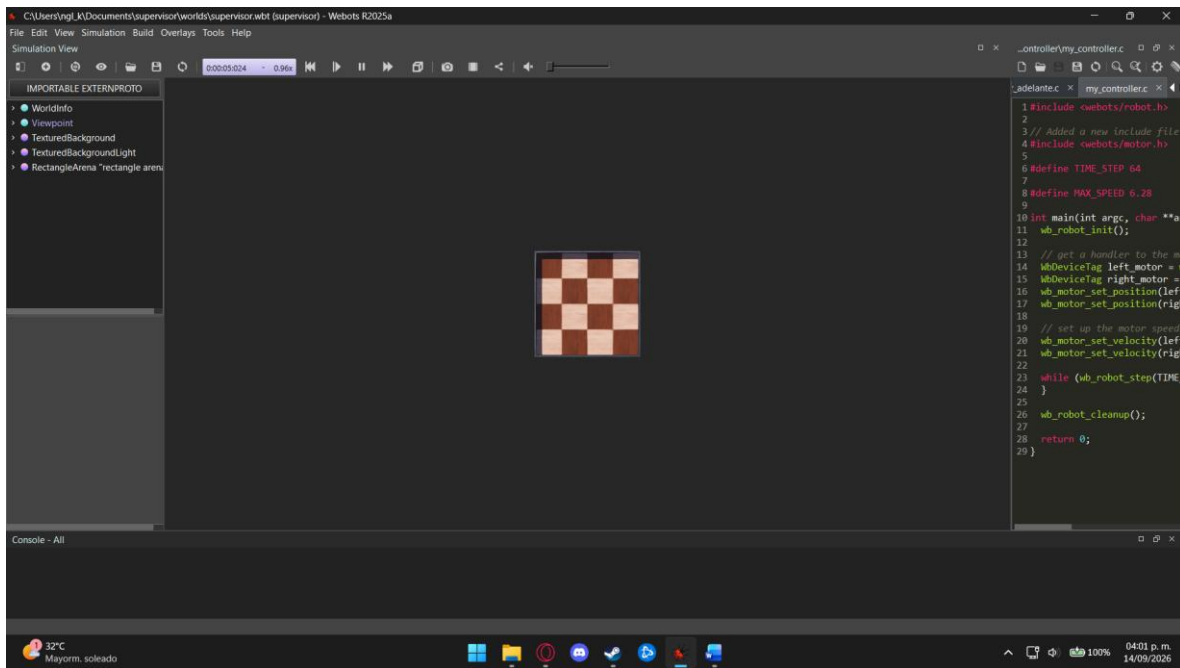
'''
]
}
}



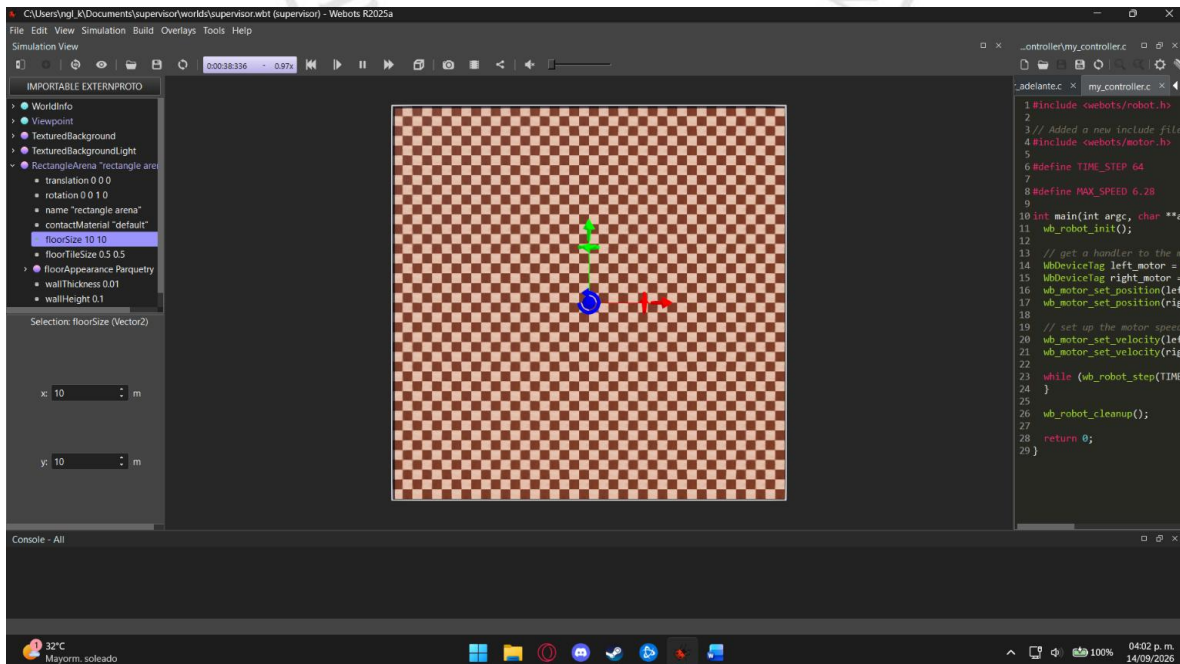
VERDAD, BELLEZA, PROBIDAD

Tutorial #8

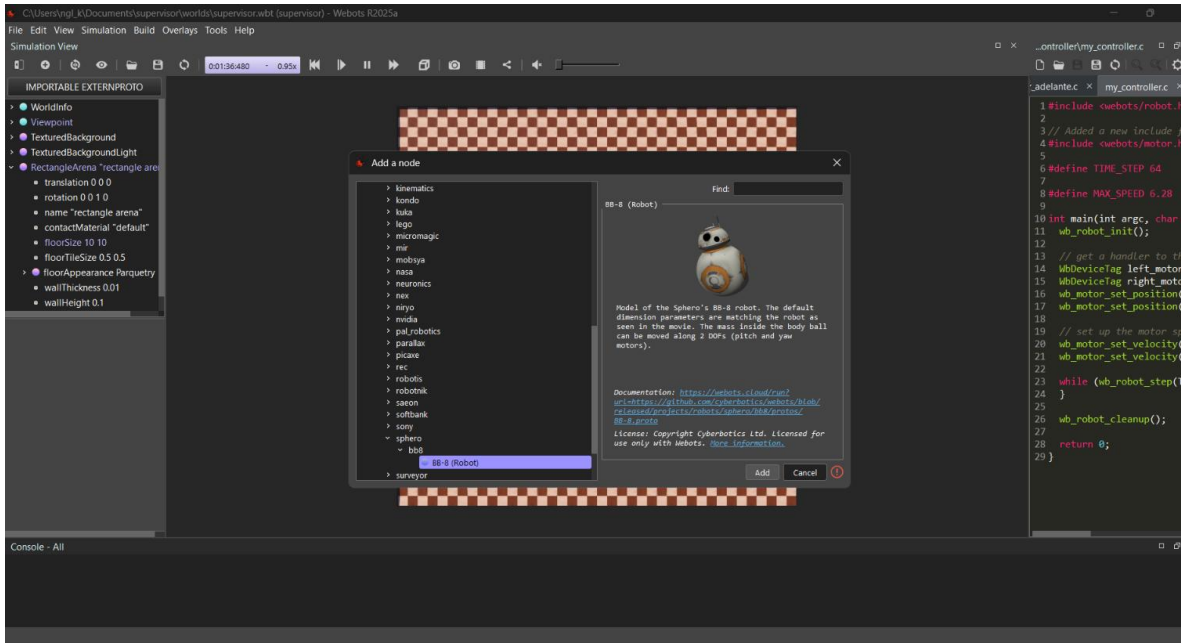
1. Creación del mundo



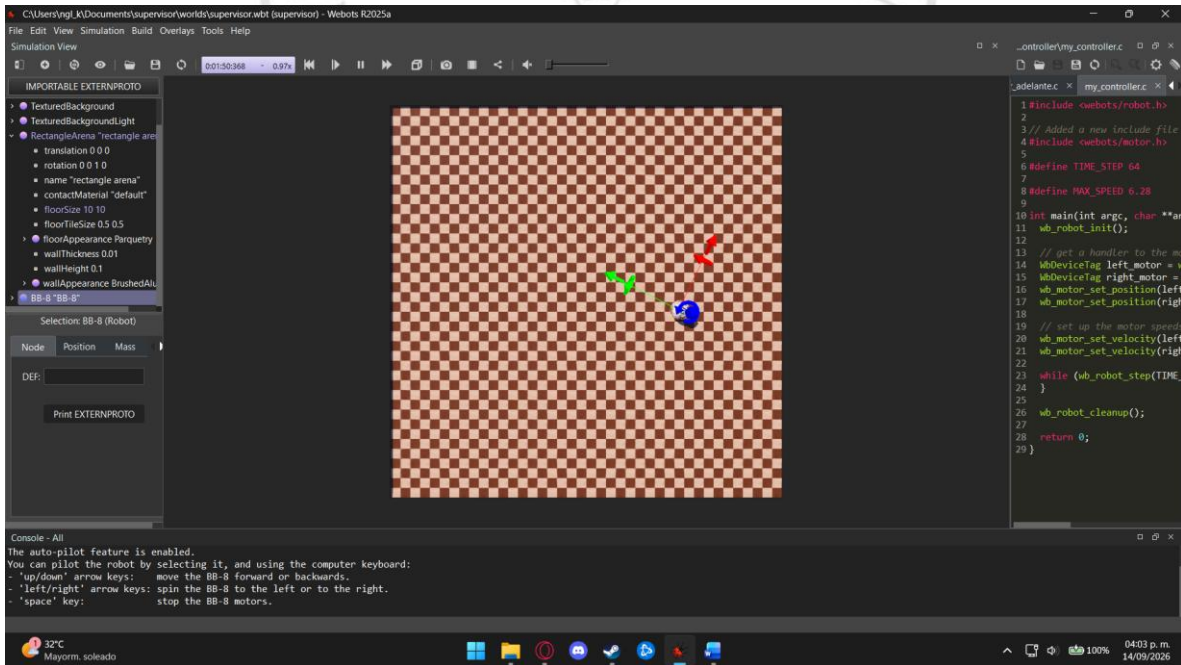
2.- Modificación del tamaño del suelo a 10 10



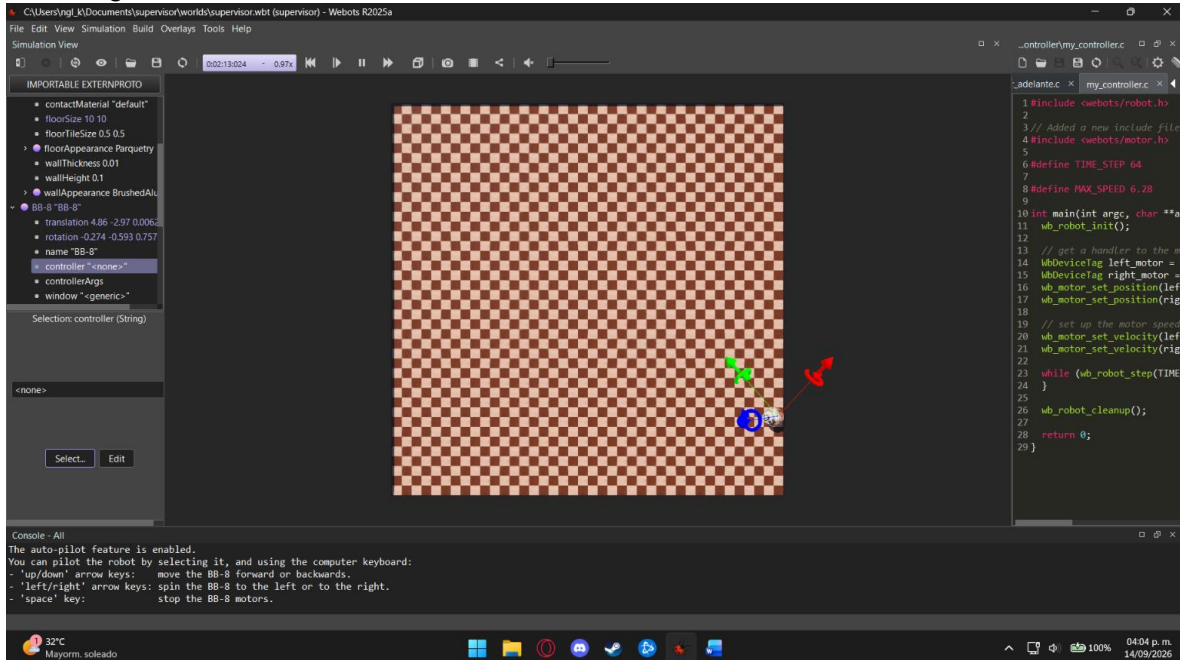
3.- Integración de robot BB-8 al mundo



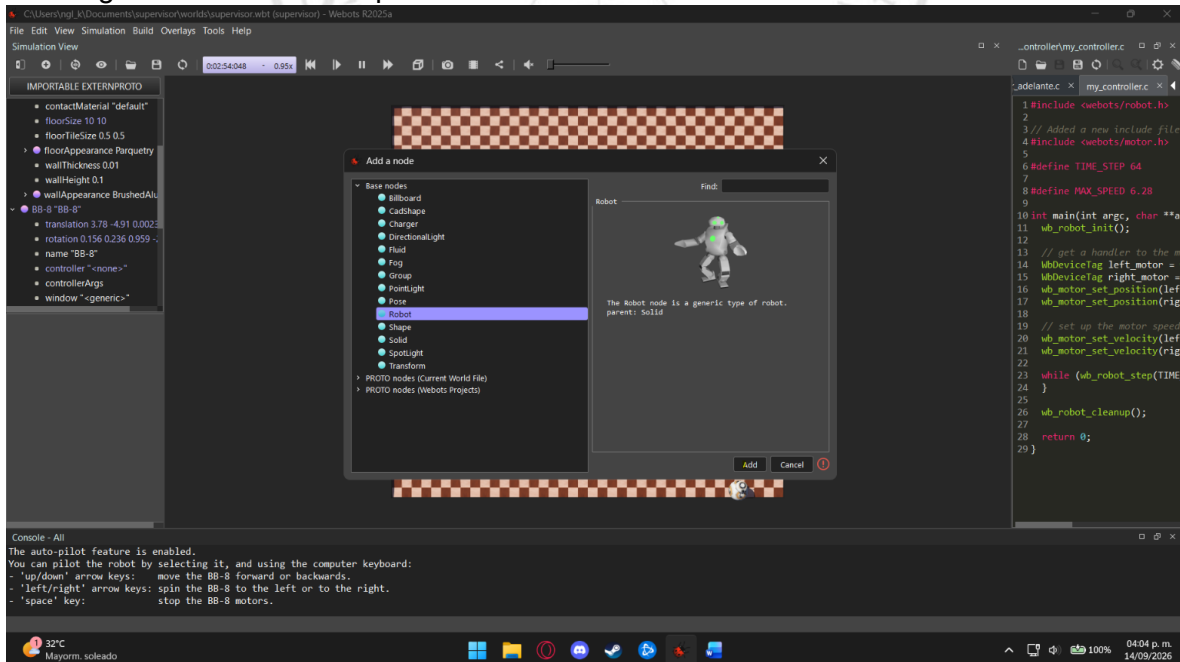
Robot BB-8



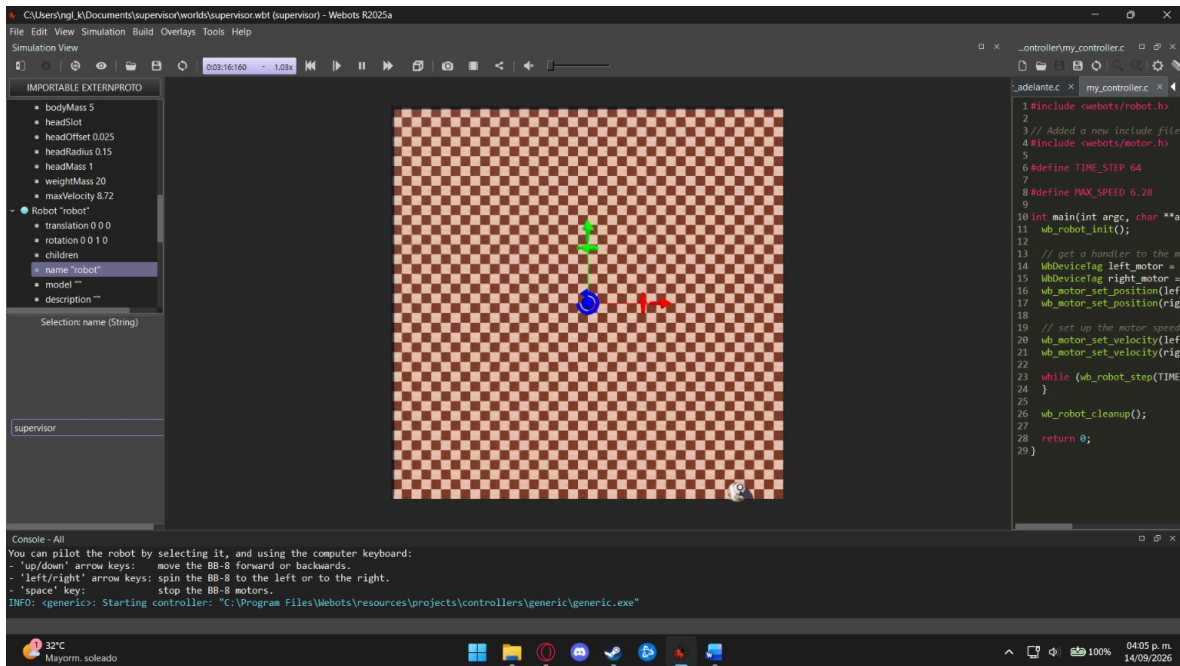
4.- Designación del controlador al Robot BB-8



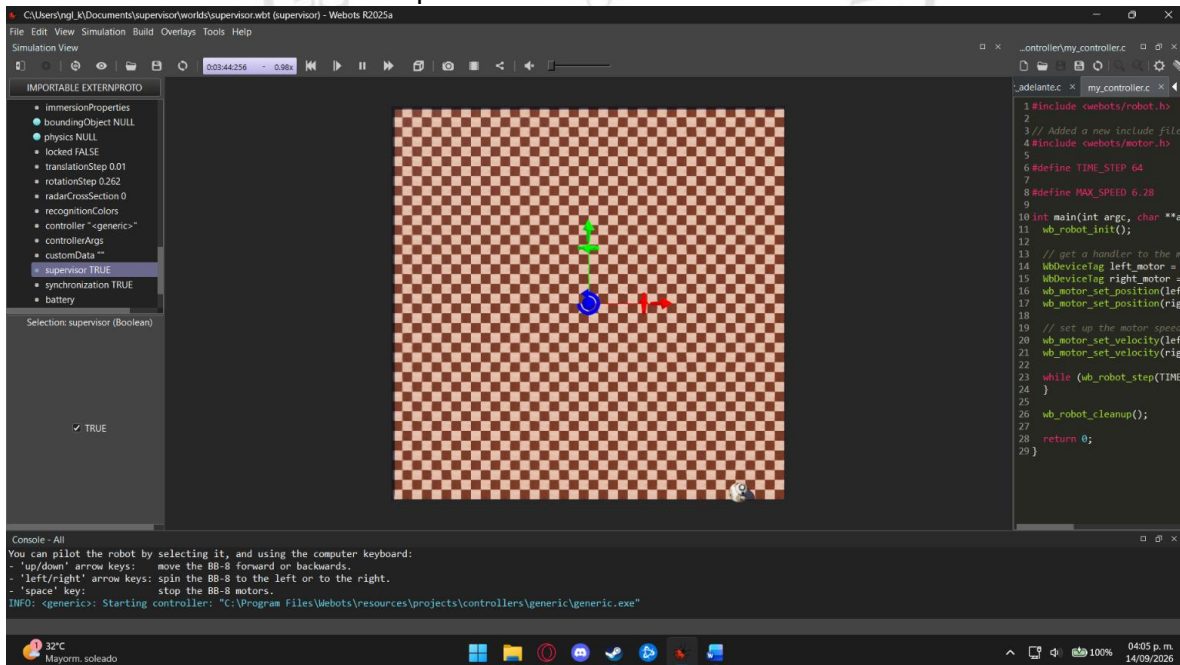
5.- Integración de Robot "Supervisor" al mundo



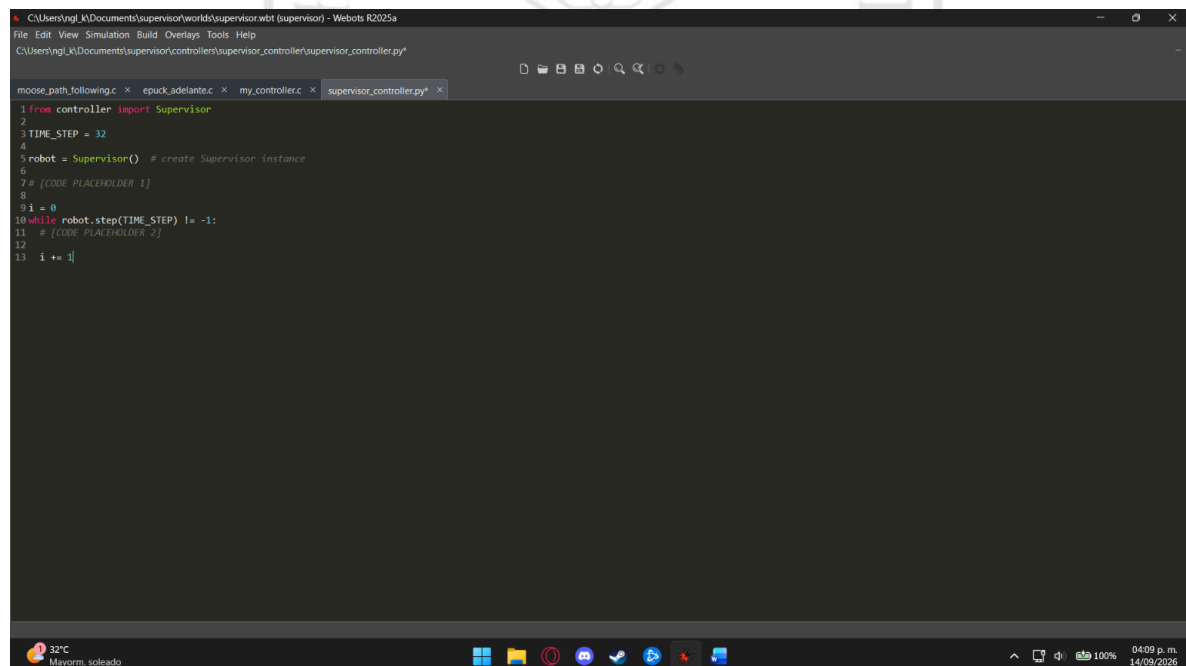
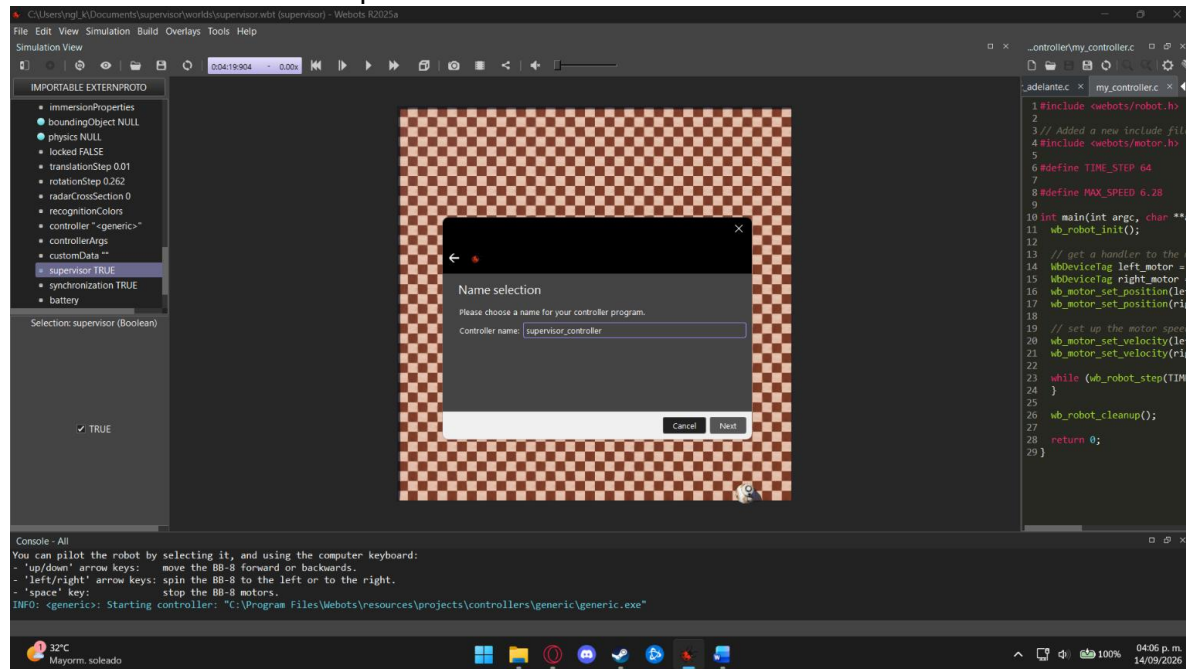
6.- Cambiar el nombre del Robot a "Supervisor"



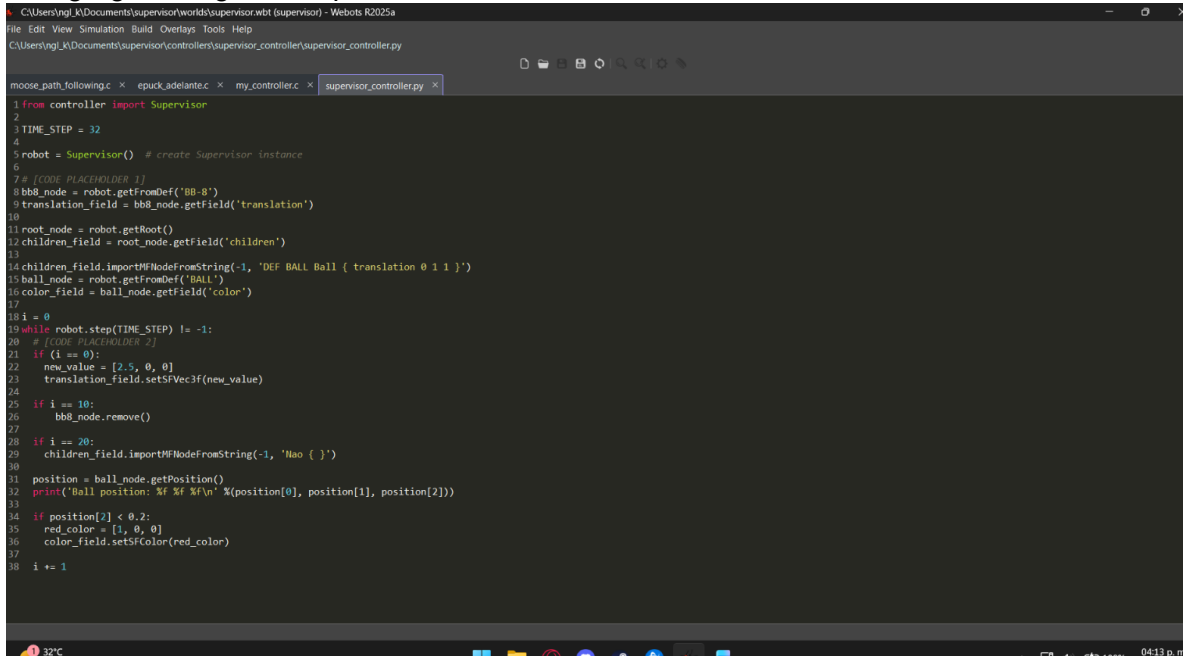
7.- Cambiar de False a True Supervisor



8.-Crear controlador del supervisor



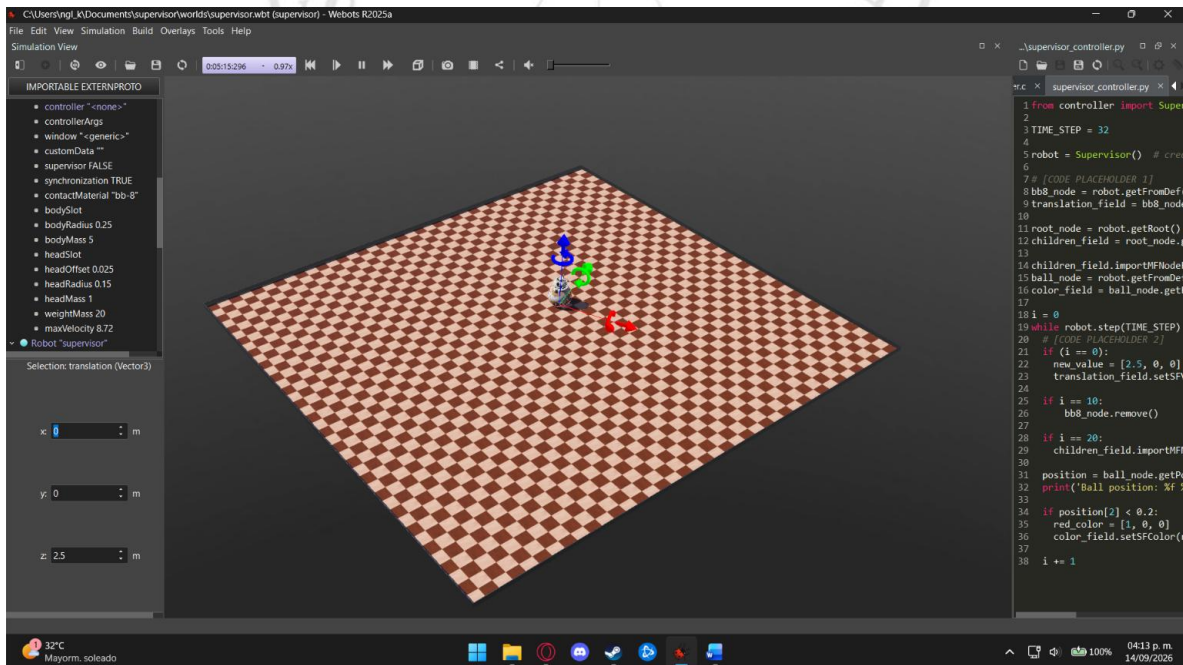
9.- Agregar código del supervisor al controlador anteriormente creado



```

1 from controller import Supervisor
2
3 TIME_STEP = 32
4
5 robot = Supervisor() # create Supervisor instance
6
7 # [CODE PLACEHOLDER 1]
8 bb8_node = robot.getFromDef('BB-8')
9 translation_field = bb8_node.getField('translation')
10
11 root_node = robot.getRoot()
12 children_field = root_node.getField('children')
13
14 children_field.importMFNodeFromString(-1, 'DEF BALL Ball { translation 0 1 1 }')
15 ball_node = robot.getFromDef('BALL')
16 color_field = ball_node.getField('color')
17
18 i = 0
19 while robot.step(TIME_STEP) != -1:
20     # [CODE PLACEHOLDER 2]
21     if (i == 0):
22         new_value = [2.5, 0, 0]
23         translation_field.setSFVec3f(new_value)
24
25     if i == 10:
26         bb8_node.remove()
27
28     if i == 20:
29         children_field.importMFNodeFromString(-1, 'Nao { }')
30
31     position = ball_node.getPosition()
32     print('Ball position: %f %f %f\n' % (position[0], position[1], position[2]))
33
34     if position[2] < 0.2:
35         red_color = [1, 0, 0]
36         color_field.setSFColor(red_color)
37
38     i += 1
  
```

Resultado del tutorial



Conclusión

Al completar la serie de ocho tutoriales de Webots, se logró comprender de manera progresiva el flujo completo de trabajo dentro del simulador: desde la creación de una simulación básica hasta la programación de controladores más complejos, la manipulación de propiedades físicas de los objetos, la creación de prototipos reutilizables y el uso del Supervisor para el control avanzado de la simulación. El desarrollo de estos tutoriales permitió reforzar conceptos fundamentales de robótica como la cinemática, el uso de sensores y actuadores, y el control autónomo de robots, sentando las bases necesarias para el desarrollo de proyectos más complejos en simulación robótica.



VERDAD, BELLEZA, PROBIDAD